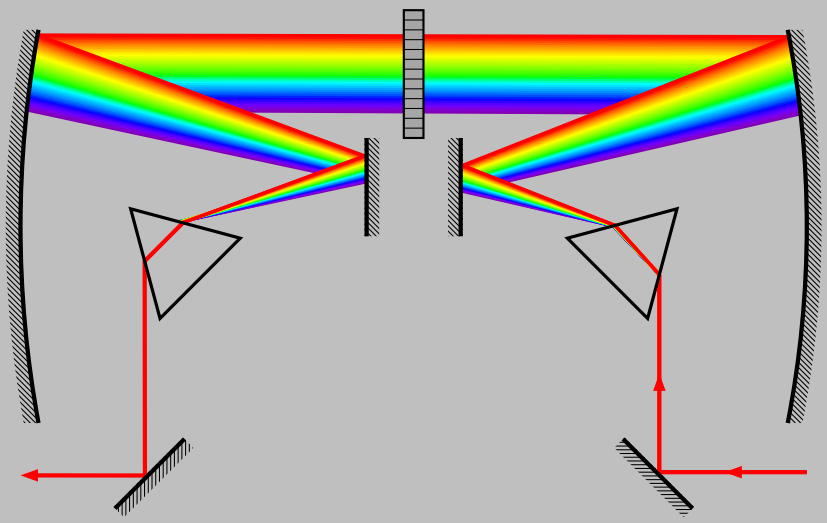
PSTricks

pst-optexp

Drawing optical experimental setups

v6.1

2022/02/26



Package author:
Christoph Bersch

Contents

1. Introduction **6**
- 1.1. About the package . . . 6
- 1.2. Requirements . . . 6
- 1.3. Distribution and installation . . . 6
- 1.4. License . . . 7
- 1.5. Backward compatibility . . . 7
- 1.6. Notation . . . 7
- 1.7. Acknowledgements . . . 8

2. Basic ideas **9**
- 2.1. The components . . . 9
- 2.2. Labels . . . 10
- 2.3. Free-ray connections . . . 10
- 2.4. Fiber connections . . . 12
- 2.5. Step-by-step examples . . . 13
  - 2.5.1. Galilean telescope . . . 13
  - 2.5.2. Fiber setup . . . 15
  - 2.5.3. Fiber setup (alternative) . . . 16
  - 2.5.4. Michelson-Interferometer . . . 17

3. General component parameters **19**
- 3.1. Labels . . . 19
- 3.2. Positioning . . . 21
- 3.3. Rotating and shifting . . . 22
- 3.4. Using psstyles . . . 24
- 3.5. Component appearance . . . 25

4. Free-ray components **27**
- 4.1. Lens . . . 27
- 4.2. Aspheric lens . . . 29
- 4.3. Optical plate . . . 31
- 4.4. Retardation plate . . . 31

4.5. Pinhole . . . 32
4.6. Box . . . 32
4.7. Arrow component . . . 33
4.8. Bar component . . . 34
4.9. Optical source . . . 35
4.10. Crystal . . . 36
4.11. Optical diode . . . 38
4.12. Dove prism . . . 38
4.13. Glan–Thompson prism . . . 39
4.14. Polarization . . . 40
4.15. Wedge . . . 41
4.16. Axicon . . . 41
4.17. Mirror . . . 42
4.18. Parabolic mirror . . . 45
4.19. Off-axis parabolic mirror . . . 46
4.20. Beamsplitter . . . 47
4.21. Optical grating . . . 47
4.22. Transmission grating . . . 49
4.23. Acousto-optic modulator . . . 50
4.24. Prism . . . 54
4.25. Right-angle prism . . . 55
4.26. Penta prism . . . 56

5. Fiber components 57

5.1. Optical fiber . . . 57
5.2. Optical amplifier . . . 58
5.3. Mach–Zehnder modulator . . . 59
5.4. Polarization controller . . . 59
5.5. Isolator . . . 59
5.6. Optical switch . . . 60
5.7. Fiber delay-line . . . 61
5.8. Polarizer . . . 61
5.9. Optical circulator . . . 61
5.10. Fiber coupler . . . 63
5.10.1. Variable node count . . . 65
5.10.2. Input and output nodes . . . 66
5.10.3. Reference nodes . . . 66
5.11. Fiber box . . . 67

6. Electrical components **69**
6.1. Coupler . . . 69
6.2. Synthesizer . . . 70
6.3. Mixer . . . 71

7. Hybrid components **73**
7.1. Optical filter . . . 73
7.2. Fiber collimator . . . 74
7.3. Detector . . . 75

8. Special nodes **77**
8.1. Component identifiers . . . 78
8.2. Reference nodes . . . 79
8.3. Center node . . . 80
8.4. Label node . . . 80
8.5. External nodes . . . 80
8.6. Interface nodes . . . 82
8.7. Rotation reference node . . . 83
8.8. Beam nodes . . . 84
8.9. Beam vector . . . 86
8.10. Node overview . . . 88

9. Connecting components **89**
9.1. Accessing components . . . 89
9.2. Drawing beams . . . 91
9.2.1. Raytracing . . . 91
9.2.2. Refractive index . . . 93
9.2.3. Initial conditions . . . 96
9.2.4. Internal beam path . . . 99
9.2.5. Connecting with nodes . . . 101
9.2.6. Automatic connections . . . 103
9.2.7. Beam appearance . . . 103
9.3. Drawing wide beams . . . 106
9.3.1. Beam appearance . . . 106
9.4. Error handling . . . 107
9.5. Custom beams . . . 109
9.6. Drawing fibers . . . 112
9.6.1. Fiber angles . . . 113
9.6.2. Fiber appearance . . . 117
9.7. Drawing wires . . . 118

9.8. Automatic fiber and wire connections 120
9.8.1. Appearance of automatic connections 123
9.8.2. Known limitations 124
9.9. Layers 124

10. Custom components **126**
10.1. Customized versions of existing components 128
10.2. Defining new objects 129
10.2.1. The component drawing 130
10.2.2. Support for raytracing 131
10.2.3. Support for external nodes 132
10.2.4. Examples of custom components 133

11. Examples **134**

12. Additional information **161**
12.1. Internal component structure 161
12.2. Overview of special nodes 163
12.2.1. External and rotation reference nodes 163
12.2.2. Interface nodes 165
12.3. Backward compatibility 167
12.3.1. Version 4.3a 167
12.3.2. Version 4.3 167
12.3.3. Version 4.2 168
12.3.4. Version 4.0 168
12.3.5. Version 3.3 168
12.3.6. Version 3.0 170

Documentation index **173**

A. Revision history **196**

1. Introduction

1.1. About the package

The package `pst-optexp` is a collection of optical components that facilitate easy sketching of optical experimental setups. A lot of different free-ray and fiber components are provided, which alignment, positioning and labelling can be achieved in very simple and flexible ways. The components can be connected with fibers or beams, realistic raytraced beam paths are also possible.

1.2. Requirements

`pst-optexp` requires LaTeX and recent versions of `pstricks` ($\geq$ 2.53), `pst-plot` ($\geq$ 1.61), `pst-node`, `multido`, `pstricks-add`, `pst-eucl`, `pst-intersect` ($\geq$ 0.4) and `environ`.

All PSTricks package rely heavily on the Postscript language so that the typical workflow involves `latex`, `dvips`, and `ps2pdf`. Of course there are several alternative ways to compile your documents.[1]

1.3. Distribution and installation

This package is available on CTAN[2] and is included in TeXLive and MiKTeX.

The `pst-optexp` package consists of the two main files `pst-optexp.ins` and `pst-optexp.dtx`. By running `tex pst-optexp.ins` the following derived files are generated:

- `pst-optexp.pro`: the Postscript prolog file

[1] http://tug.org/PSTricks/main.cgi?file=pdf/pdfoutput
[2] http://mirror.ctan.org/help/Catalogue/entries/pst-optexp.html

- `pst-optexp.sty`: the LaTeX style file

Save the files in a directory which is part of your local TeX tree.

Do not forget to run `texhash` to update this tree. For MiKTeX users, do not forget to update the file name database (FNDB).

For more detailed information see the documentation of your personal LaTeX distribution on installing packages to your local TeX system.

1.4. License

Permission is granted to copy, distribute and/or modify this software under the terms of the LaTeX Project Public License, version 1.3c.[3] This package is author-maintained.

1.5. Backward compatibility

Version 3.0 introduced a lot of advanced features which made it too difficult to maintain full backward compatibility with version 2.1. Especially the `\drawbeam` macro was broken by design, so that it was best to drop the old flaw as early as possible.

You find a more detailled list of changes and information for migration of old documents in Sec. 12.3.

1.6. Notation

Often the parameter types are self-explanatory, but in some cases a clear distinction is needed, e.g. between ⟨*num*⟩ and ⟨*psnum*⟩. Tab. 1.1 explains some commonly used types.

[3] http://www.latex-project.org/lppl.txt

Name	description
num	float number
psnum	Postscript code which evaluates to a number
int	integer number
dimen	dimension
psstyle	custom graphics parameter configuration defined with `\newpsstyle`
refpoint	Like the reference point of `\rput`, can be any comination of `c` (center), `t` (top), `b` (bottom), `l` (left), and `r` (right).

Table 1.1.: Parameter types used in the parameter reference.

1.7. Acknowledgements

I thank all the people of the PSTricks mailinglist for the continuous help, especially Herbert Voß. Thanks also to various package authors from which I learned and adopted code for this package: David Carlisle, Florent Chervet, Enrico Gregorio, Rolf Niepraschk and Heiko Oberdiek. Christine Römer convinced me with her article in the german DTK[4] to provide a german translation of the documentation. The documentation style is a mixture of the `pst-doc` class (Herbert Voß) and the `ltxdockit` package for the `biblatex` documentation (Philipp Lehman).

[4] Pakete in Deutsch dokumentieren. in Die TeXnische Komödie. Heft 2/2011, S. 28-35.

2. Basic ideas

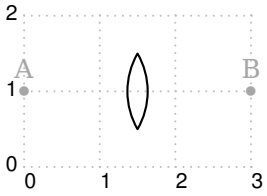
This chapter shows the basic ideas and concepts of this package. Starting with simple examples, basic functionality such as alignment, positioning (2.1) and labelling (2.2) of components is presented. Following, the connection of components with beam rays (2.3) or fiber (2.4) is demonstrated.

In Sec. 2.5 several propositions for preparing large experimental sketches are illustrated step by step. For a complete reference of all macros and their parameters, see Sec. 3–10.

A drawing consists of components, which can be positioned, shifted and rotated based on their reference nodes, and may have an optional label. Then, the components are connected with beam rays or fibers.

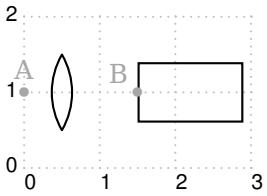
2.1. The components

A component is positioned according to the nodes which are used for its definition—the reference nodes. In the following examples, the lens is placed centered between nodes $\langle A \rangle$ and $\langle B \rangle$:




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnodes(0,1){A}(3,1){B}
  \lens(A)(B)
\end{pspicture}
```

This position can be changed with a number of parameters, such as `abspos` (see Sec. 3.2 and Sec. 3.3). In the next example, the lens is positioned `0.5` units apart from node $\langle A \rangle$, the box is placed at the end of the reference line $\overline{AB}$.

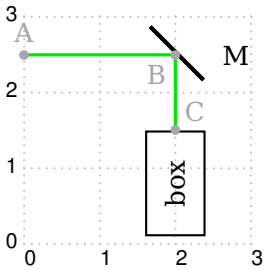



```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(1.5,1){B}
  \lens[abspos=0.5](A)(B)
  \optbox[position=end](A)(B)
\end{pspicture}
```

The components can be divided in two categories: free-ray and fiber components. These differ only in their connection possibilities: Free-ray components can have fiber or free-ray connections, whereas fiber components support fiber connections only.

2.2. Labels

Every component can have an optional label, which is positioned relative to the component.



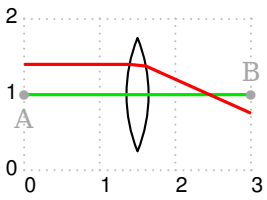

```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(0,2.5){A}\pnode(2,2.5){B}\pnode(2,1.5){C}
  \mirror[labelangle=-45](A)(B)(C){M}
  \optbox[position=start, labeloffset=0, labelref=relative](C)(B){box}
  \drawbeam(A){1}{2}
\end{pspicture}
```

2.3. Free-ray connections

Free-ray components can be connected with beam rays, which path can be calculated by raytracing. Besides the realistic tracing possibilities, there are many ways to manipulate the beam paths because you are dealing with a package for *sketching* experimental setups.

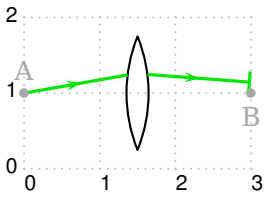
In the simplest case a single line is drawn, which optical path is determined by the refractive index `n` and Snell's law.

The example shows two beams which hit the lens with different initial conditions. The red beam starts at a distance of `0.2` from the optical axis (`beampos`) and is deflected by the lens accordingly.



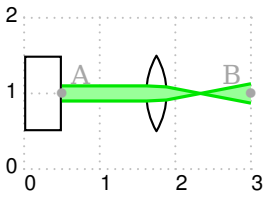

```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(3,1){B}
  \lens[n=2, lensradius=2 2, lensheight=1.5](A)(B)
  \drawbeam(A){1}(B)
  \drawbeam[beampos=0.4, linecolor=red](A){1}(B)
\end{pspicture}
```

The beams can be decorated with arbitrary dashs and arrows.



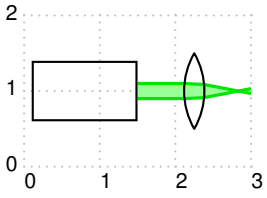

```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0,1){A}\pnode(3,1){B}
  \lens[n=2, lens=2 2 1.5](A)(B)
  \drawbeam[beamangle=10, ArrowInside=->, arrows=-|, ↩
      beaminside=false](A){1}(B)
\end{pspicture}
```

You can also draw wide beams. The marginal rays can be changed with `linestyle` etc. and the filling with `fillstyle` etc.




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(0.5,1){A}\pnode(3,1){B}
  \optbox[position=start, optboxsize=0.5 1](A)(B)
  \lens[n=2](A)(B)
  \drawwidebeam[beamwidth=0.2, fillstyle=solid, ↩
      fillcolor=green!40]{1-2}(B)
\end{pspicture}
```

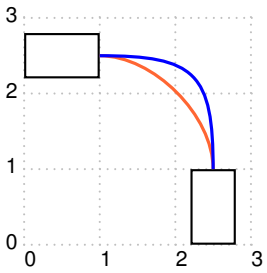
The beams can also lay behind the components. For this you must enclose all respective components and rays in a `optexp` environment (compare the previous example with the following one).




```
\begin{pspicture}[showgrid](0,-0.3)(3,2.3)
  \pnode(1.5,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox[position=start](A)(B)
    \lens[n=2](A)(B)
    \drawwidebeam[beamwidth=0.2, fillstyle=solid, ↩
        fillcolor=green!40]{1}{2}(B)
  \end{optexp}
\end{pspicture}
```

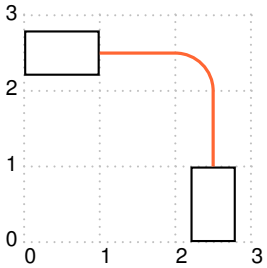
2.4. Fiber connections

Fibers (\drawfiber) are another possibility of connecting components. Here, you have several way of automatically adapting the fiber connection to the component alignment.




```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(1,2.5){A}\pnode(2.5,1){B}
  \psset{optboxsize=1 0.6}
  \optbox[position=start](A)(B|A)
  \optbox[position=end](B|A)(B)
  \drawfiber{1}{2}
  \drawfiber[linecolor=blue, ncurv=1.2]{1}{2}
\end{pspicture}
```

Usually, the fibers are drawn as \nccurve connections, other node connections are possible as well (fiberstyle).



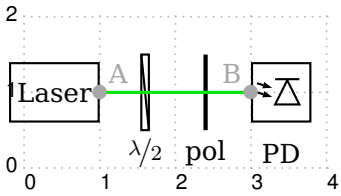

```
\begin{pspicture}[showgrid](0,-0.3)(3,3.3)
  \pnode(1,2.5){A}\pnode(2.5,1){B}
  \psset{optboxsize=1 0.6}
  \optbox[position=start](A)(B|A)
  \optbox[position=end](B|A)(B)
  \drawfiber[fiberstyle=angle, linearc=0.5]{1}{2}
\end{pspicture}
```

2.5. Step-by-step examples

In this first example, all components are distributed on a single line and connected with a beam.

First we define the start node ⟨A⟩ and the stop node ⟨B⟩. The first \optbox is positioned at the start, the wave plate and the polarizer are positioned between the nodes. The parameter position expects a value between 0 and 1 which is relative to the length of the reference line, or start/end. The detector is always positioned at the end of its reference line.

Finally, all components are connected with a simple beam ray.



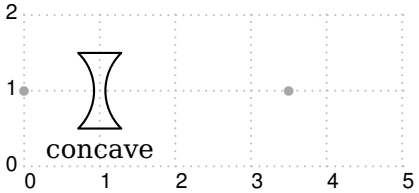

```
\begin{pspicture}[showgrid](4,2)
  \pnode(1,1){A}\pnode(3,1){B}
  \optbox[position=start, optboxwidth=1.2,
      labeloffset=0](A)(B){Laser}
  \optretplate[position=0.3](A)(B){$\nicefrac{\lambda}{2}$}
  \optplate[position=0.7](A)(B){pol}
  \optdetector[dettype=diode](A)(B){PD}
  \drawbeam{1}{2}{3}{4}
\end{pspicture}
```

2.5.1. Galilean telescope

This example describes a simple setup with a concave and a convex lens and the raytracing through them.

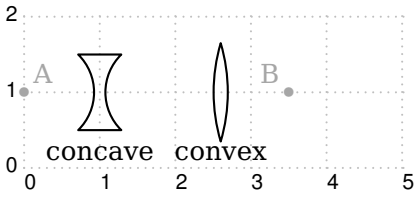
First, we define the reference nodes ⟨A⟩ and ⟨B⟩ which are used for alignment of the components. For better overview, these nodes are highlighted and a grid is drawn.

The first lens should have concave interfaces. With lensradius the interface radii are set to negative values, which correspond to concave curvatures. With abspos the lens is positioned one unit from ⟨A⟩ node.



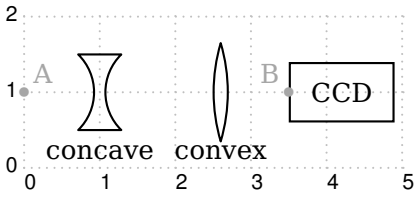

```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
\end{pspicture}
```

The second lens has a convex curvature (lensradius is positive) and is positioned 2.6 units apart from ⟨A⟩. Parameter lensheight sets the total height of the lens.



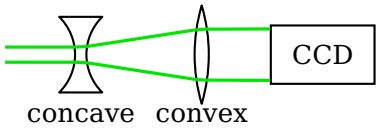

```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
\end{pspicture}
```

The beam rays are supposed to end at the CCD camera which is drawn as a \optbox. With position=end the box is positioned at the end of the reference line from ⟨A⟩ to ⟨B⟩. The label is put at the center of the component with labeloffset.




```
\begin{pspicture}[showgrid](0,-0.3)(5,2.3)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
\end{pspicture}
```

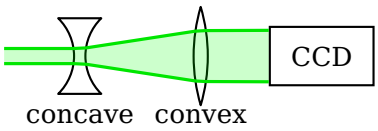
Finally, a wide beam with an initial beam width of 0.2 without divergence is sent through the components. The beam path is calculated by raytracing. The lens parameters in this example were set such, that for a collimated input beam also the output beam is collimated.




```
\begin{pspicture}(5,2)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
  \drawwidebeam[beamwidth=0.2](A){1}{2}{3}
\end{pspicture}
```

The appearance of the beam can be changed via the Beam style, and may have e.g. a semitransparent fill pattern.

Ex. 2.1



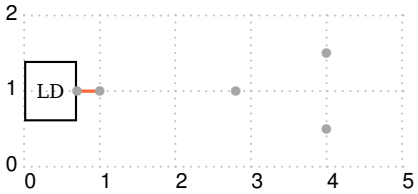

```
\begin{pspicture}(5,2)
  \pnode(0,1){A}\pnode(3.5,1){B}
  \lens[lensradius=-0.7,
        abspos=1](A)(B){concave}
  \lens[lensradius=2.3, lensheight=1.3,
        abspos=2.6](A)(B){convex}
  \optbox[position=end, labeloffset=0](A)(B){CCD}
  \addtopsstyle{Beam}{fillstyle=solid,
    fillcolor=green, opacity=0.2}
  \drawwidebeam[beamwidth=0.2](A){1}{2}{3}
\end{pspicture}
```

2.5.2. Fiber setup

This example shows how to setup a fiber drawing with automatic fiber connections.

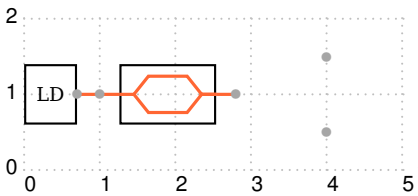
Again, the reference nodes are shown for better orientation. In contrast to the previous example, here, we need more reference nodes because the components are connected directly with them.

The laser is set at the start. The label offset is set with the `label` parameter, which would allow setting all label parameters at once. A `\optbox` is originally a free-ray component, so that for it the fiber connection must be enabled with `fiber`. The width of all `\optbox` is changed to `0.7`.



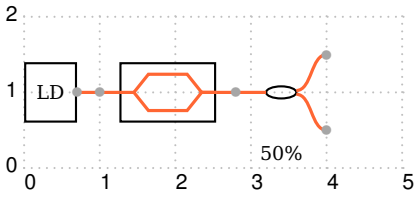

```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
\end{pspicture}
```

The Mach–Zehner modulator is drawn without label and connected automatically with fibers to its reference nodes.




```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
\end{pspicture}
```

Then, we draw a fiber coupler with one input and two output fibers. The coupler is positioned between the input node and the center of the two output nodes.

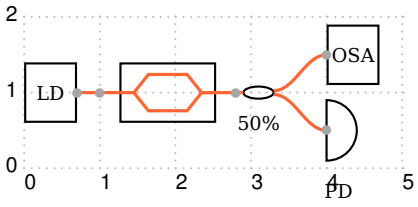



```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
  \wdmssplitter(C)(D)(E){50\%}
\end{pspicture}
```

At the ends of the coupler we put an optical spectrum analyzer (OSA) and a power meter (PD). The `\optdetector` is set by default at the end of the reference line, it has `position` preset to `end`. These two components are drawn without fibers, because their end nodes coincide with the two outputs of the coupler.

The notation `(A|E)` uses the x-coordinate of $\langle A \rangle$ and the y-coordinate of $\langle E \rangle$.

Ex. 2.2




```
\begin{pspicture}[showgrid](4.7,2)
  \pnodes(0.7,1){A}(1,1){B}(2.8,1){C}(4,1.5){D}(4,0.5){E}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize}
  \optbox[position=start, label=0, fiber](A)(B){LD}
  \optmzm(B)(C)
  \wdmsplitter[abspos=0.3, label=0.4](C)(D)(E){50\%}
  \optbox[position=end, label=0](A|D)(D){OSA}
  \optdetector(A|E)(E){PD}
\end{pspicture}
```

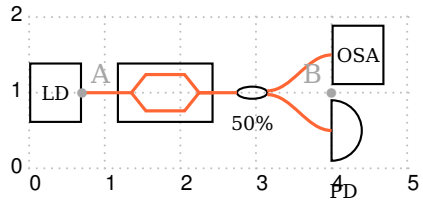
2.5.3. Fiber setup (alternative)

This example has the same result as the previous one (Ex. 2.5.2), but is constructed differently.

This time we do not use the automatic fiber connections, but draw all components first and connect them afterwards with fibers (`\drawfiber`). This requires less reference nodes, the fiber connections are suppressed globally with `fiber=none`.

The choice of the procedure depends on personal preferences, for loop setups, like for fiber lasers, it looks better if the fibers are drawn after the components with `\drawfiber`.

Ex. 2.3

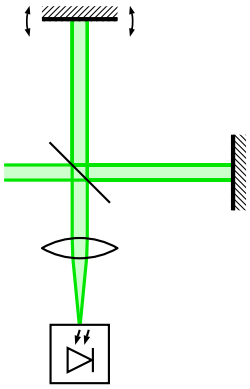



```
\begin{pspicture}[showgrid](4.9,2)
  \pnode(0.7,1){A}\pnode(4,1){B}
  \psset{optboxwidth=0.7, labelstyle=\scriptsize,
      fiber=none}
  \optbox[position=start, labeloffset=0](A)(B){LD}
  \optmzm[abspos=1.1](A)(B)
  \wdmsplitter[abspos=2.25, label=0.4](A)(B)(B){50\%}
  \optbox[position=end,label=0, compshift=0.5](A)(B){OSA}
  \optdetector[compshift=-0.5](A)(B){PD}
  \drawfiber{1-4}
  \drawfiber{3}{5}
\end{pspicture}
```

2.5.4. Michelson-Interferometer

In this example we use lenses, reflective elements (mirrors) and components which can be either transmittive or reflective (beamsplitter).

Ex. 2.4



```
1  \begin{pspicture}(3.2,5)
2    \pnodes(0,3){A}(1,3){BS}(3,3){M1}(1,5){M2}(1,1){PD}
3    \psset{mirrortype=extended, mirrordepth=0.2}
4    \begin{optexp}
5      \beamsplitter[bsstyle=plate, compname=BS](A)(BS)(PD)
6      \mirror[compname=M1](BS)(M1)(BS)
7      \mirror[compname=M2, variable](BS)(M2)(BS)
8      \lens[compname=L](BS)(PD)
9      \optdetector[compname=Det, dettype=diode](BS)(PD)
10     \addtopsstyle{Beam}{beamwidth=0.2, fillstyle=solid,
         fillcolor=green!20!white}
11     \drawwidebeam(A){BS}{M1}{BS}{M2}{BS}{L}{Det}
12   \end{optexp}
13 \end{pspicture}
```

We start with definition of the reference nodes (Line 2) and setting some basic parameters (Line 3).

Following, all components are positioned (Line 5–9). The reflective ones need three reference nodes, the input node, the node where the reflective interface is situated, and the output node. Then, the appearance of the beam is set (Line 10), and the beam is drawn (Line 11).

In this example, all components have a name (`compname`), which can be used instead of the numerical IDs—automatically assigned, unique numbers—for the raytracing. This can simplify follwing the beam path.

All components and beams are enclosed in a `optexp` environment. Inside of this, all beams are drawn behind the components.

3. General component parameters

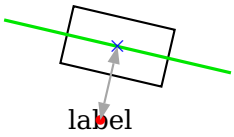
This chapter introduces generic parameters and concepts such as labels (3.1), positioning (3.2), rotating and shifting of components (3.3) and changing the appearance of components (3.4 and 3.5).

3.1. Labels

All components may have a label which position and alignment can be adjusted at will. The label is optional for every component. If the various parameters are not sufficient, you can access the label node—marked as red dot in some examples—directly (see also Sec. 8.4).

`labeloffset`=⟨*num*⟩ default: `0.8`

The offset of the label reference node (red dot) from the component center (blue cross).



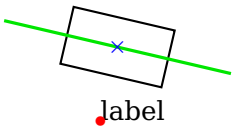

```
\begin{pspicture}(3,1.8)
  \optbox[beam, labeloffset=1](0,1.5)(3,0.8){label}
\end{pspicture}
```

`labelstyle`=⟨*macros*⟩

The text style that is used to typeset the label.

`labelalign`=⟨*refpoint*⟩ default: `c`

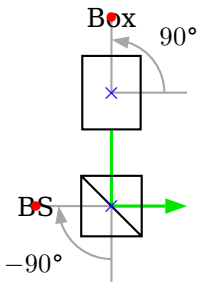
Defines the alignment of the label relative to the label node.




```
\begin{pspicture}(3,1.8)
  \psset{beam, labeloffset=1}
  \optbox[labelalign=bl](0,1.5)(3,0.8){label}
\end{pspicture}
```

`labelangle`=⟨*num*⟩ default: `0`

The rotation angle of the label node around the component center. The origin depends both on the component and the choosen reference system (see `labelref`).

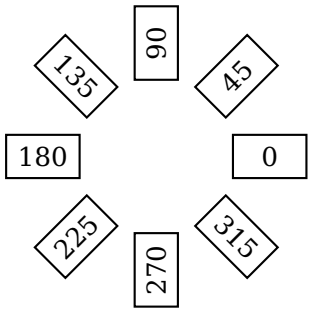



```
\begin{pspicture}(-0.3,0)(2,3.7)
  \psset{labeloffset=1, optboxwidth=1, arrowscale=1.5, arrowinset=0}
  \optbox[position=end, labelangle=90](1,1)(1,2){Box}
  \beamsplitter[labelangle=-90](1,2)(1,1)(2,1){BS}
  \drawbeam[arrows=->]{1}{2}(2,1)
\end{pspicture}
```

`labelref`=`relative`, `relgrav`, `global`, `absolute` default: `relgrav`

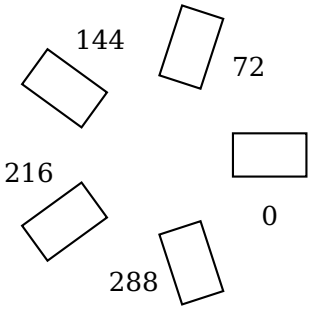
Set the reference coordinate system for the `labelangle` and the orientation of the label text.

`relative` The label is parallel to the component reference line and the actual angle is always projected to the range between $-90°$ and $+90°$.



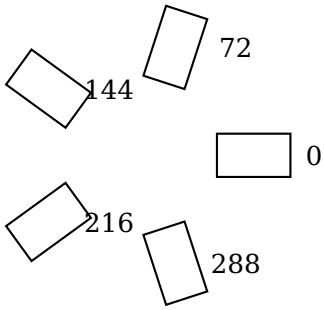

```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labeloffset=0,
      labelref=relative}
  \multido{\i=0+45}{8}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

`relgrav` The label itself is always horizontal, only the label node is rotated together with the component.




```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labelref=relgrav}
  \multido{\i=0+72}{5}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

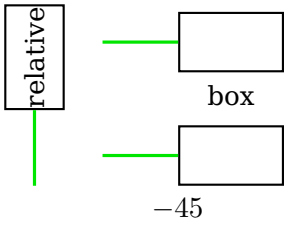
`global, absolute` The label is independent of the component alignment.




```
\begin{pspicture}(0,-2.2)(2,2.2)
  \psset{position=end, optboxsize=1 0.6, labelref=global}
  \multido{\i=0+72}{5}{%
    \optbox(0,0)(1;\i){\i}
  }
\end{pspicture}
```

`label`=⟨*offset*⟩[⟨*angle*⟩[⟨*refpoint*⟩[⟨*labelref*⟩]]]

Allows compact definition of several label parameters. It takes up to four space-separated arguments (`labeloffset`, `labelangle`, `labelalign`, and `labelref`). Unchanged intermediate arguments can be skipped with a dot.




```
\begin{pspicture}(0,0)(4,3)
  \psset{endbox, beam}
  \optbox[label=0.7](1.5,2.5)(2.5,2.5){box}
  \optbox[label=1 -45](1.5,1)(2.5,1){$-45$}
  \optbox[label=0 . . relative]%
        (0.6,0.6)(0.6,1.6){relative}
\end{pspicture}
```

`innerlabel=true`

This is an alias for `label=0 . c relative`

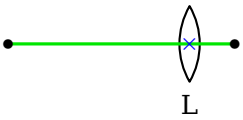
3.2. Positioning

All components except for the free-ray tripoles can be positioned between their two reference nodes `\oenodeRefA` and `\oenodeRefB`.

`position`=⟨*num*⟩, `start`, `end`

Controls the relative position of an object center (blue cross) between its two reference points (black dots), if the value is a number. In this case it is equivalent to the `npos` parameter of `\ncput`, i.e. a number in the range $[0, 1]$ (see `pst-node` documentation[1]).

[1] http://mirror.ctan.org/help/Catalogue/entries/pst-node.html

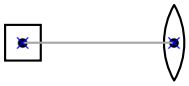



```
\begin{pspicture}(3,1.5)
  \lens[beam, position=0.8](0,1)(3,1){L}
\end{pspicture}
```

The values `start` and `end` place an object at the start or end of the reference line, respectively. The respective object interface is placed on the respective reference node (black dot) instead of the object center (blue cross) which would be the case for `position=0` or `position=1`. Achieving this with numerical values would me more difficult and unflexible, because the object size and shape must be taken into account.

```
\begin{pspicture}(3,2)
  \pnodes(0.5,1){A}(2.5,1){B}\psdot(A)\psdot(B)
  \optbox[optboxsize=0.5 0.5, position=start](A)(B)
  \lens[position=end](A)(B)
\end{pspicture}
```

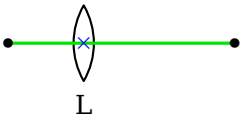
Here is for comparison the positioning with `position=0/1`.



```
\begin{pspicture}(3,2)
  \pnodes(0.5,1){A}(2.5,1){B}\psdot(A)\psdot(B)
  \optbox[optboxsize=0.5 0.5, position=0](A)(B)
  \lens[position=1](A)(B)
\end{pspicture}
```

`abspos`=⟨*num*⟩, `start`, `end`

Controls the absolute position of an object on the reference line. The values `start` and `end` are handled identically to `position`.




```
\begin{pspicture}(3,1.5)
  \lens[beam, abspos=1](0,1)(3,1){L}
\end{pspicture}
```

`endbox`=`true`, `false`

This parameter should not be used any more, it is equivalent to `position`=`end`.

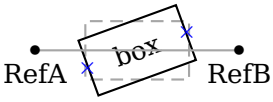
3.3. Rotating and shifting

All components can be shifted and rotated around different reference nodes.

Please note, that together with the components also their interface nodes (Sec. 8.6, marked as blue crosses in the following examples) are transformed, which may impact the connections. The transformed reference nodes are accessible as new nodes (Sec. 8.2).

`angle`=⟨*psnum*⟩ default: `0`

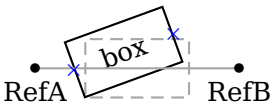
Rotate a component by an angle ⟨*num*⟩ (in degree). The original position is drawn gray dashed.




```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.7,1){B}
  \optbox[angle=20, innerlabel](A)(B){box}
\end{pspicture}
```

`rotateref`=⟨*refpoint*⟩ default: `c`

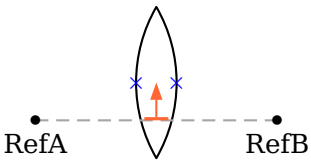
Set the reference point for the rotation of the component. Please see Sec. 8.7 for a detailed explanation and Sec. 12.2.1 for a list of the possible reference nodes of all components. The original position is drawn gray dashed.



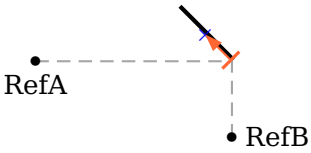

```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.7,1){B}
  \optbox[angle=20, rotateref=bl, innerlabel](A)(B){box}
\end{pspicture}
```

`compshift`=⟨*num*⟩ default: `0`

Shift a component perpendicular to its reference line (see Sec. 8.2). For reflective components it is shifted along the reflective interface.



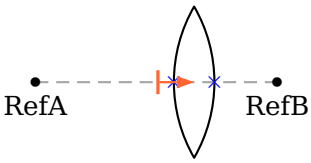

```
\begin{pspicture}(4,2)
  \pnodes(0.4, 0.5){A}(3.6,0.5){B}
  \lens[lens=2 2 2, compshift=0.5](A)(B)
\end{pspicture}
```



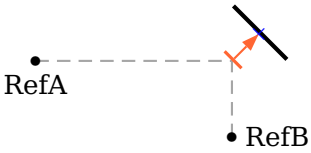

```
\begin{pspicture}(4,2)
  \pnodes(0.4,1.2){A}(3,0.2){B}
  \mirror[compshift=0.5](A)(B|A)(B)
\end{pspicture}
```

compoffset=⟨num⟩ default: 0

Shift a component perpendicular to its reference line (see Sec. 8.2). For reflective components it is shifted along the reflective interface. The displacement is vertical to a displacement with compshift. For an example see Ex. 11.



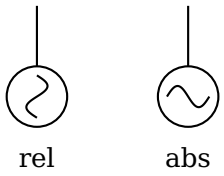

```
\begin{pspicture}(4,2)
  \pnodes(0.4, 1){A}(3.6, 1){B}
  \lens[lens=2 2 2, compoffset=0.5](A)(B)
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0.4,1.2){A}(3,0.2){B}
  \mirror[compoffset=0.5](A)(B|A)(B)
\end{pspicture}
```

innercompalign=rel, relative, abs, absolute

If this parameter is set to absolute, the "inner" component part is not rotated with the component. At the moment, this option is used by \optfilter, \elecsynthesizer, and \optdipole.



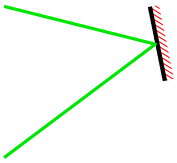

```
\begin{pspicture}(3,2)
\psset[optexp]{labelangle=-90}
\elecsynthesizer[innercompalign=rel](0.5,1.2)(0.5,2){rel}
\elecsynthesizer[innercompalign=abs](2.5,1.2)(2.5,2){abs}
\end{pspicture}
```

3.4. Using psstyles

Psstyles are custom graphics parameter configurations for PSTricks objects. The pst-optexp package makes extensive use of styles to facilitate flexible design of single, logical parts of components. This is in general not possible with the optional argument.

Existing styles can be extended with \addtopsstyle and overwritten with \newpsstyle.

As example we consider an extended mirror and want to change only the "extended" part which is controlable with the ExtendedMirror style:



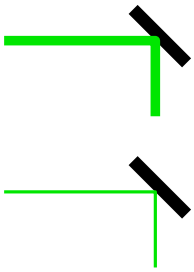
```
\begin{pspicture}(3,2)
  \addtopsstyle{ExtendedMirror}{hatchcolor=red}
  \mirror[mirrortype=extended, beam](0,0)(2,1.5)(0,2)
\end{pspicture}
```

The changeable parts of each component are listed in the respective reference.

3.5. Component appearance

OptComp ⟨*psstyle*⟩

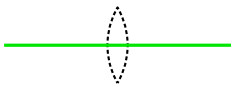
Affects the basic appearence of all optical components. Using standard graphics parameters like e.g. `linestyle` would change also the connections that are drawn together with the component (beam, fiber etc.). Use the key newOptComp and addtoOptComp to change the style for single components or for use with `\newpsobject` via the optional argument.



```
\begin{pspicture}(2.5,4)
  \psset{beam}
  % wrong, also beam width is changed
  \mirror[linewidth=3\pslinewidth](0,3)(2,3)(2,2)
  % correct result
  \addtopsstyle{OptComp}{linewidth=3\pslinewidth}
  \mirror(0,1)(2,1)(2,0)
\end{pspicture}
```

OptionalStyle ⟨*psstyle*⟩ default: `linestyle=dashed,dash=1.5pt 1pt`

This style is applied to components marked as optional in addition to OptComp.



```
\begin{pspicture}(3,1.5)
\lens[optional, beam](0,1)(3,1)
\end{pspicture}
```

OptionalStyle is applied after OptComp so that it can overwrite the general settings.

```
\begin{pspicture}(3,1.5)
\addtopsstyle{OptComp}{linewidth=3\pslinewidth, linecolor=red}
\newpsstyle{OptionalStyle}{linecolor=blue}
\lens(0,1)(2,1)
% linecolor overwritten, linewidth kept
\lens[optional](0,1)(3,1)
\end{pspicture}
```

VariableStyle ⟨*psstyle*⟩ default: `linewidth=0.8\pslinewidth, arrowinset=0, arrowscale=0.8, arrows=<->`

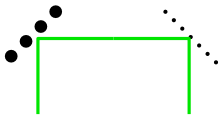
The style of the arrows of variable components (`\mirror` and `\optgrating`).

addtoOptComp=⟨*list*⟩

The `OptComp` style is extended by locally adding the parameters contained in ⟨*list*⟩. The ⟨*list*⟩ must be surrounded by curly braces.

newOptComp=⟨*list*⟩

Similar to `addtoOptComp`, but an existing `OptComp` style is overwritten with the new parameter set.



```
\begin{pspicture}(3,1.5)
  \psset{beam}
  \newpsstyle{OptComp}{linewidth=3\pslinewidth}
  \mirror[addtoOptComp={linestyle=dotted}](0.5,0)(0.5,1)(1.5,1)
  % overwrites the linewidth settings
  \mirror[newOptComp={linestyle=dotted}](1.5,1)(2.5,1)(2.5,0)
\end{pspicture}
```

optional=`true`, `false` default: `false`

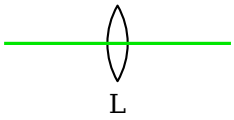
Mark an object as optional by applying the `OptionalStyle` style.

4. Free-ray components

This chapter describes all free-ray components and their options. Free-ray components differ from fiber components in that they provide transmittive or reflective interfaces which can be used to raytrace beams. They are divided in two types: some components like lenses need only two nodes for positioning (4.1–4.14), whereas e.g. mirrors require three nodes to determine the orientation (4.17–4.26).

4.1. Lens

`\lens[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

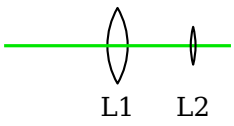



```
\begin{pspicture}(3,1.5)
  \lens[beam](0,1)(3,1){L}
\end{pspicture}
```

A lens is defined by its height and the radii of the input and output interfaces.

`lensheight=`⟨*num*⟩ default: 1

Set the height of the lens.



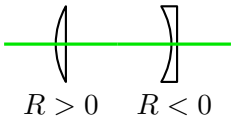

```
\begin{pspicture}(3,1.5)
  \lens[beam](0,1)(3,1){L1}
  \lens[lensheight=0.5](2,1)(3,1){L2}
\end{pspicture}
```

`lensradiusleft=`⟨*num*⟩ default: 1

Set the left radius of the lens. A positive ⟨*num*⟩ is for convex, a negative one for concave curvatures. Use zero for a plain surface.

`lensradiusright=`⟨*num*⟩ default: 1

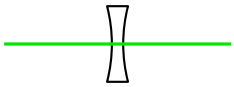
Same as `lensradiusleft` but for the right surface.




```
\begin{pspicture}(3,1.5)
  \psset{lensradiusright=0, beam}
  \lens[lensradiusleft=1](0,1)(1.5,1){$R > 0$}
  \lens[lensradiusleft=-1](1.5,1)(3,1){$R < 0$}
\end{pspicture}
```

lensradius=⟨*left*⟩[⟨*right*⟩]

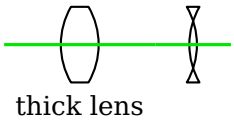
Set the left and right lens curvatures. If only a single value is given, it is used for both curvatures.



```
\begin{pspicture}(3,1.5)
  \lens[lensradius=-2, beam](0,1)(3,1)
\end{pspicture}
```

lenswidth=⟨*num*⟩ default: 0

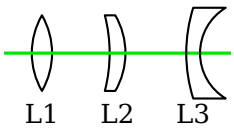
Usually only the height and the two radii are used to construct the lens, the width is calculated automatically. For lenswidth greater than zero, this width is used instead. This is only useful if you want to draw thick lenses and it can give ugly results if the value is choosen too small.




```
\begin{pspicture}(3,2)
  \lens[lenswidth=0.5, beam](0,1)(2,1){thick lens}
  \lens[lenswidth=0.1, beam](2,1)(3,1)
\end{pspicture}
```

lens=⟨*radiusleft*⟩[⟨*radiusright*⟩[⟨*height*⟩[⟨*width*⟩]]]

A convenience option to specify all lens parameters with a single option. Intermediate values can be skipped with a dot, values at the end can be omitted (e.g. `lens=. 1 1`).




```
\begin{pspicture}(3,1.5)
  \lens[lens=1](0,1)(1,1){L1}
  \lens[lensradius=-2 1, beam](0,1)(3,1){L2}
  \lens[lens=2 -0.7 1.2](2,1)(3,1){L3}
\end{pspicture}
```

thicklens=`true`, `false` default: `false`

This option was necessary until version 2.1 in order to draw a thick lens of width lenswidth instead of using the calculated lens width. This option is superfluous since version 3.0 because setting lenswidth to a positive value automatically draws a thick lens.

4.2. Aspheric lens

\asphericlens[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

asphereheight=⟨*num*⟩ default: 1

The lens height.

aspherewidth=⟨*num*⟩ default: 0

The lens center thickness. Is use only, if the given value is larger than the minimum calculated value.

asphereradiusleft=⟨*num*⟩ default: 1

Curvature radius of the left, aspheric surface.

asphereradiusright=⟨*num*⟩ default: 0

Curvature radius of the right surface. IF the value is zero, the surface is plane.

asphereconstant=⟨*num*⟩

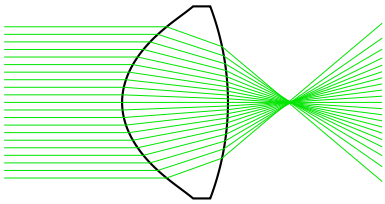
The conic constant k of the left surface.

asphererecoefficients=⟨A_4⟩ [⟨A_6⟩ [⟨A_8⟩ [⟨A_{10}⟩]]]

The aspheric terms of fourth to tenth order. One to four values can be specified.

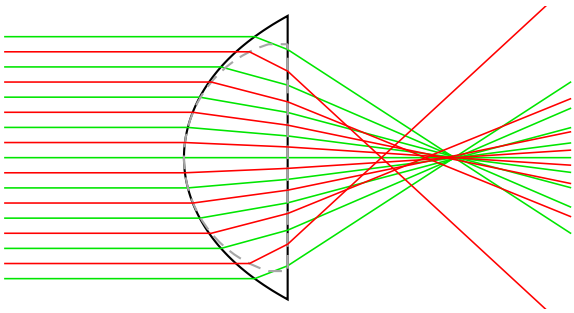
As example we use the parameters of a commercially available condensor lens[1]:

[1] http://www.thorlabs.de/thorproduct.cfm?partnumber=ACL25416U



```
\begin{pspicture}(0,-0.3)(5,2.3)
  \pnodes(0,1){A}(5,1){B}
  \asphericlens[%
    plotstyle=cspline,
    asphereheight=2.54,
    aspherewidth=1.4,
    asphereradiusleft=0.8818197,
    aspherecoefficients=8.682e-3 6.376e-3 2.4073e-2
        -1.719e-2,
    aspherеconstant=-0.9992,
    asphereradiusright=6.99995](A)(B)
  \multido{\r=-1+0.1}{21}{%
    \drawbeam[beampos=\r,
        linewidth=0.3\pslinewidth](A){}(B)}
\end{pspicture}
```

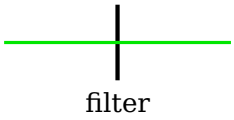
Another example illustrates the difference between an aspheric lens an a spheric lens with equal curvature:



```
\begin{pspicture*}(15,8)
  \pnodes(0,4){A}(15,4){B}
  \asphericlens[aspherеheight=7.5, asphereradiusleft=3.12848,
    aspherеconstant=-1.991145, aspherecoefficients=4.98e-3](A)(B)
  \multido{\r=-3.2+0.8}{9}{%
    \drawbeam[beampos=\r, linewidth=0.5\pslinewidth](A){}(B)}
  \lens[lens=3.12848 0 6 5.49, abspos=6.13, style=Refline, linestyle=dashed](A)(B)
  \multido{\r=-2.8+0.8}{8}{%
    \drawbeam[beampos=\r, linewidth=0.5\pslinewidth, linecolor=red](A){}(B)}%
\end{pspicture*}
```

4.3. Optical plate

\optplate[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.5)
  \optplate[beam](0,1)(3,1){filter}
\end{pspicture}
```

plateheight=⟨num⟩ default: 1

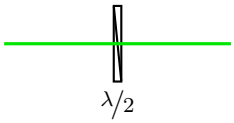
The height of the plate.

platelinewidth=⟨num⟩ or ⟨dimen⟩ default: 2\pslinewidth

The linewidth of the plate. This could be defined also with the generic linewidth option. But this parameter allows setting globally the linewidth of all plates.

4.4. Retardation plate

\optretplate[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.5)
  \optretplate[beam](0,1)(3,1){$\nicefrac{\lambda}{2}$}
\end{pspicture}
```

plateheight=⟨num⟩ default: 1

The height of the plate.

platewidth=⟨num⟩ default: 0.1

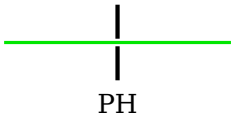
The width of the plate.

platesize=⟨width⟩ ⟨height⟩

The width and height of the plate, is equivalent to calling both platewidth and plateheight.

4.5. Pinhole

\pinhole[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \pinhole[beam](0,1)(3,1){PH}
\end{pspicture}
```

outerheight=⟨*num*⟩ default: 1

The total height of the pinhole.

innerheight=⟨*num*⟩ default: 0.1

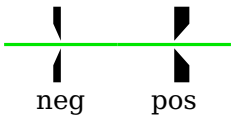
The height of the hole.

phlinewidth=⟨*num*⟩ or ⟨*dimen*⟩ default: 2\pslinewidth

The linewidth of the pinhole. This could be defined also with the generic linewidth option. But this parameter allows setting globally the linewidth of all pinholes.

phwidth=⟨*num*⟩ default: 0

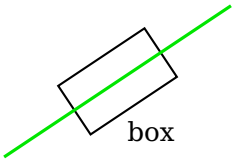
The pinhole is drawn in a more plastic style if phwidth is not zero, in this case the phlinewidth is ignored. For negative values the shape is mirrored.




```
\begin{pspicture}(3,1.5)
  \pinhole[beam, phwidth=-0.1](0,1)(1.5,1){neg}
  \pinhole[beam, phwidth=0.2](1.5,1)(3,1){pos}
\end{pspicture}
```

4.6. Box

\optbox[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optbox[beam](0,0)(3,2){box}
\end{pspicture}
```

optboxwidth=⟨*num*⟩ default: 1.4

The width of the box.

optboxheight=⟨num⟩ default: 0.8

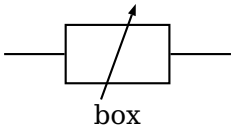
The height of the box.

optboxsize=⟨width⟩ ⟨height⟩

The width and height of the box, is equivalent to calling both optboxwidth and optboxheight.

4.7. Arrow component

\optarrowcomp[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.7)
  \optarrowcomp[wire](0,1)(3,1){box}
\end{pspicture}
```

A general purpose component, similar to \fiberdelayline, but much more flexible.

arrowcompwidth=⟨num⟩ default: 1.4

The width of the box.

arrowcompheight=⟨num⟩ default: 0.8

The height of the Komponente.

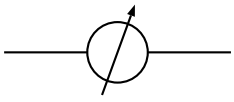
arrowcompsize=⟨size⟩ or ⟨width⟩ ⟨height⟩

The width and height of the component, is equivalent to calling both arrowcompwidth and arrowcompheight.

arrowcompangle=⟨num⟩ default: 70

The angle of the arrow with respect to the x-axis.

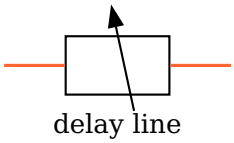
arrowcompshape=rectangle, circle default: rectangle



```
\begin{pspicture}(3,1)
  \optarrowcomp[arrowcompshape=circle, wire](0,0.5)(3,0.5)
\end{pspicture}
```

ArrowCompStyle ⟨*psstyle*⟩ default: `arrowinset=0, arrows=->`

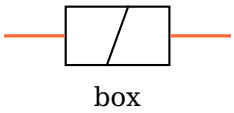
The style of the arrow. This can be especially useful to adapt the length and orientation of the arrow.




```
\begin{pspicture}(3,1.5)
  \addtopsstyle{ArrowCompStyle}{xunit=-1, arrowscale=2}
  \optarrowcomp[fiber](0,1)(3,1){delay line}
\end{pspicture}
```

4.8. Bar component

\optbarcomp[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \optbarcomp[fiber](0,1)(3,1){box}
\end{pspicture}
```

A general purpose component, similar to \optfiberpolarizer, but much more flexible.

barcompwidth=⟨*num*⟩ default: `1.4`

The width of the box.

barcompheight=⟨*num*⟩ default: `0.8`

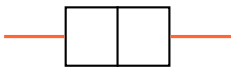
The height of the box.

barcompsize=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩

The width and height of the box, is equivalent to calling both barcompwidth and barcompheight.

barcompangle=⟨*num*⟩ default: `70`

The angle of the bar with respect to the x-axis.



```
\begin{pspicture}(3,1)
  \optbarcomp[barcompangle=90, fiber](0,0.5)(3,0.5)
\end{pspicture}
```

barcompshape=`rectangle`, `circle` default: `rectangle`

```
\begin{pspicture}(3,1)
  \optbarcomp[barcompshape=circle, fiber](0,0.5)(3,0.5)
\end{pspicture}
```

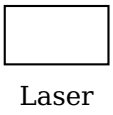
BarCompStyle ⟨*psstyle*⟩

The style of the bar.

```
\begin{pspicture}(3,1)
  \newpsstyle{BarCompStyle}{unit=0.7, arrows=->}
  \optbarcomp[barcompshape=circle, barcompangle=45,
      fiber](0,0.5)(3,0.5)
\end{pspicture}
```

4.9. Optical source

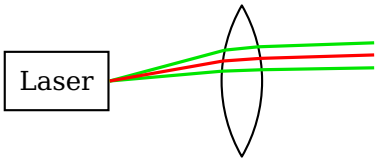
\optsource[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optsource(1.5,1)(3,1){Laser}
\end{pspicture}
```

An optical source predefines some properties of any outgoing beam, if it is the first component in a beam path. The definable beam properties are beamangle, beamwidth, beamdiv and beamalign. The properties defined by the optical source cannot be overwritten by the beam properties. If a property wasn't explicitely defined for a source, it is taken from the beam properties.

The default settings for the component position are position=start.




```
\begin{pspicture}(5,2)
  \pnodes(1.5,1){A}(5,1){B}
  \optsource[innerlabel, beamdiv=10,
      beamangle=10](A)(B){Laser}
  \lens[lens=2 2 2](A)(B)
  \optplane(B)
  \drawwidebeam{-}
  \drawbeam[linecolor=red]{-}
\end{pspicture}
```

sourcewidth=⟨num⟩ default: 1.4

The width of the box.

sourceheight=⟨num⟩ default: 0.8

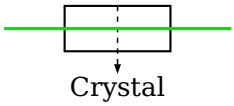
The height of the box.

sourcesize=⟨width⟩ ⟨height⟩

The width and height of the box, is equivalent to calling both sourcewidth and sourceheight.

4.10. Crystal

\crystal[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.3)
  \crystal[beam](0,1)(3,1){Crystal}
\end{pspicture}
```

crystalwidth=⟨num⟩ default: 1.4

The width of the crystal.

crystalheight=⟨num⟩ default: 0.6

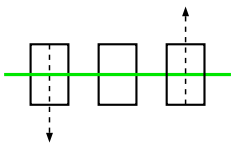
The height of the crystal.

crystalsize=⟨width⟩ ⟨height⟩

The width and height of the crystal, is equivalent to calling both crystalwidth and crystalheight.

caxislength=⟨num⟩ default: 0.3

The additional length of the c-axis, is dropped when set to 0 and is mirrored if the values is negative. The total length is caxislength plus crystalheight.



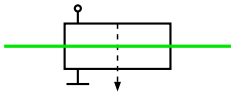
```
\begin{pspicture}(0,0.4)(3,1.5)
  \psset{crystalsize=0.5 0.8, label=-45 . l}
  \crystal[position=0.2, caxislength=0.5](0,1)(3,1){pos}
  \crystal[beam, caxislength=0](0,1)(3,1){0}
  \crystal[position=0.8, caxislength=-0.5](0,1)(3,1){neg}
\end{pspicture}
```

`caxisinv`=`true`, `false` default: `false`

Invert the direction of the c-axis, this is equivalent to changing the sign of `caxislength`.

`voltage`=`true`, `false` default: `false`

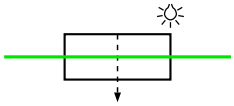
Draw a voltage connection and a ground symbol.



```
\begin{pspicture}(0,0.4)(3,1.5)
  \crystal[voltage, beam](0,1)(3,1)
\end{pspicture}
```

`lamp`=`true`, `false` default: `false`

Draw a lamp near the crystal.



```
\begin{pspicture}(0,0.4)(3,1.7)
  \crystal[lamp, beam](0,1)(3,1)
\end{pspicture}
```

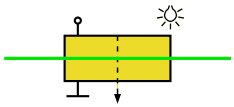
`CrystalCaxis` ⟨*psstyle*⟩ default: `linewidth=0.7\pslinewidth, linestyle=dashed, dash=2pt 2pt, arrowinset=0, arrows=->`

The style of the c-axis including the arrow type.

```
\begin{pspicture}(0,0.5)(3,1.6)
  \newpsstyle{CrystalCaxis}{linecolor=red, arrows=->>}
  \crystal[beam](0,1)(3,1)
\end{pspicture}
```

`CrystalLamp` ⟨*psstyle*⟩ default: `linewidth=0.6\pslinewidth`

The style of the crystal background lamp.




```
\begin{pspicture}(3,1.7)
  \crystal[fillstyle=solid, fillcolor=yellow!90!black,
           label=1.2 -45, voltage,
           lamp, beam](0,1)(3,1){SBN:Ce}
\end{pspicture}
```

4.11. Optical diode

\optdiode[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}

Diode

```
\begin{pspicture}(3,2)
  \optdiode[beam](0,1)(3,1){Diode}
\end{pspicture}
```

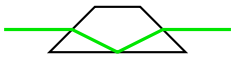
The optical diode has a default setting of allowbeaminside=false.

optdiodesize=⟨num⟩ default: 0.8

The side length of the optical diode.

4.12. Dove prism

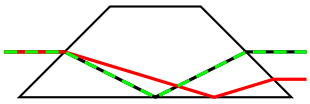
\doveprism[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}


Dove

```
\begin{pspicture}(3,1.5)
  \doveprism[beam](0,1)(3,1){Dove}
\end{pspicture}
```

doveprismsize=⟨num⟩ or ⟨width⟩ ⟨height⟩ default: 0.6

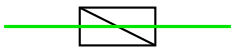
A single number defines the height of the prism, the total width is set to three times the height. Two number define the width and height of the dove prism. The angles at the input and output faces are always 45°.



```
\begin{pspicture}(4,1.5)
  \optplane(0,1)
  \doveprism[doveprismsize=1.2](0,1)(4,1)
  \optplane(4,1)
  \drawbeam[raytrace=false, linecolor=black]{-}
  \drawbeam[n=1.5, linecolor=red]{-}
  \drawbeam[n=*sqrt(5), linecolor=green, linestyle=dashed]{-}
\end{pspicture}
```

4.13. Glan–Thompson prism

\glanthompson[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \glanthompson[beam](0,1)(3,1){Glan-Thompson}
\end{pspicture}
```

glanthompsonwidth=⟨*num*⟩ default: 1.0

The width of the prism.

glanthompsonheight=⟨*num*⟩ default: 0.5

The height of the prism.

glanthompsonsize=⟨*width*⟩ ⟨*height*⟩

The width and height of the prism, is equivalent to calling both glanthompsonwidth and glanthompsonheight.

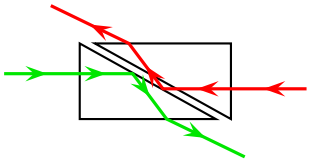
glanthompsongap=⟨*num*⟩ default: 0

The separation of the two prisms in x-direction. The total width does not depend on this value, but the angle of the reflective plane is affected.

```
\begin{pspicture}(3,1)
  \glanthompson[glanthompsongap=0.1, beam](0,0.5)(3,0.5)
\end{pspicture}
```

The implementation of the Glan–Thompson prism for glanthompsongap=0 is equivalent to a beamsplitter which is positioned with two nodes only and which is not quadratic.

Please note, that the Prism cannot be used properly from both sides for glanthompsongap> 0 because the current implementation allows only a single ambiguous interface (reflective or transmittive). In the following example you see, that the red beam is reflected at the wrong interface.



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \glanthompson[glanthompsongap=0.2, glanthompsonsize=2 1](A)(B)
  \optplane[angle=90](2,0)\optplane[angle=90](2,2)
  \addtopsstyle{Beam}{ArrowInside=->, arrowscale=1.5}
  \drawbeam[beampos=0.1](A){1-2}
  \drawbeam[beampos=0.1, linecolor=red](B){1}{3}
\end{pspicture}
```

Ex. 11 shows a further setup with a Glan–Thompson prism.

4.14. Polarization

`\polarization`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

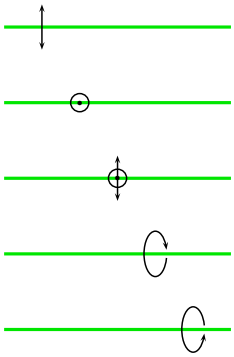
```
\begin{pspicture}(3,1)
  \polarization[beam](0,0.5)(3,0.5)
\end{pspicture}
```

`polsize`=⟨*num*⟩ default: `0.6`

The size of the polarization symbol, the circle for `perp` and `polmisc` is half of this.

`poltype`=`parallel`, `perp`, `misc`, `lcirc`, `rcirc` default: `parallel`

This parameter choses the polarization type.



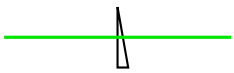
```
\begin{pspicture}(0,-0.3)(3,4.3)
  \psset[optexp]{beam}
  \begin{optexp}
  \polarization[poltype=parallel, abspos=0.5](0,4)(3,4)
  \polarization[poltype=perp, abspos=1](0,3)(3,3)
  \polarization[poltype=misc, abspos=1.5](0,2)(3,2)
  \polarization[poltype=lcirc, abspos=2](0,1)(3,1)
  \polarization[poltype=rcirc, abspos=2.5](0,0)(3,0)
  \end{optexp}
\end{pspicture}
```

`Polarization` ⟨*psstyle*⟩ default: `arrowscale=0.8, linewidth=0.7\pslinewidth, dotsize=3\pslinewidth`

Affects the style of the polarisation symbols.

4.15. Wedge

\optwedge[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optwedge[beam](0,1)(3,1){Wedge}
\end{pspicture}
```

wedgeheight=⟨*num*⟩ default: 0.8

The height of the wedge.

wedgeangleright=⟨*num*⟩ default: 10

The upper angle of the right interface with respect to a vertical line.

wedgeangleleft=⟨*num*⟩ default: 0

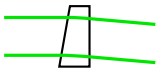
The upper angle of the left interface with respect to a vertical line.

wedgeangles=⟨*left*⟩[⟨*right*⟩]

Allows setting both angles at once. If only a single value is given, it is used for both angles.

wedgewidth=⟨*num*⟩ default: 0

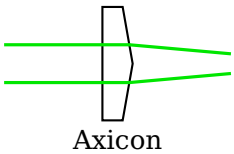
If this value is larger than the calculated base width, then the wedge's width is increase to this value.



```
\begin{pspicture}(2,1)
  \optwedge[wedgeangles=10 0, wedgewidth=0.4](0,0.5)(2,0.5)
  \drawwidebeam[beamwidth=0.5](0,0.5){}(2,0.5)
\end{pspicture}
```

4.16. Axicon

\axicon[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \axicon[label=1](0,1)(3,1){Axicon}
  \drawwidebeam[beamwidth=0.5](0,1){}(3,1)
\end{pspicture}
```

axiconheight=⟨*num*⟩ default: 1.5

The height of the axicon.

axiconwidth=⟨*num*⟩ default: 0.4

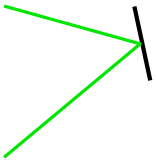
The width of the axicon.

axiconangle=⟨*num*⟩ default: 10

The upper angle of the right interface with respect to a vertical line.

4.17. Mirror

\mirror[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \mirror[beam](0,0)(1.8,1.5)(0,2){M}
\end{pspicture}
```

mirrorwidth=⟨*num*⟩ default: 1

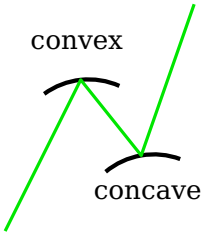
The width of the mirror.

mirrorlinewidth=⟨*num*⟩ or ⟨*dimen*⟩ default: 2\pslinewidth

The linewidth of the mirror. This could be defined also with the generic linewidth option. But this parameter allows setting globally the linewidth of all mirrors.

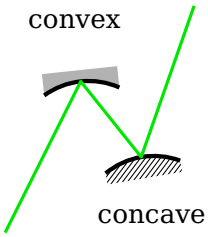
mirrorradius=⟨*radius*⟩[0] default: 0

This parameter defines the curvature of the mirror. A value of 0 is for a plain mirror, a negative radius for a convex mirror and a positive radius gives you a concave mirror.



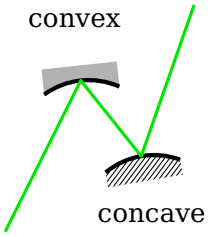

```
\begin{pspicture}(3,3)
  \psset[optexp]{labeloffset=0.5}
  \mirror[mirrorradius=1](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

If no second value is specified, the curvature of the second interface of an `extended` or `semitrans` mirror is the same as that of the main interface (see previous example). If zero is given as second value (other values are not supported), the second interface is plain.




```
\begin{pspicture}(3,3)
  \mirror[mirrorradius=1 0,
      mirrortype=semitrans](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1 0,
      mirrortype=extended](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

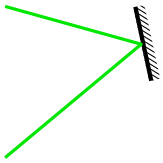
The value of `mirrordepth` always refers to the minimum depth of the mirror.




```
\begin{pspicture}(3,3)
  \psset[optexp]{mirrordepth=0.2}
  \mirror[mirrorradius=1 0,
      mirrortype=semitrans](0,0)(1,2)(1.8,1){convex}
  \mirror[mirrorradius=-1 0,
      mirrortype=extended](1,2)(1.8,1)(2.5,3){concave}
  \drawbeam(0,0){-}(2.5,3)
\end{pspicture}
```

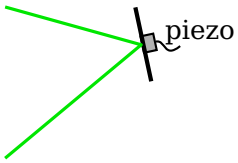
`mirrortype`=`plain`, `piezo`, `extended`, `semitrans` default: `plain`

The `mirrortype` selects between different types of mirrors. The style is controlled with `PiezoMirror`, `ExtendedMirror`, and `SemitransMirror`.



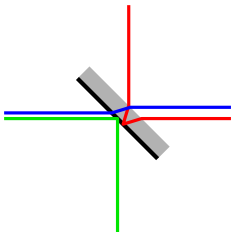

```
\begin{pspicture}(3,2)
  \mirror[mirrortype=extended,
          beam](0,0)(1.8,1.5)(0,2){ext}
\end{pspicture}
```

Note, that the default piece of wire is omitted when using `extnode` with a piezo mirror.

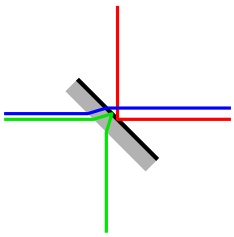



```
\begin{pspicture}(3,2)
\mirror[mirrortype=piezo,
        beam](0,0)(1.8,1.5)(0,2){piezo}
\end{pspicture}
```

A semi-transparent mirror (`semitrans`) has two interfaces, in contrast to the other mirror types. Their distance and position depends on the value and sign of `mirrordepth`.



```
\psset{unit=1.5}
\begin{pspicture}(2,2)
  \mirror[mirrortype=semitrans](0,1)(1,1)(1,0)
  \drawbeam(0,1){}(1,0)
  \drawbeam[linecolor=red](2,1){}(1,2)
  \drawbeam[beampos=0.05, linecolor=blue](0,1){}(2,1)
\end{pspicture}
```

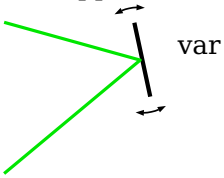


```
\psset{unit=1.5}
\begin{pspicture}(2,2)
  \mirror[mirrortype=semitrans, mirrordepth=-0.15](0,1)(1,1)(1,0)
  \drawbeam(0,1){}(1,0)
  \drawbeam[linecolor=red](2,1){}(1,2)
  \drawbeam[beampos=0.05, linecolor=blue](0,1){}(2,1)
\end{pspicture}
```

A negative `mirrordepth` is equivalent to a positive depth combined with `angle=180`.

`variable`=`true`, `false` default: `false`

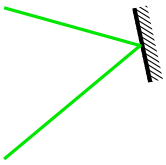
Draw an adjustable mirror which has two additional curved arrows on both sides. The appearance including the arrow types is determined by `VariableStyle`.




```
\begin{pspicture}(3,2)
  \mirror[beam, variable](0,0)(1.8,1.5)(0,2){var}
\end{pspicture}
```

`mirrordepth`=⟨*num*⟩ default: `0.1`

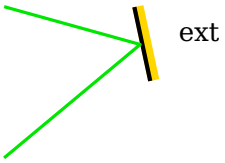
The total depth of an extended mirror.



```
\begin{pspicture}(3,2)
  \mirror[mirrortype=extended, mirrordepth=0.2,
          beam](0,0)(1.8,1.5)(0,2)
\end{pspicture}
```

`ExtendedMirror` ⟨*psstyle*⟩ default: `linestyle=none, hatchwidth=0.5\pslinewidth, hatchsep=1.4\pslinewidth, fillstyle=hlines`

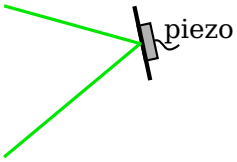
The style for the `extended` mirror.




```
\begin{pspicture}(3,2)
  \newpsstyle{ExtendedMirror}{fillstyle=solid,
                              fillcolor=Gold,
                              linestyle=none}
  \mirror[mirrortype=extended,
          beam](0,0)(1.8,1.5)(0,2){ext}
\end{pspicture}
```

PiezoMirror ⟨*psstyle*⟩ default: `fillstyle=solid, fillcolor=black!30`

This style defines the appearance of the `piezo` mirror. This can also be used to change the size of piezo part, as the example shows.




```
\begin{pspicture}(3,2)
  \addtopsstyle{PiezoMirror}{xunit=2}
  \mirror[mirrortype=piezo,
          beam](0,0)(1.8,1.5)(0,2){piezo}
\end{pspicture}
```

SemitransMirror ⟨*psstyle*⟩ default: `linestyle=none, fillstyle=solid, fillcolor=black!30`

This style defines the appearance of the `semitrans` mirror.

4.18. Parabolic mirror

`\parabolicmirror[`⟨*options*⟩`](`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`

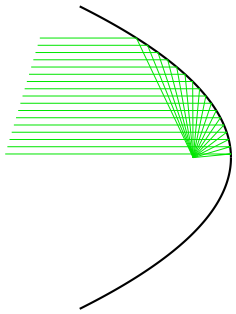
```
\begin{pspicture}(3,2)
  \parabolicmirror(0,1)(2,1){PM}
\end{pspicture}
```

parmirrorwidth=⟨*num*⟩ default: 1

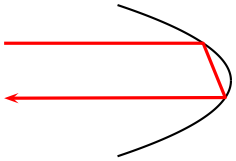
The width of the parabolic mirror.

parmirrorheight=⟨*num*⟩ default: 1

The height of the parabolic mirror.



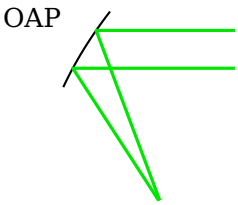
```
\begin{pspicture}(0,-2)(3,2)
  \parabolicmirror[parmirrorheight=4, parmirrorwidth=2](0,0)(3,0)
  \addtopsstyle{Beam}{beamalign=abs, linewidth=0.2\pslinewidth,
      beaminsidelast, stopinsidecount=1}
  \multido{\r=-1.55+0.1}{33}{%
    \drawbeam[beampos=\r](0,0){1}}
\end{pspicture}
```



```
\begin{pspicture}(0,-1)(3,1)
  \optplane(0,0)
  \parabolicmirror[parmirrorheight=2, parmirrorwidth=1.5](0,0)(3,0)
  \addtopsstyle{Beam}{arrows=->, beamalign=abs}
  \drawbeam[beampos=0.5, linecolor=red]{1}{2}{1}
\end{pspicture}
```

4.19. Off-axis parabolic mirror

\oapmirror[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*focus*⟩){⟨*label*⟩}



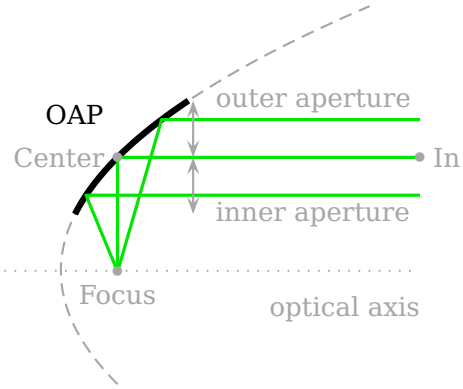

```
\begin{pspicture}(3,2.5)
  \oapmirror(3,2)(1,2)(2,0){OAP}
  \drawwidebeam[beamwidth=0.5](3,2){1}(2,0)
\end{pspicture}
```

oapmirroraperture=⟨*num*⟩ or ⟨*inner*⟩ ⟨*outer*⟩ default: 1

Mirror aperture with respect to the input direction. If a single number is given, it specifies the whole aperture, measured perpendicularly and symmetric to the connection from ⟨*in*⟩ to ⟨*center*⟩. Two numbers give the inner and the outer apertures.

The mirror construction using the given nodes is done as follows: the line from node ⟨*in*⟩ to ⟨*center*⟩ is parallel to the optical axis, ⟨*focus*⟩ specifies the focal point of the parabola. The mirror aperture is given by the parameter oapmirroraperture, measured perpendicular to the incoming beam.

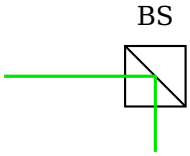
The different parameters and the whole original parabola are shown in the following illustration:




```
\begin{pspicture}(6,5)
\pnodes(5.5,3){In}(1.5,3){Center}(1.5,1.5){Focus}
\oapmirror[oapmirroraperture=1.5,
    linewidth=3\pslinewidth,
    plotpoints=200](In)(Center)(Focus){OAP}
\multido{\r=-0.5+0.5}{3}{%
  \drawbeam[beampos=\r](In){}(Focus)}
\end{pspicture}
```

4.20. Beamsplitter

\beamsplitter[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}



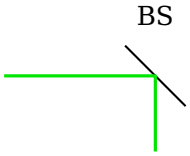

```
\begin{pspicture}(3,2)
  \beamsplitter[beam](0,1)(2,1)(2,0){BS}
\end{pspicture}
```

bssize=⟨*num*⟩ default: 0.8

The beamsplitter size.

bsstyle=cube, plate default: cube

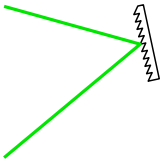
Selects between two types of beamsplitters: the beamsplitter cube (cube) and the semitransparent mirror (plate). This semitransparent mirror ist equivalent to \mirror with mirrortype=semitrans and mirrordepth=0.




```
\begin{pspicture}(3,2)
  \beamsplitter[bsstyle=plate, beam](0,1)(2,1)(2,0){BS}
\end{pspicture}
```

4.21. Optical grating

\optgrating[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \optgrating[beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingwidth`=⟨*num*⟩ default: `1`

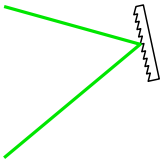
The width of the grating.

`gratingheight`=⟨*num*⟩ default: `0.15`

The total height of the grating.

`gratingdepth`=⟨*num*⟩ default: `0.075`

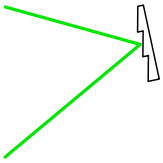
The modulation depth of the grating structure. The total height is adapted to this value if it is smaller than the modulation depth. Otherwise the total height is not affected.




```
\begin{pspicture}(3,2)
  \optgrating[gratingdepth=0.05,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingcount`=⟨*int*⟩ default: `10`

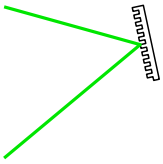
The number of grating grooves.




```
\begin{pspicture}(3,2)
  \optgrating[gratingcount=3,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingtype`=`blazed`, `binary` default: `blazed`

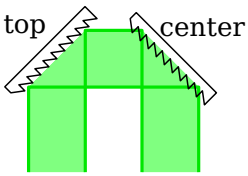
Select between a `binary` and a `blazed` grating.




```
\begin{pspicture}(3,2)
  \optgrating[gratingtype=binary,
              beam](0,2)(1.8,1.5)(0,0){G}
\end{pspicture}
```

`gratingalign`=`t`, `top`, `c`, `center` default: `top`

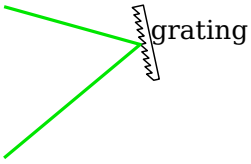
Selects if the reflection plane resides on `top` of the grating or in its `center`. If you choose `center`, you should use the `optexp` environment or a semitransparent beam filling.




```
\begin{pspicture}(3,2)
\pnodes(0.5,0){A}(0.5,1.5){B}(2,1.5){C}(2,0){D}
\psset{unit=1.5, labeloffset=0.4}
\begin{optexp}
  \optgrating[gratingalign=top](A)(B)(C){top}
  \optgrating[gratingalign=center](B)(C)(D){center}
  \drawwidebeam[fillstyle=solid, fillcolor=green!50,
      beamwidth=0.5](A){-}(D)
\end{optexp}
\end{pspicture}
```

`reverse`=`true`, `false` default: `false`

Reverse the slope of the grooves of the `blazed` grating.



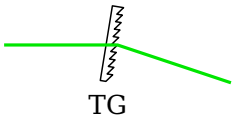

```
\begin{pspicture}(3,2)
  \optgrating[reverse, beam](0,2)(1.8,1.5)(0,0){grating}
\end{pspicture}
```

`gratinglinewidth`=⟨*num*⟩ or ⟨*dimen*⟩ default: `0.7\pslinewidth`

The linewidth of the grating. This could be defined also with the generic `linewidth` option. But this parameter allows setting globally the linewidth of all gratings.

4.22. Transmission grating

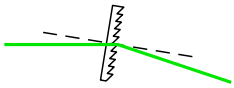
`\transmissiongrating`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*label*⟩}



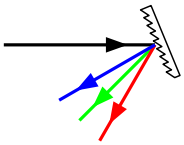

```
\begin{pspicture}(3,2)
  \transmissiongrating[beam](0,1)(1.5,1)(3,0.5){TG}
\end{pspicture}
```

The transmission grating supports all options of `\optgrating`. The beam path is calculated such, that the beam is reflected at an interface which is perpendicular to the drawn grating. This virtual interface is shown in the following example:

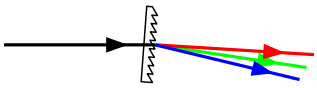
virtual reflection interface



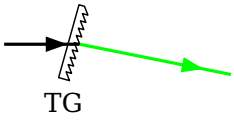
reflection grating \optgrating



transmission grating \transmissiongrating



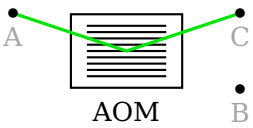
If this is not what you want, you may e.g. rotate the component further with angle, and then adjust also the beamangle.




```
\begin{pspicture}(3,1.4)
  \pnodes(0,1){A}(1,1){B}(3,0.6){C}
  \transmissiongrating[angle=-10, reverse](A)(B)(C){TG}
  \addtopsstyle{Beam}{ArrowInside=->, ArrowInsidePos=0.8,
      arrowinset=0, arrowscale=1.5}
  \drawbeam[linecolor=black](A){1}
  \drawbeam[linecolor=green, loadbeam, beamangle=20]{1}(C)
\end{pspicture}
```

4.23. Acousto-optic modulator

\optaom[⟨*options*⟩](⟨*in*⟩)(⟨*trans*⟩)(⟨*diff*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[beam](A)(B)(C){AOM}
\end{pspicture}
```

This macro draw an acousto-optic modulator, which either be transmittive or reflective, like a \beamsplitter. Please note, that the nodes do not correspond to those of a usual tripole like an \optgrating. The second node is in the direction of the transmitted beam, the third node gives the direction of the first diffraction order.

This first implementation in version 4.8 is still experimental, because I didn't have enough test cases to check all possible cases. Therefore, future versions may have changes, which are not backwards compatible.

aomheight=⟨*num*⟩ default: 1

The height of the modulator.

aomwidth=⟨*num*⟩ default: 1.5

The width of the modulator.

aomsize=⟨*width*⟩ ⟨*height*⟩

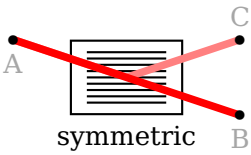
The width and height of the modulator, is equivalent to calling both aomwidth and aomheight.

aomgratingcount=⟨*int*⟩ default: 9

Changes the number of grating lines. For other changes of the grating you must use the aomcomp parameter to redefine the whole grating.

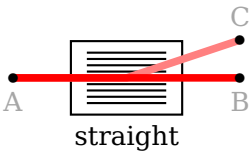
aomalign=symmetric, straight default: symmetric

The alignment of an \optaom with respect to the three reference nodes can be set with this option. The symmetric alignment is shown in the following example:




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomalign=symmetric](A)(B)(C){symmetric}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.1}
  \drawwidebeam[fillcolor=red!50](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
\end{pspicture}
```

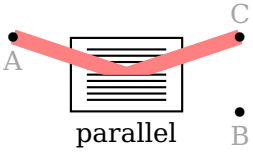
For straight alignment the sketch looks like




```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}(3,1.5){C}
  \optaom[aomalign=straight](A)(B)(C){straight}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.1}
  \drawwidebeam[fillcolor=red!50](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
\end{pspicture}
```

aomreflalign=perp, parallel default: parallel

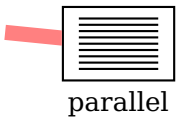
The reflective interface can be oriented either parallel or perpendicular to the component. This option was necessary in first place for technical reasons, the direction of the reflected ray is not affected by this option.




```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomreflalign=parallel](A)(B)(C){parallel}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```

```
\begin{pspicture}(3,2)
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomreflalign=perp](A)(B)(C){perp}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```

The following example show a use case, which doesn't work properly with aomreflalign=parallel, because the incident angle is so shallow, that the intersection points with the interface lay outside of the component.




```
\begin{pspicture}(3,1.5)
  \pnodes(0,1.1){A}(3,0.8){B}(3,1.1){C}
  \optaom[aomreflalign=parallel](A)(B)(C){parallel}
  \addtopsstyle{Beam}{fillstyle=solid, linestyle=none, beamwidth=0.2}
  \drawwidebeam[fillcolor=red!50](A){}(C)
\end{pspicture}
```

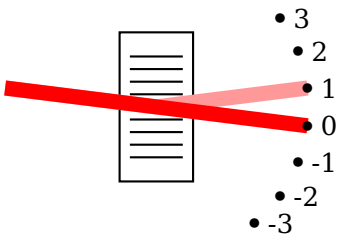
aomcomp=default, ⟨*macro*⟩ default: default

Changes the grating lines to an arbitrary drawing.

diffractionorders=⟨*int*⟩ default: 2

The number or diffraction orders (positive and negative) which can be selected with beamdiffractionorder, i.e. $-\langle num\rangle, -\langle num\rangle+1, \ldots, \langle num\rangle$.

At the same time the nodes \oenode{DO-1}{<comp>}, \oenode{DO0}{<comp>}, \oenode{DO1}{<comp>} etc. are defined.



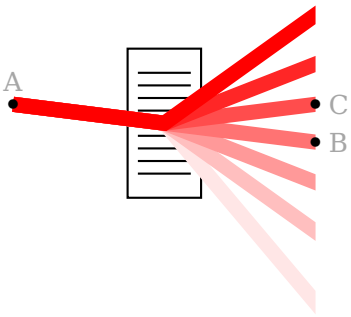

```
\begin{pspicture}(0,0.6)(4.3,3.6)
  \pnodes(0,2.5){A}(4,2){B}(4,2.5){C}
  \optaom[aomsize=1 2, aomreflalign=perp,
      diffractionorders=3](A)(B)(C)
  \addtopsstyle{Beam}{beamwidth=0.2, linestyle=none,
      fillstyle=solid}
  \drawwidebeam[fillcolor=red!40](A){}(C)
  \drawwidebeam[fillcolor=red](A){}(B)
  \multido{\i=-3+1}{7}{%
    \psdot(\oenode{DO\i}{})\uput[0](\oenode{DO\i}{}){\i}}%
\end{pspicture}%
```

beamdiffractionorder=⟨*int*⟩ default: {}

The diffraction order, which is used for a reflected beam going through the aom. The option is ignored if its value is empty {}.

By using this option you automatically select the behaviour of the interface, which then doesn't depend on the relative position of AOM and the following component. For beamdiffractionorder$\neq 0$ a beam gets reflected, for the value 0 it gets transmitted. The parameter beammode is ignored.

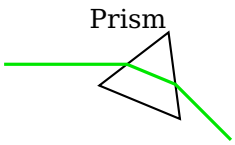
You can select only those orders, where have been defined for the respective component with diffractionorders. If an undefined order is selected, the closest one is choosen instead and a Postscript warning is issued (see pswarning).




```
\begin{pspicture}(0,-0.2)(4.4,3.8)
  \pnodes(0,2.5){A}(4,2){B}(4,2.5){C}
  \optaom[aomsize=1 2, aomreflalign=perp, diffractionorders=3](A)(B)(C)
  \optplane(B)
  \addtopsstyle{Beam}{beamwidth=0.2, linestyle=none, fillstyle=solid}
  \multido{\i=-3+1,\r=10+15}{7}{%
    \drawwidebeam[fillcolor=red!\r, beamdiffractionorder=\i](A){-}}
\end{pspicture}%
```

4.24. Prism

\optprism[⟨options⟩](⟨in⟩)(⟨center⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \optprism[beam](0,1)(2,1)(3,0){Prism}
\end{pspicture}
```

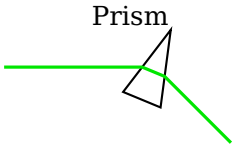
The prism is always placed symmetric between ⟨*in*⟩ and ⟨*out*⟩ nodes. For asymmetric beam traces see Sec. 9.

prismsize=⟨*num*⟩ default: 1

The height of the prism.

prismangle=⟨*num*⟩ default: 60

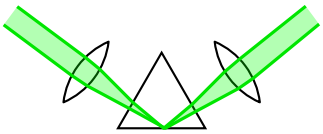
The upper angle of the prism.




```
\begin{pspicture}(3,2)
  \optprism[prismangle=30, beam](0,1)(2,1)(3,0){Prism}
\end{pspicture}
```

prismtype=transmittive, reflective default: transmittive

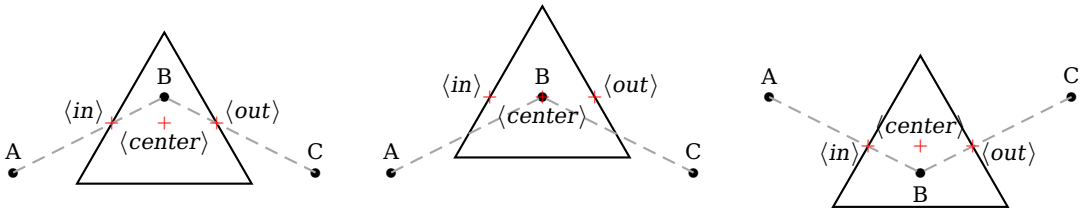
If this parameter is set to reflective, the prism base is used as additional, reflective interface.



```
\begin{pspicture}(1.5,0.6)(5.1,2.2)
  \optprism[prismtype=reflective](0,1)(3,0.9)(6,1)
  \lens(1,2)(\oenodeIfc{2}{1})
  \lens(\oenodeIfc{2}{1})(5,2)
  \drawwidebeam[beamwidth=0.3, fillstyle=solid,
      fillcolor=green, opacity=0.3](1,2){2}{1}{3}(5,2)
\end{pspicture}
```

prismalign=auto, center default: auto

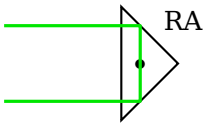
Sets the vertical alignment of the prism with respect to the “reflection” node ⟨*center*⟩. For auto, the prism is shifted such, that the interface nodes lay on the connection between the respective reference node and the “reflection” node. If the value is center, the “reflection” node coincides with the component center. See the following examples for further explanation.



prismalign=auto prismtype=transmittive | prismalign=center prismtype=transmittive | prismalign=auto prismtype=reflective

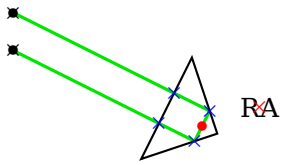
4.25. Right-angle prism

\rightangleprism[⟨options⟩](⟨in⟩)(⟨center⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \psdot(1.8,1)
  \rightangleprism[beam](0,1.5)(1.8,1)(0,0.5){RA}
\end{pspicture}
```

The right-angle prisms is align such that the incoming and reflected beam are parallel and the ⟨*center*⟩ node is vertically centered in the prism.




```
\begin{pspicture}(3.5,2)
  \pnodes(0,2){A}(2.5,0.5){G}(0,1.5){B}
  \begin{optexp}
  \rightangleprism[beam, showifcnodes, showoptdots](A)(G)(B){RA}
  \end{optexp}
\end{pspicture}
```

raprismsize=⟨*num*⟩ default: 1.5

The length of the input plane.

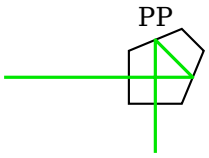
raprismalign=auto, center default: auto

Switch between automatic and fixed alignment of the component vertical to the entrance interface:

raprismalign=auto | raprismalign=center

4.26. Penta prism

\pentaprism[⟨options⟩](⟨in⟩)(⟨center⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \pentaprism[beam](0,1)(2,1)(2,0){PP}
\end{pspicture}
```

pentaprismsize=⟨num⟩ default: 0.7

The length of the input and output plane.

5. Fiber components

This chapter describes all fiber components and their options. They differ from free-ray components in that they are connected by default with fibers to their reference nodes, and that they cannot be used for raytracing. Instead of automatic fiber connections the components can also be connected manually with fibers, see Sec. 9.6.

Most components require only two reference nodes and are handled like the free-ray dipoles. Some special components like couplers (Sec. 5.10) and circulators (Sec. 5.9) are treated differently.

`usefiberstyle`=`true`, `false`

For some components (e.g. `\optfilter` or `\optmzm`) it can be nice to highlight some internals. For example, if this option is enabled the passing parts of the optical filter are drawn with the `Fiber` style. In the documentation this parameter is enabled to show the parts which are affected. Takes precedence over `usewirestyle`.

`usewirestyle`=`true`, `false`

Can be used to draw internal connections als `Wire`. Sets `usefiberstyle` to `false`, therefore it can be useful to define e.g. an electrical filter:

```
\newpsobject{elecfilter}{optfilter}{usewirestyle}
```

5.1. Optical fiber

`\optfiber`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

```
\begin{pspicture}(3,1.5)
  \optfiber[label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

`fiberloops`=⟨*int*⟩ default: 3

Number of the fiber loops.

```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloops=2, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

`fiberloopradius`=⟨*num*⟩ default: `0.4`

Radius of the fiber loops.

```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloopradius=0.2, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

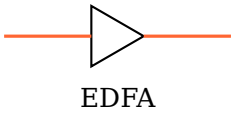
`fiberloopsep`=⟨*num*⟩ default: `0.2`

Separation between two successive fiber loops.

```
\begin{pspicture}(3,1.5)
  \optfiber[fiberloopsep=0.6, label=0.3](0,0.5)(3,0.5){SSMF}
\end{pspicture}
```

5.2. Optical amplifier

`\optamp`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



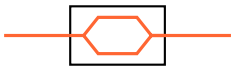

```
\begin{pspicture}(3,1.5)
  \optamp(0,1)(3,1){EDFA}
\end{pspicture}
```

`optampsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

A single number gives the side length of the amplifier, two numbers the width and height. Note, that `optampsize=1` and `optampsize=1 1` do not give the same result. You may also change the relation of width and height with `xunit` and `yunit`.

5.3. Mach–Zehnder modulator

`\optmzm`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.5)
  \optmzm(0,1)(3,1){MZM}
\end{pspicture}
```

`optmzmsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

A single number gives the modulator height, the width is `1.6` times the height. Two numbers define width and height directly.

5.4. Polarization controller

`\polcontrol`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

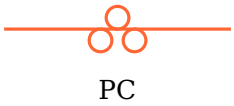
```
\begin{pspicture}(3,1.5)
  \polcontrol(0,1)(3,1){PC}
\end{pspicture}
```

`polcontrolsize`=⟨*num*⟩ default: `0.15`

The radius of the polarization controller circles.

`polcontroltype`=`linear`, `triangle` default: `linear`

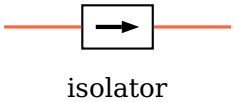
The type of polarization controller.




```
\begin{pspicture}(3,1.5)
  \polcontrol[polcontroltype=triangle](0,1)(3,1){PC}
\end{pspicture}
```

5.5. Isolator

`\optisolator`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



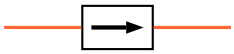

```
\begin{pspicture}(3,1.5)
  \optisolator(0,1)(3,1){isolator}
\end{pspicture}
```

`isolatorsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.6`

A single number gives the isolator height, the width is `1.6` times the height. Two numbers define width and height directly.

`IsolatorArrow` ⟨*psstyle*⟩ default: `linewidth=2\pslinewidth, arrowinset=0`

The style of the isolator arrow. This can be especially useful to adapt the length of the arrow.

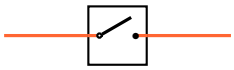


isolator

```
\begin{pspicture}(3,1.5)
  \addtopsstyle{IsolatorArrow}{xunit=1.2}
  \optisolator(0,1)(3,1){isolator}
\end{pspicture}
```

5.6. Optical switch

`\optswitch`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}



opened switch

```
\begin{pspicture}(3,1.5)
  \optswitch(0,1)(3,1){opened switch}
\end{pspicture}
```

`switchsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

A single number defines the side length of the switch, otherwise the height and width can be specified separately.

`switchstyle`=`opened`, `closed` default: `opened`

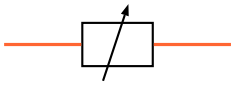
Indicate the switch as `opened` or `closed`.

closed switch

```
\begin{pspicture}(3,1.5)
  \optswitch[switchstyle=closed](0,1)(3,1){closed switch}
\end{pspicture}
```

5.7. Fiber delay-line

\fiberdelayline[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}




```
\begin{pspicture}(3,1.5)
  \fiberdelayline(0,1)(3,1){delay line}
\end{pspicture}
```

fdlsize=⟨num⟩ or ⟨width⟩ ⟨height⟩ default: 0.6

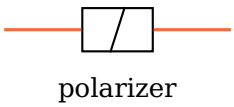
A single number gives the component height, the width is 1.6 times the height. Two numbers define width and height directly.

FdlArrow ⟨psstyle⟩ default: arrowinset=0, arrows=->

See ArrowCompStyle.

5.8. Polarizer

\optfiberpolarizer[⟨options⟩](⟨in⟩)(⟨out⟩){⟨label⟩}



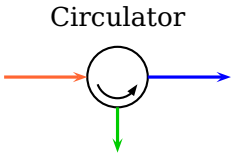

```
\begin{pspicture}(3,1.5)
  \optfiberpolarizer(0,1)(3,1){polarizer}
\end{pspicture}
```

fiberpolsize=⟨num⟩ or ⟨width⟩ ⟨height⟩ default: 0.6

A single number gives the component height, the width is 1.6 times the height. Two numbers define width and height directly.

5.9. Optical circulator

\optcirculator(⟨left⟩)(⟨right⟩)(⟨bottom⟩){⟨label⟩}




```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrows=->}
  \addtopsstyle{FiberOut1}{linecolor=blue}
  \addtopsstyle{FiberOut2}{linecolor=green!80!black}
  \optcirculator(0,1)(3,1)(1.5,0){Circulator}
\end{pspicture}
```

optcircsize=⟨*num*⟩ default: 0.8

The diameter of the circulator.

optcircangleA=⟨*num*⟩ default: -160

The starting angle of the internal arrow.

optcircangleB=⟨*num*⟩ default: -20

The ending angle of the internal arrow.

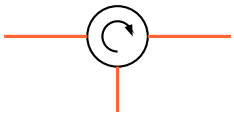
optcircangle=⟨*num*⟩ ⟨*num*⟩

Short notation to specifying both angles.

```
\begin{pspicture}(3,1.5)
  \optcirculator[optcircangle=0 -90](0,1)(3,1)(1.5,0)
\end{pspicture}
```

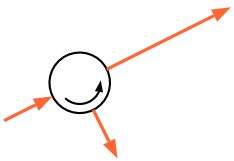
OptCircArrow ⟨*psstyle*⟩ default: unit=0.7, arrows=->, arrowinset=0

The style of the internal arrow. It specifies the direction of the arrow and the size of the arc.



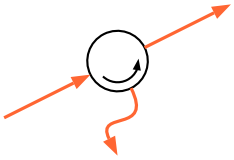
```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptCircArrow}{unit=0.5, arrows=<-, arrowinset=0}
  \optcirculator[optcircangle=0 -90](0,1)(3,1)(1.5,0)
\end{pspicture}
```

The circulator is positioned by default such that the input and output fibers are orthogonal to each other, like shown in the example:



```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0}
  \optcirculator(0,0.5)(3,2)(1.5,0)
\end{pspicture}
```

The positioning parameters (Sec. 3.2) refer to the reference nodes ⟨*left*⟩ and ⟨*right*⟩ as usual and are not related to the automatic position. This implies, that the default position is not equivalent to position=0.5.

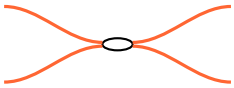


```
\begin{pspicture}(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0,
      ncurv=1.5}
  \optcirculator[position=0.5](0,0.5)(3,2)(1.5,0)
\end{pspicture}
```

5.10. Fiber coupler

The package provides three fiber couplers which all share the same shapes and parameters.

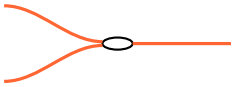
\optcoupler(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}



optcoupler

```
\begin{pspicture}(0,-0.4)(3,1)
  \optcoupler(0,1)(0,0)(3,1)(3,0){optcoupler}
\end{pspicture}
```

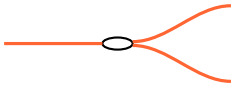
\wdmcoupler(⟨*tl*⟩)(⟨*...*⟩)(⟨*bl*⟩)(⟨*r*⟩){⟨*label*⟩}



wdmcoupler

```
\begin{pspicture}(0,-0.4)(3,1)
  \wdmcoupler(0,1)(0,0)(3,0.5){wdmcoupler}
\end{pspicture}
```

\wdmsplitter(⟨*l*⟩)(⟨*tr*⟩)(⟨*...*⟩)(⟨*br*⟩){⟨*label*⟩}



wdmsplitter

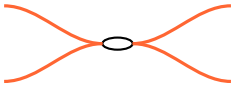
```
\begin{pspicture}(0,-0.4)(3,1)
  \wdmsplitter(0,0.5)(3,1)(3,0){wdmsplitter}
\end{pspicture}
```

couplersize=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: 0.2

For a single number the width is twice this value, the height 0.8 times this value. Two numbers define width and height directly.

couplersep=⟨*num*⟩ default: 0.05

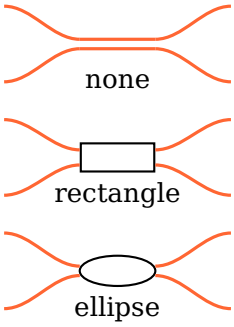
The vertical distance between two fiber ports.



coupler

```
\begin{pspicture}(0,-0.5)(3,1)
  \optcoupler[couplersep=0](0,1)(0,0)(3,1)(3,0){coupler}
\end{pspicture}
```

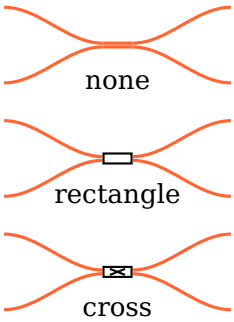
If this value is set to {}, the separation is calculated automatically according to the couplertype and couplersize.




```
\begin{pspicture}(3,4.5)
  \psset{couplersep={}, couplersize=0.5, labeloffset=0.5}
  \optcoupler[couplertype=none]%
      (0,4.5)(0,3.5)(3,4.5)(3,3.5){none}
  \optcoupler[couplertype=rectangle]%
      (0,3)(0,2)(3,3)(3,2){rectangle}
  \optcoupler[couplertype=ellipse]%
      (0,1.5)(0,0.5)(3,1.5)(3,0.5){ellipse}
\end{pspicture}
```

`couplertype`=`none`, `ellipse`, `rectangle`, `cross` default: `ellipse`

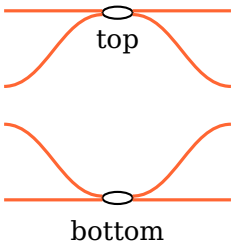
Select between different coupler types.




```
\begin{pspicture}(3,4.5)
  \psset{labeloffset=0.5}
  \optcoupler[couplertype=none]%
      (0,4.5)(0,3.5)(3,4.5)(3,3.5){none}
  \optcoupler[couplertype=rectangle]%
      (0,3)(0,2)(3,3)(3,2){rectangle}
  \optcoupler[couplertype=cross]%
      (0,1.5)(0,0.5)(3,1.5)(3,0.5){cross}
\end{pspicture}
```

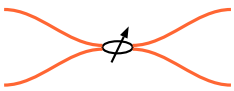
`coupleralign`=`t`, `top`, `b`, `bottom`, `c`, `center` default: `center`

The alignment of the coupler with respect to the reference nodes. Note, that the exact position of the coupler center and accordingly that of the label depends on this setting (see 8.3).




```
\begin{pspicture}(3,3)
  \psset{labeloffset=0.4}
  \optcoupler[coupleralign=top]%
             (0,3)(0,2)(3,3)(3,2){top}
  \optcoupler[coupleralign=bottom]%
             (0,1.5)(0,0.5)(3,1.5)(3,0.5){bottom}
\end{pspicture}
```

`variable`=`true`, `false` default: `true`



```
\begin{pspicture}(3,1)
  \optcoupler[variable](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```

VariableCoupler ⟨*psstyle*⟩ default: `arrowinset=0, arrows=->`

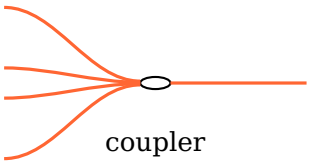
The style of the arrow of the variable coupler.

5.10.1. Variable node count

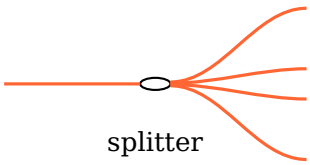
Every coupler has a variant which needs only two nodes, one input and one output node. For \optcoupler, both nodes are used twice, \wdmsplitter uses node ⟨*out*⟩ twice, and \wdmcoupler uses node ⟨*in*⟩ twice. This can be very useful if you do not use the automatic connections but connect the components manually.

```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \psset[optexp]{fiber=none}
  \wdmsplitter[position=0.2, coupleralign=b](A)(B)
  \wdmcoupler[position=0.8, coupleralign=t](A)(B)
  \drawfiber{(A)}{1}{2}{(B)}
  \drawfiber{1}{([offset=0.5, Xnodesep=1]\oenodeIfc{2}{1})}
  \drawfiber{([offset=-0.5, Xnodesep=-1]\oenodeIfc{2}{2})}{2}
\end{pspicture}
```

A \wdmcoupler can have more than two input nodes, a \wdmsplitter more than two output nodes.

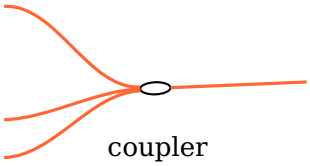



```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(0,1.2){B}(0,0.8){C}(0,0){D}(4,1){E}
  \wdmcoupler(A)(B)(C)(D)(E){coupler}
\end{pspicture}
```

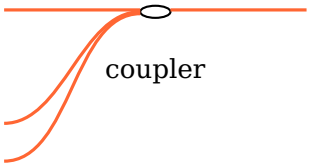



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,2){B}(4,1.2){C}(4,0.8){D}(4,0){E}
  \wdmsplitter(A)(B)(C)(D)(E){splitter}
\end{pspicture}
```

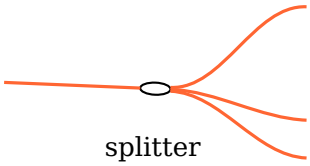
The component's orientation is calculated from the median of all input nodes (\wdmcoupler) or all output nodes (\wdmsplitter), unless coupleralign is used.



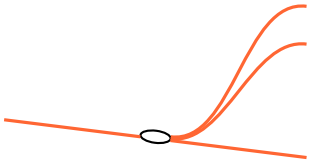

```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(0,0.5){B}(0,0){C}(4,1){D}
  \wdmcoupler(A)(B)(C)(D){coupler}
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0,2){E}(0,0.5){F}(0,0){G}(4,2){H}
  \wdmcoupler[coupleralign=t](E)(F)(G)(H){coupler}
\end{pspicture}
```



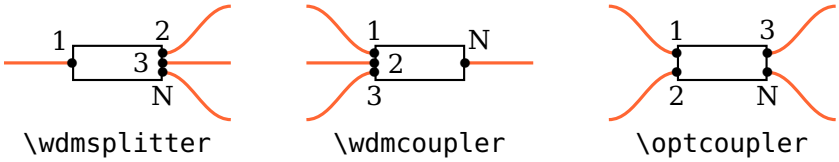

```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,2){B}(4,0.5){C}(4,0){D}
  \wdmsplitter(A)(B)(C)(D){splitter}
\end{pspicture}
```




```
\begin{pspicture}(4,2)
  \pnodes(0,0.5){E}(4,2){F}(4,1.5){G}(4,0){H}
  \wdmsplitter[coupleralign=b](E)(F)(G)(H){splitter}
\end{pspicture}
```

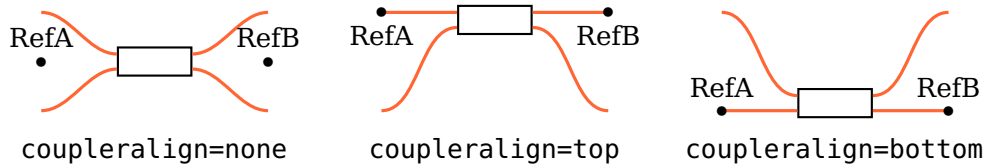
5.10.2. Input and output nodes

The definition of input and output nodes from Sec. 8.6 cannot be applied to couplers. Here, the nodes are simply numbered from 1 (left top) to N (right bottom).



5.10.3. Reference nodes

The definition of reference nodes from Sec. 8.2 cannot be applied to couplers and depends on `coupleralign`.



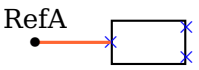
5.11. Fiber box

\fiberbox(⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

\fiberbox(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}

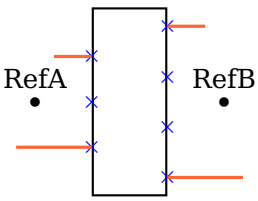
A \fiberbox can be positioned either with two, or four nodes which affects the positioning of the component and the interface nodes, and the automatic fiber connections.

If only two nodes are used these are the reference nodes and the separation between the components input and output nodes must be specified manually (see fiberboxsepin and fiberboxsepout). Only the side of the component is connected automatically which has a single node only (see fiberboxcount and the following example).




```
\begin{pspicture}(3,2)
  \fiberbox[fiberboxcount=1x2, showifcnodes](0,1)(3,1)
\end{pspicture}
```

When four nodes are used the interface nodes and their separation are determined automatically. The component is positioned between the center of the two input and the center of the two output nodes. The first input interface node is aligned with the ⟨*tl*⟩ node, the last input node is aligned with ⟨*bl*⟩, all other input nodes are equally distributed between these. The output nodes are handled accordingly. The four specified nodes are connected automatically to the component, each with the interface node it is aligned to.



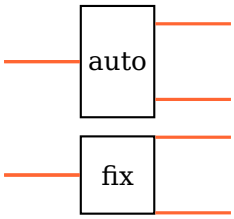

```
\begin{pspicture}(3,2)
  \fiberbox[fiberboxcount=3x4, ↵
      showifcnodes](0.5,1.6)(0,0.4)(2.5,2)(3,0)
\end{pspicture}
```

fiberboxwidth=⟨*num*⟩ default: 1

The width of the fiber box.

fiberboxheight=⟨*num*⟩ default: 0

The height of the fiber box. If the height is lower than the space required by the input or output nodes, it is calculated automatically. The calculated height is $\max(\text{sepin} \times (\text{N}-1), \text{sepout} \times (\text{M}-1))$.




```
\begin{pspicture}(3,2.8)
  \psset[optexp]{innerlabel, fiberboxwidth=1, fiberboxcount=1x2}
  \fiberbox(0,2)(0,2)(3,2.5)(3,1.5){auto}
  \fiberbox[fiberboxheight=1.05](0,0.5)(0,0.5)(3,1)(3,0){fix}
\end{pspicture}
```

fiberboxsize=⟨*width*⟩ ⟨*height*⟩

The width and height of the fiber box, is equivalent to calling both fiberboxwidth and fiberboxheight.

fiberboxsepin=⟨*num*⟩ default: 0.2

The separation between two input nodes. This value is used only if the fiber box is positioned with two nodes. Otherwise the separation is calculated automatically.

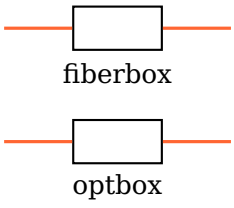
fiberboxsepout=⟨*num*⟩ default: 0.2

Equivalent to fiberboxsepout, but for the output nodes.

fiberboxcount=⟨*N*⟩x⟨*M*⟩ default: 2x2

Number of input (⟨*N*⟩) and output nodes (⟨*M*⟩).

For certain parameter configurations a \fiberbox is equivalent to an \optbox.

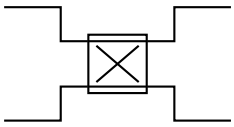



```
\begin{pspicture}(0,0.3)(3,2.6)
  \psset[optexp]{label=0.6}
  \fiberbox[fiberboxsize=1.2 0.6, ↩
      fiberboxcount=1x1](0,2.5)(3,2.5){fiberbox}
  \optbox[optboxsize=1.2 0.6, fiber](0,1)(3,1){optbox}
\end{pspicture}
```

6. Electrical components

6.1. Coupler

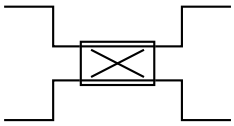
\eleccoupler(⟨*tl*⟩)(⟨*bl*⟩)(⟨*tr*⟩)(⟨*br*⟩){⟨*label*⟩}



```
\begin{pspicture}(3,1.5)
  \eleccoupler(0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

eleccouplersize=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: 0.8

The width and height of the couplers, if only one value is given the component is squared.



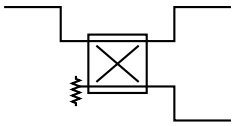
```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplersize=1 0.6](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

eleccouplersep=⟨*num*⟩ default: {}

The vertical distance between the two input and output ports. If the value is empty {}, then the separation is set to 75% of the height.

eleccouplertype=standard, directional default: standard

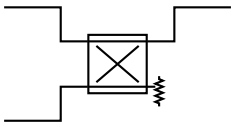
Select between different coupler types.



```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplertype=directional](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

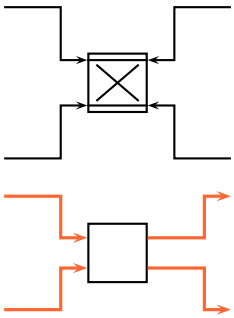
eleccouplerinput=left, right default: left

Selects the input side for the directional coupler.



```
\begin{pspicture}(3,1.5)
  \eleccoupler[eleccouplertype=directional,
      eleccouplerinput=right](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

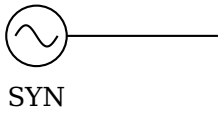
The automatic connections of the electrical coupler are all directed towards the component, in contrast to the optical coupler. This gives a symmetric layout of the automatic electrical coupler connections when using the default of `wirestyle=angle`.



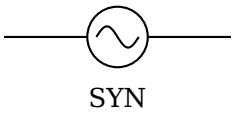
```
\begin{pspicture}(3,4)
  \addtopsstyle{Wire}{arrows=->}
  \addtopsstyle{Fiber}{arrows=->, fiberstyle=angle}
  \eleccoupler(0,4)(0,2)(3,4)(3,2)
  \optcoupler[couplersize=0.8 0.8, couplersep={},
      couplertype=rectangle](0,1.5)(0,0)(3,1.5)(3,0)
\end{pspicture}
```

6.2. Synthesizer

`\elecsynthesizer(`⟨*in*⟩`)(`⟨*out*⟩`){`⟨*label*⟩`}`




```
\begin{pspicture}(3,1.5)
  \elecsynthesizer(1,1)(3,1){SYN}
\end{pspicture}
```



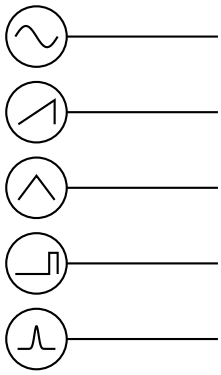

```
\begin{pspicture}(3,1.5)
  \elecsynthesizer[position=0.5](0,1)(3,1){SYN}
\end{pspicture}
```

`synthsize`=⟨*size*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.8`

The size of the component, if this is of circular shape and two values were given, the smaller one is used as diameter.

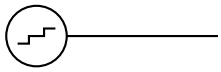
`synthtype`=`sine`, `pulse`, `sawtooth`, `rectangle`, `triangle`, `custom` default: `sine`

Select between different synthesizer types.



```
\begin{pspicture}(0,0.6)(3,5.4)
  \elecsynthesizer[synthtype=sine](1,5)(3,5)
  \elecsynthesizer[synthtype=sawtooth](1,4)(3,4)
  \elecsynthesizer[synthtype=triangle](1,3)(3,3)
  \elecsynthesizer[synthtype=rectangle](1,2)(3,2)
  \elecsynthesizer[synthtype=pulse](1,1)(3,1)
\end{pspicture}
```

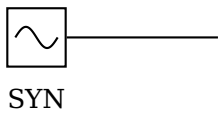
You can also use a freely customizable waveform.



```
\begin{pspicture}(0,0)(3,2)
  \makeatletter
  \def\elecsynthesizer@custom{%
    \psline(-0.25,-0.1)(-0.1,-0.1)(-0.1,0)%
          (0.1,0)(0.1,0.1)(0.25,0.1)}
  \makeatother
  \elecsynthesizer[synthtype=custom](1,1)(3,1)
\end{pspicture}
```

`synthshape`=`circle`, `rectangle` default: `circle`

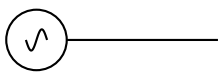
Select between circular and rectangular outer shape.




```
\begin{pspicture}(3,1.5)
  \elecsynthesizer[synthshape=rectangle](1,1)(3,1){SYN}
\end{pspicture}
```

`SynthStyle` ⟨*psstyle*⟩

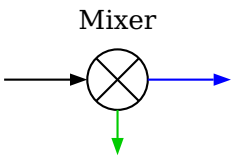
Change the style of the internal synthesizer symbol.



```
\begin{pspicture}(3,1)
  \newpsstyle{SynthStyle}{xunit=0.5, yunit=-1}
  \elecsynthesizer(1,0.5)(3,0.5)
\end{pspicture}
```

6.3. Mixer

`\elecmixer`(⟨*left*⟩)(⟨*right*⟩)(⟨*bottom*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,2)
  \newpsstyle{Wire}{arrowscale=1.5, arrowinset=0, arrows=->}
  \addtopsstyle{WireOut1}{linecolor=blue}
  \addtopsstyle{WireOut2}{linecolor=green!80!black}
  \elecmixer(0,1)(3,1)(1.5,0){Mixer}
\end{pspicture}
```

`elecmixersize`=⟨*num*⟩ default: `0.8`

The diameter of the mixer.

For further details about the reference nodes and the positioning, please refer to `\optcirculator`.

7. Hybrid components

This chapter describes the components which can be used both for free-ray and fiber optics but which differ from these two categories.

The \optfilter is connected by default with automatic fiber connections, but can likewise be used with free-ray beams, in contrast to the fiber components (Sec. 5). The \fibercollimator has one interface only, the other connection is a fiber and the \optdetector has one interface, the other connection is a wire.

7.1. Optical filter

\optfilter[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}

bandpass

```
\begin{pspicture}(3,1.5)
  \optfilter(0,1)(3,1){bandpass}
\end{pspicture}
```

filtersize=⟨*num*⟩ default: 0.8

The size of the filter.

filtertype=bandpass, bandstop, lowpass, highpass default: bandpass

Select between different filter types.

bandstop

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=bandstop](0,1)(3,1){bandstop}
\end{pspicture}
```

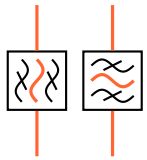
lowpass

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=lowpass](0,1)(3,1){lowpass}
\end{pspicture}
```

```
\begin{pspicture}(3,1.5)
  \optfilter[filtertype=highpass](0,1)(3,1){highpass}
\end{pspicture}
```

`filterangle`=⟨*num*⟩ default: `0`

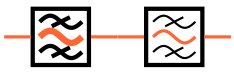
Rotates the “inner” part of the filter relativ to its frame. Alternatively `innercompalign` can be used.



```
\begin{pspicture}(2,2)
  \optfilter(0.5,0)(0.5,2)
  \optfilter[filterangle=90](1.5,0)(1.5,2)
\end{pspicture}
```

`FilterStyle` ⟨*psstyle*⟩

Change the style of the internal filter lines.



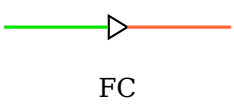
```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptComp}{linewidth=2\pslinewidth}
  \optfilter(0,1)(1.5,1)
  \newpsstyle{FilterStyle}{linewidth=0.5\pslinewidth}
  \optfilter(1.5,1)(3,1)
\end{pspicture}
```

Usage as free-ray component, `allowbeaminside` is set to `false` by default:

```
\begin{pspicture}(3,1.5)
  \optfilter[fiber=none](0,1)(3,1)
  \addtopsstyle{Beam}{beamwidth=0.3,
     fillstyle=solid, fillcolor=green, opacity=0.2}
  \drawwidebeam(0,1){}(3,1)
\end{pspicture}
```

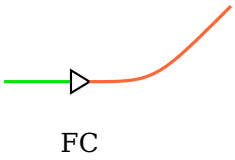
7.2. Fiber collimator

`\fibercollimator`(⟨*in*⟩)(⟨*A*⟩)(⟨*B*⟩)(⟨*out*⟩){⟨*label*⟩}



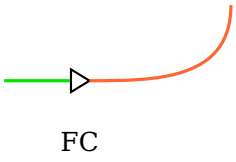

```
\begin{pspicture}(3,1.5)
  \fibercollimator[beam](0,1)(3,1){FC}
\end{pspicture}
```

The fiber collimator can be used with two, three or four points. With two points, the collimator is placed like any other dipole component between ⟨*in*⟩ and ⟨*out*⟩ node. For three nodes, the fiber is drawn as `\psbezier` curve for which the central node ⟨*A*⟩ is used twice. Positioning parameters (see Sec. 3.2) can be used to shift the object between ⟨*in*⟩ and ⟨*A*⟩ nodes.




```
\begin{pspicture}(3,2)
  \fibercollimator[beam](0,1)(2,1)(3,2){FC}
\end{pspicture}
```

For four nodes, the fiber is drawn as `\psbezier` curve with the specified nodes. Positioning parameters (siehe Sec. 3.2) can be used to shift the object between the first two nodes (⟨*in*⟩ and ⟨*A*⟩).



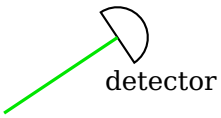

```
\begin{pspicture}(3,2)
  \fibercollimator[beam](0,1)(2,1)(3,1)(3,2){FC}
\end{pspicture}
```

`fibercolsize`=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: `0.3`

A single number gives the side length of the collimator, two numbers the width and height. Note, that `fibercolsize=1` and `fibercolsize=1 1` do not give the same result. You may also change the relation of width and height with `xunit` and `yunit`.

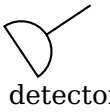
7.3. Detector

`\optdetector`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*label*⟩}




```
\begin{pspicture}(3,1.2)
  \optdetector[beam](0,0)(1.5,1){detector}
\end{pspicture}
```

The detector is placed by default at the end of the reference line (`position`=`end`). If this setting is overwritten, the output connection is treated like that of an electrical component.



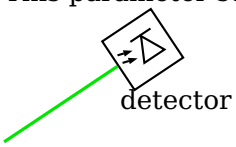

```
\begin{pspicture}(3,1.2)
  \optdetector[position=0.5](0,0)(1.5,1){detector}
\end{pspicture}
```

detsize=⟨*num*⟩ or ⟨*width*⟩ ⟨*height*⟩ default: 0.8

If a single number is given it is the side length (diode) or the diameter (round) of the detector. Two numbers define the width and height of the detector. Note, that detsize=1 and detsize=1 1 are not equivalent for dettype=round.

dettype=round, diode default: round

This parameter selects between different types of detectors.




```
\begin{pspicture}(3,1.5)
  \optdetector[beam, dettype=diode](0,0)(1.5,1){detector}
\end{pspicture}
```

DetectorStyle ⟨*psstyle*⟩

The style of the diode for dettype=diode.

```
\begin{pspicture}(3,1.5)
  \newpsstyle{OptComp}{linewidth=3\pslinewidth}
  \newpsstyle{DetectorStyle}{linewidth=0.333\pslinewidth}
  \optdetector[dettype=diode](0,1)(2,1)
\end{pspicture}
```

8. Special nodes

Every `pst-optexp` object of an experimental setup provides several special nodes which are related to its geometry and positioning. They can be accessed and used for related positioning and drawing.

You should always use the dedicated macros to access these node names.

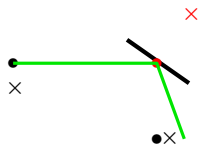
`\oenode`{⟨*node*⟩}{⟨*comp*⟩}

This is the basic command to access any node associated with a certain component. The first argument ⟨*node*⟩ is the identifier of the requested node. The second argument ⟨*comp*⟩ is the name of the target component (according to Sec. 8.1). If left empty, it uses the last component defined.

For most special nodes an appropriate macro is provided, which you are strongly advised to use, because the naming conventions may change. Using the macros makes sure, that you always get the correct node names. Therefore, the available identifiers are not listed explicitely.

`showoptdots`=`true`, `false` default: `false`

Draw some special component nodes for debugging: The black points are the normal and the black crosses are the transformed reference nodes (Sec. 8.2), the red point is the center node (Sec. 8.3), and the red cross is the label node (Sec. 8.4).



```
\begin{pspicture}(3,2)
\mirror[showoptdots, angle=10, beam](0,1)(1.9,1)(1.9,0)
\end{pspicture}
```

8.1. Component identifiers

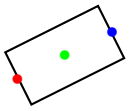
All components of a setup drawing are numbered automatically in increasing order according to their definition in the code, starting with 1. The components as well as their special nodes can always be accessed by this number (ID).

ID specifications relative to the current ID can be used with the syntax <⟨*num*⟩: <1 is the previous ID, <2 is the current ID minus two etc. The previous ID can also be accessed by only <, and the current ID is <0, which is equivalent to an empty parameter.

compname=⟨*string*⟩

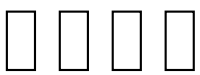
Assigns a name identifier to a component, which can then be referenced both by this name and by its ID. The parameter can be assigned only locally, i.e. in the optional argument of a pst-optexp component, and should be unique within one pspicture environment.

Examples for different methods to access the same component.



```
\begin{pspicture}(2,2)
  \optbox[compname=MyBox](0,1)(2,2)
  \psdot[linecolor=red](\oenodeIn{MyBox})% use the compname
  \psdot[linecolor=blue](\oenodeOut{1}) % use the ID
  \psdot[linecolor=green](\oenodeCenter{}) % the last component
\end{pspicture}
```

Using relative IDs can be very handy to define a component relative to the previous one. It is not possible to use e.g. \oenodeOut{} as reference node, because at the time these coordinates are evaluated, they reference to the new, not yet defined component. A call like \optbox(\oenodeOut{})(B) is, therefore, not possible and gives an error. With relative IDs one can achieve that with \optbox(\oenodeOut{<})(B):



```
\begin{pspicture}(4,2)
  \psset{optboxsize=0.4 0.8, abspos=0.5}
  \optbox(0,1)(4,1)
  \multido{}{3}{\optbox(\oenodeOut{<})(4,1)}
\end{pspicture}
```

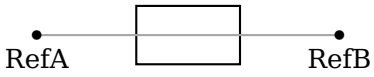
8.2. Reference nodes

\oenodeRefA{⟨comp⟩}

\oenodeRefB{⟨comp⟩}

The input and output reference nodes.

These are the original nodes which were used for the component positioning, \oenodeRefA is the first and \oenodeRefB the last node. This definition is not valid couplers (5.10.3) and circulators (5.9).



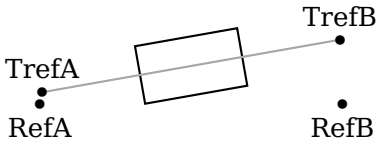

```
\begin{pspicture}(5,1)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox(A)(B)
  \psline[style=Refline](\oenodeRefA{})(\oenodeRefB{})
  \psdot(\oenodeRefA{})\uput[-90](\oenodeRefA{}){RefA}
  \psdot(\oenodeRefB{})\uput[-90](\oenodeRefB{}){RefB}
\end{pspicture}
```

\oenodeTrefA{⟨comp⟩}

\oenodeTrefB{⟨comp⟩}

The transformed input and output reference nodes.

These are the input and output reference nodes which are transformed together with the component according to the compshift and angle parameters.

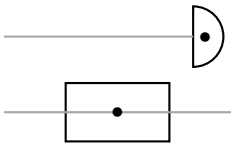



```
\begin{pspicture}(5,1.8)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox[compshift=0.5, angle=10](A)(B)
  \psdot(\oenodeRefA{})\uput[-90](\oenodeRefA{}){RefA}
  \psdot(\oenodeRefB{})\uput[-90](\oenodeRefB{}){RefB}
  \psdot(\oenodeTrefA{})\uput[90](\oenodeTrefA{}){TrefA}
  \psdot(\oenodeTrefB{})\uput[90](\oenodeTrefB{}){TrefB}
\end{pspicture}
```

8.3. Center node

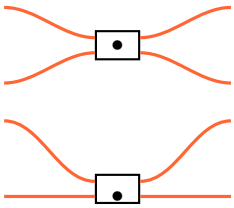
\oenodeCenter{⟨comp⟩}

This node lays, except for a few components like e.g. \optcirculator and \optcoupler (see below), in the center of the component.



```
\begin{pspicture}(3,2)
  \optbox(0,0.5)(3,0.5)
  \optdetector(0,1.5)(2.5,1.5)
\end{pspicture}
```

For \optcoupler the position of the component center depends on the coupleralign settings.

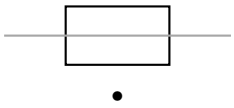


```
\begin{pspicture}(0,0.3)(3,3)
  \psset[optexp]{couplersize=0.6 0.4, couplersep=0.2,
     couplertype=rectangle}
  \optcoupler(0,3)(0,2)(3,3)(3,2)
  \psdot(\oenodeCenter{})
  \optcoupler[coupleralign=b](0,1.5)(0,0.5)(3,1.5)(3,0.5)
  \psdot(\oenodeCenter{})
\end{pspicture}
```

8.4. Label node

\oenodeLabel{⟨comp⟩}

The component label is placed at this node. It is also available if no label was specified. The label node is identical with the center node for labeloffset=0.



```
\begin{pspicture}(3,1.5)
  \optbox(0,1)(3,1)
\end{pspicture}
```

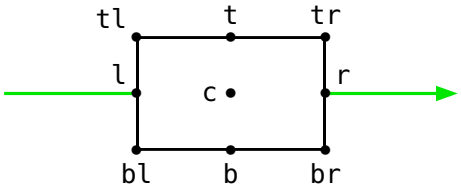
8.5. External nodes

\oenodeExt{⟨comp⟩}

An external node can be placed at different positions along the component boundary. It gets defined only for external usage and does not affect the component in any way.

extnode=⟨*refpoint*⟩

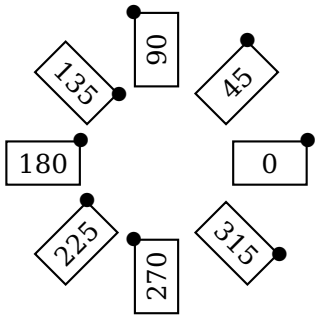
Set the position of the external node. Like the reference point of \rput, can be any comination of c (center), t (top), b (bottom), l (left), and r (right). Not all components support any possible combination. The allowed positions of each component are listed in Sec. 12.2.1.



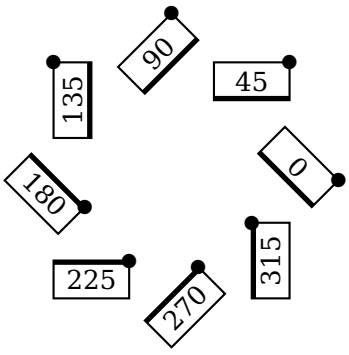
extnodealign=rel, relative, abs, absolute default: abs

The definition of "top" (t) and the other extnode refpoint parameters can be absolute or relative to the component.

In the following example the external node is always placed "top right" independent of the actual order of the reference nodes (extnodealign=abs). The behaviour is identical to the label rotation for labelref=relative.

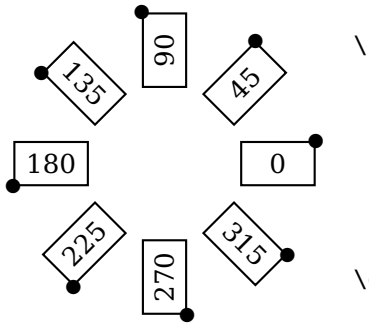



```
\begin{pspicture}(-2,-2)(2,2)
  \psset{endbox, optboxsize=1 0.6, dotscale=1.5}
  \psset{extnodealign=abs, extnode=tr}
  \multido{\i=0+45}{8}{%
    \optbox[innerlabel](0,0)(1;\i){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

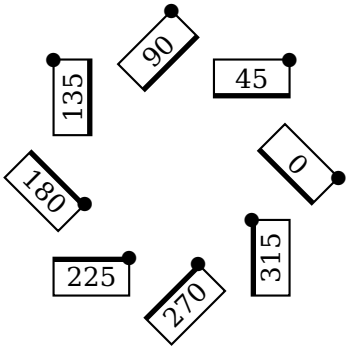



```
\begin{pspicture}(-2.2,-2.2)(2.2,2.2)
  \psset{dotscale=1.5, mirrortype=extended, ↵
      mirrordepth=0.5, label=0.25 . . relative, ↵
      extnodealign=abs, extnode=tr}
  \newpsstyle{ExtendedMirror}{}
  \multido{\i=0+45,\ii=-45+45}{8}{%
    \mirror(0,0)(1.5;\i)(2.1212;\ii){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

In the next example the position is relative to the order of input and output reference node (extnodealign=rel).



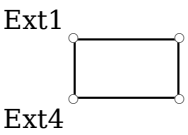

```
\begin{pspicture}(-2,-2)(2,2)
  \psset{endbox, optboxsize=1 0.6, dotscale=1.5}
  \psset{extnodealign=rel, extnode=tr}
  \multido{\i=0+45}{8}{%
    \optbox[innerlabel](0,0)(1;\i){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```




```
\begin{pspicture}(-2.2,-2.2)(2.2,2.2)
  \psset{dotscale=1.5, mirrortype=extended,
      mirrordepth=0.5, label=0.25 . . relative,
      extnodealign=abs, extnode=tr}
  \newpsstyle{ExtendedMirror}{}
  \multido{\i=0+45,\ii=-45+45}{8}{%
    \mirror(0,0)(1.5;\i)(2.1212;\ii){\i}
    \psdot(\oenodeExt{})
  }%
\end{pspicture}
```

extnodes=⟨*list*⟩

Define multiple external nodes, with their ⟨*refpoints*⟩ given in the ⟨*list*⟩ and separated by commas. The first node can be accessed as usual with \oenodeExt (is equivalent to \oenode{Ext} and \oenode{Ext1}). All additional nodes must be accessed with \oenode and the node label Ext⟨*num*⟩: \oenode{Ext1} for the first one, \oenode{Ext2} for the second, etc.




```
\begin{pspicture}(3,2)
  \optbox[extnodes={tl,tr,br,bl}](0,1)(3,1)
  \multido{\i=1+1}{4}{\psdot[dotstyle=o](\oenode{Ext\i}{})}%
\end{pspicture}
```

8.6. Interface nodes

\oenodeIfc{⟨*num*⟩}{⟨*comp*⟩}

The interface node ⟨*num*⟩, where the number is a positive integer or the character N. The last node is always defined as N but can also be accessed via its number.

\oenodeIn{⟨comp⟩}

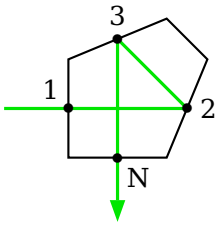
The interface node ⟨1⟩ is the input node and should always be accessed via \oenodeIn.

\oenodeOut{⟨comp⟩}

The interface node ⟨N⟩ is the output node and should always be accessed via \oenodeOut.

"Input" and "output" can define only relative orientations. The input node is by definition the node to which a beam coming from the reference node \oenodeRefA is connected to. Correspondingly is the definition of the output node. This definition breaks down only for fiber couplers (see Sec. 5.10.2) and beamsplitter (4.20).

\oenodeIn is equivalent to \oenodeIfc{1} and \oenodeOut is equivalent to \oenodeIfc{N}. You should use the \oenodeIfc macro only to access the nodes which are not the input and output nodes as for those explicit macros are provided.




```
\begin{pspicture}(2.8,3)
  \pentaprism[pentaprismsize=1.3](0,1.5)(1.5,1.5)(1.5,0)
  \drawbeam[arrows=->, arrowinset=0, arrowscale=1.5](0,1.5){}(1.5,0)
  \psdot(\oenodeIn{})\uput[135](\oenodeIn{}){1}
  \psdot(\oenodeIfc{2}{})\uput[0](\oenodeIfc{2}{}){2}
  \psdot(\oenodeIfc{3}{})\uput[90](\oenodeIfc{3}{}){3}
  \psdot(\oenodeOut{})\uput[-45](\oenodeOut{}){N}
\end{pspicture}
```

See Sec. 12.2.2 for a complete list of all components with their interface nodes.

8.7. Rotation reference node

\oenodeRotref{⟨comp⟩}

The node around which a component is rotated by angle. The position is defined with the rotateref parameter and can take the same values as extnode. See Sec. 12.2.1 for possible rotation reference nodes of all components.

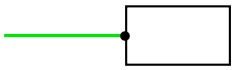
```
\begin{pspicture}(0,0.1)(12,1.8)
  \optbox[angle=20](0.5,1)(4.5,1)
  \psdot(\oenodeRotref{})\uput[-90](\oenodeRotref{}){Rotref}
  \optbox[angle=20, rotateref=bl](7.5,1)(11.5,1)
  \psdot(\oenodeRotref{})\uput[-90](\oenodeRotref{}){Rotref}
\end{pspicture}
```

8.8. Beam nodes

The end points of a recent \drawbeam or \drawwidebeam command can be reused if savebeampoints is set.

\oenodeBeam{⟨*num*⟩}

Access the end point of a preceeding \drawbeam command. If ⟨*num*⟩ is left empty it defaults to 1 which usually refers to the last beam ray. However, generally it depends on the settings of savebeampoints and the choice of ⟨*num*⟩ which beam you refer to.

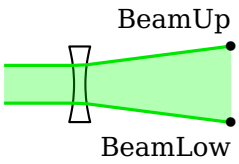


```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(1.6,1){B}
  \optbox[position=end](A)(B)
  \drawbeam[beaminsidelast=false](A){1}
  \psdot(\oenodeBeam{})
\end{pspicture}
```

\oenodeBeamUp{⟨*num*⟩}

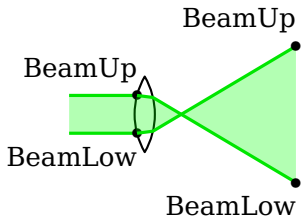
\oenodeBeamLow{⟨*num*⟩}

Access the upper or lower end point of a preceeding \drawwidebeam command. If ⟨*num*⟩ is left empty it defaults to 1, which usually refers to the last beam ray. However, generally it depends on the settings of savebeampoints and the choice of ⟨*num*⟩ which beam you refer to.



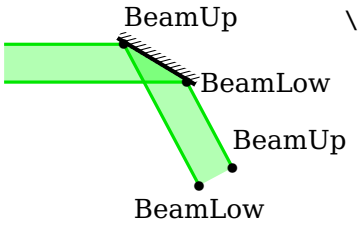

```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=1, lensradius=-2](A)(B)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}(B)
\end{pspicture}
```

Note, that “upper” and “lower” is defined with respect to the propagation direction of the beam. A marginal beam which started as “upper” can change to “lower” if it crosses the other marginal beam. This is shown in the following example, where the marginal rays exchange their roles:




```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=1, n=2](A)(B)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}
  \drawwidebeam[loadbeam, beaminsidefirst]{}(B)
\end{pspicture}
```

After a reflection the roles can also be exchanged.




```
\begin{pspicture}(0,-1)(4,2)
  \pnodes(0,1.5){A}(2,1.5){B}(2.8,0){C}
  \mirror[mirrorwidth=1.2, mirrortype=extended](A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.5](A){}
  \drawwidebeam[loadbeam]{}(C)
\end{pspicture}
```

Note, that the orientation of “upper” and “lower” changed with version 3.3, for more details see Sec. 12.3.5.

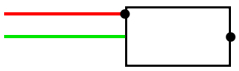
`\oeBeamCenter`{⟨*num*⟩}

Get the mean of upper and lower beam node. The coordinates are available only at Postscript level, in contrast to the `\oenodeBeam`∗ macros. For an example see Ex. 8.9.

The beam nodes can be accessed only within the `pspicture` environment they were defined in. If you want to use one of these nodes from outside, you must explicitely redefine them beforehand:

```
\pnode(\oenodeBeam{}){MyBeamNode}
```

The end points are also affected by stopinside. See Sec. 9.5 for further details.



```
\begin{pspicture}(0,0)(3,2)
  \pnodes(0,1){A}(1.6,1){B}
  \optbox[position=end](A)(B)
  \psset{beaminsidelast=false}
  \drawbeam[linecolor=red, beampos=0.3](A){1}
  \psdot(\oenodeBeam{})
  \drawbeam[stopinside](A){1}
  \psdot(\oenodeBeam{})
\end{pspicture}
```

8.9. Beam vector

In addition to the beam nodes (see Sec. 8.8) also the normalized vectors of the last saved beams (see savebeam) can be accessed at Postscript level.

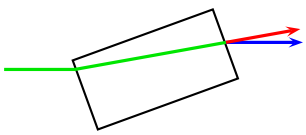
These are again separated in single, upper and lower beams. ⟨*num*⟩ has the same meaning as for \oenodeBeam.

\oeBeamVec{⟨*num*⟩}

\oeBeamVecUp{⟨*num*⟩}

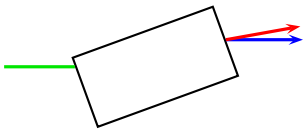
\oeBeamVecLow{⟨*num*⟩}

The saved values are those of the normalized direction vector of the last beam segment and are drawn in red in the next example. In contrast to this, the vector saved with savebeam is that behind the last interface and would be used for a following beam. This is shown in blue in the following example.



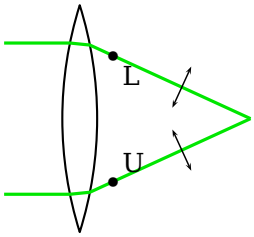
```
\begin{pspicture}(4,2)
  \optbox[optboxsize=2 1, angle=20, n=2](0,1)(4,1)
  \drawbeam[beaminsidelast, beamalign=abs](0,1){1}
  \pnode(\oenodeBeam{}){Out}
  \nodexn{(Out)+(!\oeBeamVec{})}{C}
  \drawbeam[linecolor=blue, arrows=->, loadbeam]{1}(C)
  \drawbeam[linecolor=red, arrows=->](Out)(C)
\end{pspicture}
```

This behaviour is independent of whether the last beam was only calculated or also drawn. The next example uses `stopinside` instead of `beaminsidelast` compared to the previous example; the beam vectors are not affected by this.



```
\begin{pspicture}(4,2)
  \optbox[optboxsize=2 1, angle=20, n=2](0,1)(4,1)
  \drawbeam[stopinside, beamalign=abs](0,1){1}
  \pnode(\oenodeBeam{}){Out}
  \nodexn{(Out)+(!\oeBeamVec{})}{C}
  \drawbeam[linecolor=blue, arrows=->, loadbeam]{1}(C)
  \drawbeam[linecolor=red, arrows=->](Out)(C)
\end{pspicture}
```

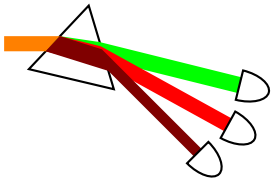
A possible application is marking of the polarization of a beam. In the following example we define the nodes $\langle U \rangle$ and $\langle L \rangle$ which are shifted with `\nodexn` from the end point of the upper and lower beam along the respective beam trajectory.




```
\begin{pspicture}(3,3)
  \pnodes(0,1.5){A}(3,1.5){B}
  \lens[abspos=1, lens=5 5 3, n=2.0](A)(B)
  \optplane[angle=90](B)
  \drawwidebeam[beamwidth=2](A){1-2}
  \nodexn{(\oenodeBeamUp{})-2(!\oeBeamVecUp{})}{U}
  \nodexn{(\oenodeBeamLow{})-2(!\oeBeamVecLow{})}{L}
  \polarization(U)(\oenodeBeamUp{})
  \polarization(L)(\oenodeBeamLow{})
\end{pspicture}
```

`\oeBeamVecMedian{`$\langle num \rangle$`}`The mean of upper and lower beam vector.

Ex. 8.1

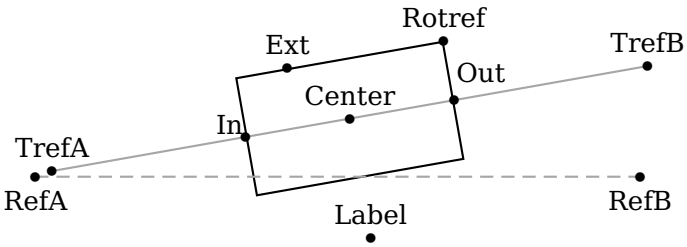


```
\begin{pspicture}(3.5,2.6)
  \pnodes(0,2){A}(1,2){B}(3,1){C}
  \optprism(A)(B)(C)
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid,
      beamwidth=0.2, beamnodealign=vector}
  \drawwidebeam[fillcolor=orange](A){}
  \psset[optexp]{beampathskip=1}
  \drawwidebeam[fillcolor=green, n=1.2, savebeam=2](A){}(C)
  \drawwidebeam[fillcolor=red, n=1.4, savebeam=3](A){}(C)
  \drawwidebeam[fillcolor=red!50!black, n=1.57, savebeam=4](A){}(C)
  \multido{\i=2+1}{3}{%
    \backlayer{%
      \pnode(!\oeBeamCenter{\i}){Det\i}
      \nodexn{(Det\i)-(!\oeBeamVecMedian{\i})}{Det\i'}}
    \optdetector[detsize=0.4 0.4](Det\i')(Det\i)}
\end{pspicture}
```

Further example are Ex. 11 and Ex. 11.

8.10. Node overview

This is an overview of all available component nodes.




```
\begin{pspicture}(0.3,0.1)(4.8,1.6)
  \pnodes(0.5,0.5){A}(4.5,0.5){B}
  \optbox[compshift=0.5, angle=10, rotateref=tr, extnode={-0.5,1}](A)(B)
\end{pspicture}
```

9. Connecting components

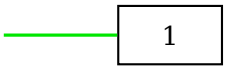
The `pst-optexp` package provides several methods for automatic and manual connections of components with beams, fibers, or wires.

- Sec. 9.1 describes how to access objects and nodes for use with the connection macros.
- Beam drawing in general and the behavior of single beams is explained in Sec. 9.2.
- Sec. 9.3 extends this description of beam drawing to wide beams and their special properties.
- Sec. 9.4 shows how errors in beam connections (missed interfaces etc.) are handled.
- Sec. 9.5 explains how customized, piecewise defined beam paths, like e.g. in a telescope, can be easily constructed.
- Manual fiber connections are described in Sec. 9.6, manual wire connections, which show the same behavior, are treated in Sec. 9.7.
- Automatic connections are explained in Sec. 9.8.
- Sec. 9.9 introduces the concept of front and back layer for the drawings.

9.1. Accessing components

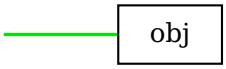
All macros for connections can take either an ID, a `compname` identifier (see Sec. 8.1) or PSTricks nodes as arguments, which are denoted with $\langle obj_1 \rangle$, $\langle obj_2 \rangle$, ... To distinguish between nodes and components, nodes must be either enclosed in parenthesis within the brackets, or enclosed only in parenthesis: `\drawbeam{(node)}{comp}`, or `\drawbeam(node){comp}`.

1. All components can always be accessed via their ID.



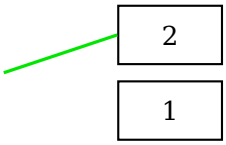

```
\begin{pspicture}(3,1.5)
  \pnodes(0,1){A}(1.5,1){B}
  \optbox[position=end, labeloffset=0](A)(B){1}
  \drawbeam(A){1}
\end{pspicture}
```

2. A component which has been assigned a name with `compname` can also be accessed via this instead of its ID.




```
\begin{pspicture}(3,2)
  \psset{position=end, labeloffset=0}
  \optbox[compname=obj](0,1)(1.5,1){obj}
  \drawbeam(0,1){obj}
\end{pspicture}
```

3. If the argument is left empty, use the last component defined.




```
\begin{pspicture}(3,2)
  \psset{position=end, labeloffset=0}
  \optbox(0,0.5)(1.5,0.5){1}
  \optbox(0,1.5)(1.5,1.5){2}
  \drawbeam(0,1){}
\end{pspicture}
```

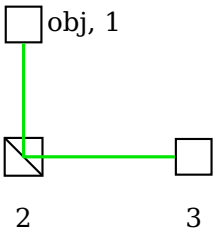
4. You can also use number ranges, valid range specifications are

   x-y From x to y, if x is greater than y, the numbers are decremented. If the whole range is not defined, an error is raised.

   x- From x to the last component.

   -y From the first component to y. If y is outside the valid range, a warning is raised.

   - Connect all components.




```
\begin{pspicture}(3,3)
  \psset{position=end, optboxsize=0.5 0.5, bssize=0.5}
  \optbox[compname=obj](0.5,0.5)(0.5,2.5){obj, 1}
  \beamsplitter(0.5,2.5)(0.5,1)(2.5,1){2}
  \optbox(0.5,1)(2.5,1){3}
  \drawbeam{1-3}
\end{pspicture}
```

9.2. Drawing beams

The package provides several ways to connect components and nodes with beams, which can be either single rays or wide beams. Most parameters have the same effect on both single and wide beams so most examples will be shown for single beams only. For options specific to wide beams please see Sec. 9.3.

`\drawbeam`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

`\drawwidebeam`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

These are the two macros for beam connections; they can take a variable number of arguments (minimum of two) which can be either an ID, an ID range, a `compname`, or a PSTricks nodes (see Sec. 9.1). To distinguish between nodes and components, nodes must be either enclosed in parenthesis within the brackets, or enclosed only in parenthesis: `\drawbeam{(node)}{comp}`, or `\drawbeam(node){comp}`.

9.2.1. Raytracing

The beam connection macros support two modes of tracing the beam across the optical components: one is to use refractive indices and Snell's law to determine the beam path. The other mode is to connect the components only, regardless of their optical properties.

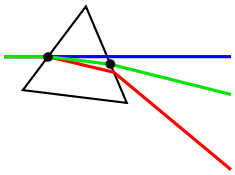
You must always keep in mind, that this is a package for *sketching* experimental setups. Therefore, we do not provide a comprehensive raytracing framwork, because deviations from the actual physical path are often desired for sketches: beam angles and divergences should be in many cases more extreme than in the real setup in order to highlight certain aspects. You should also consider the refractive index (see Sec. 9.2.2) only as a tool to optimize the visual effects of the beam path, and not stick with its physically correct values.

`raytrace`=`true`, `false` default: `true`

Chooses the mode of tracing the beam path, `true` uses the refractive index ("raytracing"), `false` selects component connection ("connect" mode).

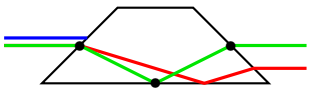
In most cases, the "connect" mode is equivalent to "raytracing" with `n`=1, except for `\optprism` and `\doveprism`, where the interface nodes (Sec. 8.6) are simply connected. See the following two example for the differences between the tracing modes using these two components. The interface nodes are highlighted for clarity.

The raytraced beam with n=1 (blue) passes the component without changing its direction, the refraction for n=1.5 is calculated (red), and the “connect” beam (green) just passes through the two interface nodes.



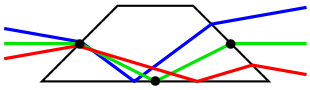
```
\begin{pspicture}(3,2.2)
  \pnodes(0,1.5){A}(1,1.5){B}(3,1){C}
  \optprism[prismsize=1.2](A)(B)(C)
  \optplane(C)
  \drawbeam[n=1, linecolor=blue](A){1}{2}
  \drawbeam[n=1.5, linecolor=red](A){1}{2}
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}{2}
\end{pspicture}
```

For the \doveprism the same applies as for the \optprism, but the raytraced beam with n=1 (blue) misses the second interface because it is not refracted and stops therefore at the first interface (see Sec. 9.4). Again, the green “connect” ray passes through all interface nodes.

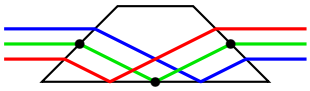


```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \drawbeam[n=1, linecolor=blue, beampos=0.1](A){1}(B)
  \drawbeam[n=1.5, linecolor=red](A){1}(B)
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}(B)
\end{pspicture}
```

The “connect” mode also supports changing the initial conditions (9.2.3).



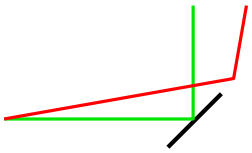
```
\begin{pspicture}(3.7,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \psset[optexp]{raytrace=false}
  \drawbeam[linecolor=blue, beampos=0.2, beamangle=-10](A){1}(B)
  \drawbeam[raytrace=false, linecolor=DGreen](A){1}(B)
  \drawbeam[linecolor=red, beampos=-0.2, beamangle=10](A){1}(B)
\end{pspicture}
```



```
\begin{pspicture}(3.7,2)
  \pnodes(0,1){A}(4,1){B}
  \doveprism[doveprismsize=1](A)(B)
  \psset[optexp]{raytrace=false}
  \drawbeam[linecolor=blue, beampos=0.2](A){1}(B)
  \drawbeam[linecolor=DGreen](A){1}(B)
  \drawbeam[linecolor=red, beampos=-0.2](A){1}(B)
\end{pspicture}
```

useNA=true, false default: true

The beam path is interrupted if an interface is missed. In some cases, e.g. to estimate the correct refractive index for a ray trace, it can be useful if the numerical aperture of the components is not considered. The red beam in the following example is drawn, although it hits the mirror outside the drawn mirror surface.



```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(2.5,0.5){B}(2.5,2){C}
  \mirror(A)(B)(C)
  \psset{useNA=false}
  \drawbeam(A){}(C)
  \drawbeam[linecolor=red, beamangle=10](A){}(C)
\end{pspicture}
```

9.2.2. Refractive index

The raytracing depends on the refractive index, which may be set for each component separately. The refractive index is always relative to the background which has a constant value for the whole sketch.

n=⟨*code*⟩ default: 1.5

Sets the relative refractive index respect to the background. This parameter has two scopes, you can either set the refractive index of each component or of each beam ray:

1. Using n as global parameter or component argument sets a fixed refractive index for each affected component. Without further actions, all beam paths experience this refractive index.
2. As parameter for \drawbeam or \drawwidebeam you can overwrite or even change the refractive index of the components for specific beam paths. This can be very useful e.g. to simulate chromatic dispersion. To set this globally for all beams you must add the respective parameter definition to the Beam style because global definitions of n affect only the components and not the beams.

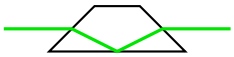
The ⟨*code*⟩ can in general be any Postscript code which evaluates to a number, e.g. 1.5, or 5 sqrt. If the ⟨*code*⟩ starts with a ∗, it will be interpreted as an algebraic expression[1], e.g. ∗sqrt(5) is equivalent to 5 sqrt. Inside the ⟨*code*⟩ you can access

[1] http://mirror.ctan.org/graphics/pstricks/base/doc/pst-news08.pdf

the predefined refractive index of the component via `n` which allows you to change the index relative to its original value. Use e.g. `n=n 1.01 mul` to change `n` by one percent.

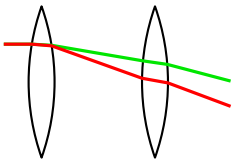
In the following you find several examples about the different aspects of using `n`.

1. Set a fixed refractive index for the component. Interestingly, for the default shape of the `\doveprism` a refractive index of $\sqrt{5}$ gives the ideal output direction which is the same as the input direction.



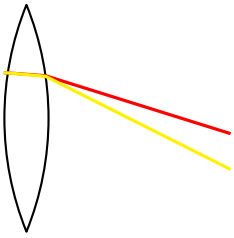
```
\begin{pspicture}(3,1.5)
  \pnodes(0,1){A}(3,1){B}
  \doveprism[n=*sqrt(5)](A)(B)
  \drawbeam(A){}(B)
\end{pspicture}
```

2. In this example one beam uses the default refractive index of the components (green), the other one overwrites it with a value of 2 (red).



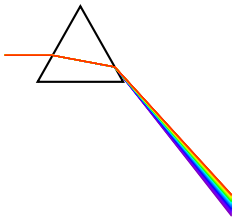
```
\begin{pspicture}(3,3)
  \pnodes(0,1){A}(3,1){B}
  \optplane(A)
  \psset{lens=3 3 2, n=1.5}
  \lens[abspos=0.5](A)(B)
  \lens[abspos=2](A)(B)
  \optplane(B)
  \addtopsstyle{Beam}{beampos=0.5}
  \drawbeam{-}
  \drawbeam[linecolor=red, n=2]{-}
\end{pspicture}
```

3. Now, the red beam uses the default refractive index of the component, the yellow beam adds `0.5` to the default index.



```
\begin{pspicture}(3,3)
  \pnodes(0,1.5){A}(3,1.5){B}
  \lens[lens=4 4 3, n=2, abspos=0.3](A)(B)
  \optplane[compname=Plane](B)
  \psset{beampos=0.6}
  \drawbeam[linecolor=red](A){1}{Plane}
  \drawbeam[linecolor=yellow, n=*(n+0.5)](A){1}{Plane}
\end{pspicture}
```

4. Dynamically changing the refractive index is very useful to emulate chromatic dispersion. Here, we sketch the dispersion of a prism by calculating the refractive index with Sellmaier's equation[2] for different wavelengths.



```
\begin{pspicture}(3,3)
  \pnodes(-1,0){A}(1,2.2){B}(3,0){C}
  \optplane(0,2.15)
  \optprism[prismalign=center, prismangle=59](A)(B)(C)
  \optplane(C)
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin tx@OptexpDict begin
      Wavelength wavelengthToRGB
      Red Green Blue
    end end }%
  \addtopsstyle{Beam}{linecolor=bl,
      linewidth=0.4\pslinewidth, beamalign=abs}
  \multido{\iLambda=400+4}{60}{%
    \pstVerb{/Wavelength \iLambda\space def }%
    \drawbeam[n=Wavelength Sellmaier]{-}%
  }%
\end{pspicture}
```

5. Using the numerical aperture of a pinhole as spectral filter.

```
\begin{pspicture}(0.3,0)(3.3,5.2)
  \pnodes(0.3,1){A}(2,1){B}(3,2){C}(2.5,5){D}
  \optplane(A)
  \optgrating[reverse](A)(B)(C)
  \pinhole[phwidth=-0.1, innerheight=0.03, abspos=0.5](D)(B)
  \optplate[position=end, plateheight=1.5](B)(D)
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin Red Green Blue end}%
  \drawbeam[linecolor=red]{1-2}
  \addtopsstyle{Beam}{linecolor=bl, linewidth=0.4\pslinewidth}
  \multido{\i=0+1}{60}{%
    \pstVerb{%
      \i\space 650 400 sub 59 div mul 400 add
      tx@addDict begin wavelengthToRGB end }%
    \drawbeam[beamangle=\i\space 0.1 mul 3 sub]{2-}%
  }%
\end{pspicture}
```

[2] http://en.wikipedia.org/wiki/Sellmeier_equation

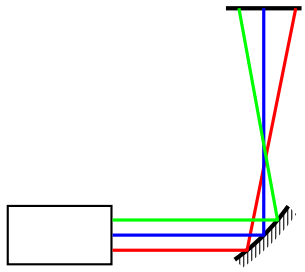
refractiveindex=⟨*code*⟩

This parameter is deprecated since version 3.0, use n instead.

9.2.3. Initial conditions

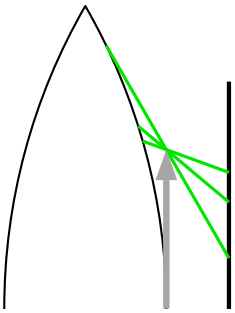
beampos=[⟨*x*⟩]⟨*y*⟩ default: 0

This is the start position (⟨*x*⟩, ⟨*y*⟩) of the beam at the first interface. Both values are of ⟨*psnum*⟩ type. If a single number is given, the x-coordinate is set to 0.



```
\begin{pspicture}(4,3.8)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended, mirrorradius=3](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawbeam[linecolor=red, beampos=-0.2]{1-3}
  \drawbeam[linecolor=blue, beampos=0]{1-3}
  \drawbeam[linecolor=green, beampos=0.2]{1-3}
\end{pspicture}
```

The start position is relative to the respective interface node, as it is illustrated in the following example. The beam start position is fixed, but depending on e.g. the beamangle, the actual intersection with the curved interface is at a different position.

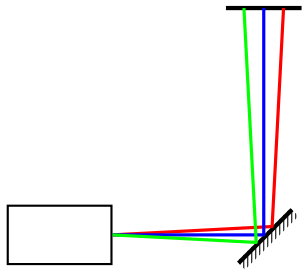


```
\begin{pspicture*}(0,2)(3.05,6)
  \lens[lens=8 8 8, abspos=0.6](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{beampos=2.1}
  \drawbeam[beamangle=-60]{1}{2}
  \drawbeam[beamangle=-40]{1}{2}
  \drawbeam[beamangle=-20]{1}{2}
\end{pspicture*}
```

Which interface the beam starts at is determined by the options beaminsidefirst and startinside.

beamangle=⟨*psnum*⟩ default: 0

This is the start angle of the beam. Usually, it is relative to the connection between the first two components, use beamalign to change this.

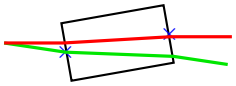


```
\begin{pspicture}(4,4)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawbeam[linecolor=red, beamangle=3]{-3}
  \drawbeam[linecolor=blue, beamangle=0]{1-}
  \drawbeam[linecolor=green, beamangle=-3]{-}
\end{pspicture}
```

`beamalign`=`rel`, `relative`, `abs`, `absolute`, `firstcomp` default: `relative`

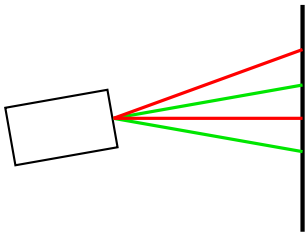
Select whether the `beamangle` is relative to the connection between the first two components, or if it is an absolute angle.

A typical situation where an absolute angle is helpful is with rotated or shifted components. By rotating or shifting a component also its interface nodes are rotated (see Sec. 3.3) which are then used to determine the input angle for relative alignment. Accordingly, in the following example, the green beam with the relative alignment is not horizontal.



```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.75){A}(3,0.75){B}
  \optbox[angle=10, showifcnodes](A)(B)
  \drawbeam(A){}(B)
  \drawbeam[linecolor=red, beamalign=abs](A){}(B)
\end{pspicture}
```

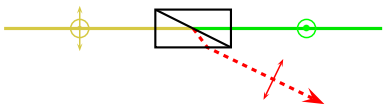
Both components in the following example are align on the horizontal line $\overline{AB}$. The green beam with `beamalign`=`rel` is accordingly aligned horizontally. The red beam with `beamalign`=`firstcomp` is aligned according to the rotation of the first component:



```
\begin{pspicture}(4,3)
  \pnodes(1.5,1.5){A}(4,1.5){B}
  \optbox[position=start, rotateref=r, angle=10](A)(B)
  \optplate[position=end, plateheight=3](A)(B)
  \addtopsstyle{Beam}{beamdiv=20}
  \drawwidebeam{-}
  \drawwidebeam[beamalign=firstcomp, linecolor=red]{-}
\end{pspicture}
```

beampathskip=⟨*num*⟩ default: 0

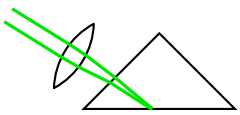
Skip ⟨*num*⟩ segments at the beginning of the beam path. This option determines only at which segment the beam is drawn, the ray tracing always starts at the first interface. To change this, see option startinsidecount.



```
\begin{pspicture}(5,2)
\pnodes(0,1){A}(5,1){B}(2,0){C}
\begin{optexp}
  \glanthompson(A)(B)
  \optplane(B)\optplane[angle=90](C)
  \polarization[abspos=1, poltype=misc, linecolor=yellow!80!black](A)(B)
  \drawbeam[beaminsidelast, linecolor=yellow!80!black](A){1}
  \drawbeam[beampathskip=2](A){1-2}
  \drawbeam[beampathskip=2, savebeam, linestyle=dashed, dash=2pt 2pt, ↵
      linecolor=red,arrowscale=1.5, arrows=->](A){1}{3}
\end{optexp}
\nodexn{(\oenodeBeam{})-1.5(!\oeBeamVec{})}{D}
\polarization[poltype=parallel, linecolor=red](D)(\oenodeBeam{})
\polarization[abspos=4, poltype=perp, linecolor=green, dotscale=1.5](A)(B)
\end{pspicture}
```

beampathcount=⟨*num*⟩ default: -1

Number of beam segments, which are drawn. Segments which are skipped at the beam start with beampathskip do not count.



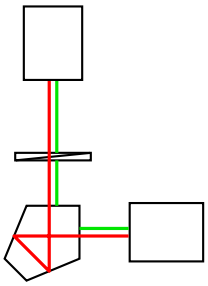
```
\begin{pspicture}(3,2)
  \pnodes(0,2){A}(2,0.8){B}(4,2){C}
  \optprism[compname=P, prismtype=reflective, prismangle=90](A)(B)(C)
  \lens[abspos=1.33](\oenodeIfc{2}{P})(A)%
  \drawwidebeam[beamwidth=0.2, beaminsidelast, ↵
      beampathcount=4](A){2}{1}
\end{pspicture}
```

9.2.4. Internal beam path

A beam path is composed of rays between components and rays inside the components. Depending on some options these internal beams can be drawn or not. Generally, the first, the last, and all other components are treated separately.

`beaminside`=`true`, `false` default: `true`

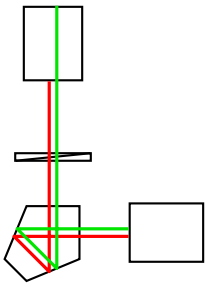
Draw the internal beams in all intermediate components, the first and last components are not affected.



```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminside=false]{-}
\end{pspicture}
```

`beaminsidefirst`=`true`, `false` default: `false`

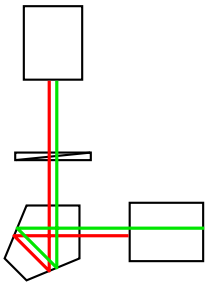
Draw the internal beams of the first component.



```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminsidefirst]{-}
\end{pspicture}
```

`beaminsidelast`=`true`, `false` default: `false`

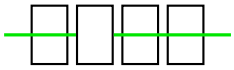
Draw the internal beams of the last component.



```
\begin{pspicture}(0.3,0.3)(3,4)
  \psset{optboxwidth=1}
  \optbox[position=end](1,1)(1,3)
  \optretplate(1,1)(1,3)
  \pentaprism(1,3)(1,1)(2,1)
  \optbox[position=end](1,1)(2,1)
  \drawbeam[beampos=-0.05, linecolor=red]{-}
  \drawbeam[beampos=0.05, beaminsidelast]{-}
\end{pspicture}
```

allowbeaminside=true, false default: true

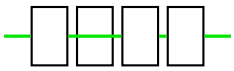
Define whether a beam is drawn inside a component. The option affects components only and overwrites all settings of beaminside, beaminsidefirst, and beaminsidelast if set to false. With this you can forbid any internal beams for components where it does not make any sense (e.g. \optfilter and \optdiode).



```
\begin{pspicture}(3,2)
  \psset{optboxwidth=0.5}
  \pnodes(0,1){A}(3,1){B}
  \optbox[position=0.2](A)(B)
  \optbox[position=0.4, allowbeaminside=false](A)(B)
  \optbox[position=0.6](A)(B)
  \optbox[position=0.8](A)(B)
  \drawbeam(A){-}(B)
\end{pspicture}
```

forcebeaminside=true, false default: false

Forces to draw a beam inside a component. The option affects components only and overwrites all settings of allowbeaminside, beaminside, beaminsidefirst, and beaminsidelast if set to true. With this you can force an internal beams for single components within a beam path.



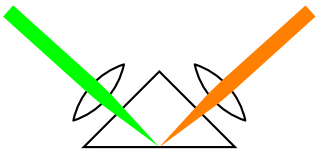
```
\begin{pspicture}(3,2)
  \psset{optboxwidth=0.5}
  \pnodes(0,1){A}(3,1){B}
  \optbox[position=0.2](A)(B)
  \optbox[position=0.4, forcebeaminside](A)(B)
  \optbox[position=0.6](A)(B)
  \optbox[position=0.8](A)(B)
  \drawbeam[beaminside=false](A){-}(B)
\end{pspicture}
```

startinsidecount=⟨num⟩ default: -1

Number of segments, which are used inside the first component. This option is in effect only with beaminsidefirst or startinside set to true.

stopinsidecount=⟨num⟩ default: -1

Number of segments, which are used inside the last component. This option is in effect only with beaminsidelast or stopinside set to true.



```
\begin{pspicture}(4,2)
  \pnodes(0,2){A}(2,0){B}(4,2){C}
  \optprism[compname=P, n=1, prismtype=reflective,
      prismangle=90](A)(B)(C)
  \lens[abspos=1.08](\oenodeIfc{2}{P})(A)
  \lens[abspos=1.08](\oenodeIfc{2}{P})(C)
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid}
  \drawwidebeam[fillcolor=green, beamwidth=0.2,
      beaminsidelast, stopinsidecount=1](A){2}{1}
  \drawwidebeam[fillcolor=orange, beaminsidefirst,
      startinsidecount=1, loadbeam]{1}{3}(C)
\end{pspicture}
```

beammode=refl, trans, reflective, transmittive, auto default: auto

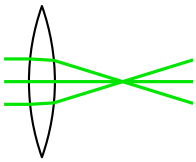
With this option one can force the beam path through e.g. a \beamsplitter to reflective or transmittive independent of the choosen nodes and their position relative to the component.

9.2.5. Connecting with nodes

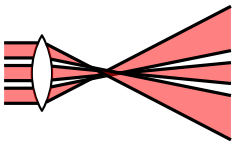
As mentioned earlier in this chapter, beams can connect components also with nodes. For raytracing you need a plane, which can be created automatically or manually in different ways.

beamnodealign=vec, conn, vector, connection default: connection

This parameter controls the alignment of the automatically created node interfaces. By default (connection) the interface is aligned perpendicular to the direct connection between node and the previous interface node.

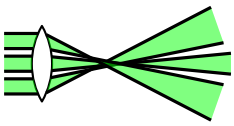


```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.5,1){B}
  \lens[lens=3 3 2, n=2.5, abspos=0.5](A)(B)
  \multido{\r=-0.3+0.3}{3}{%
    \drawbeam[beampos=\r](A){}(B)}
\end{pspicture}
```



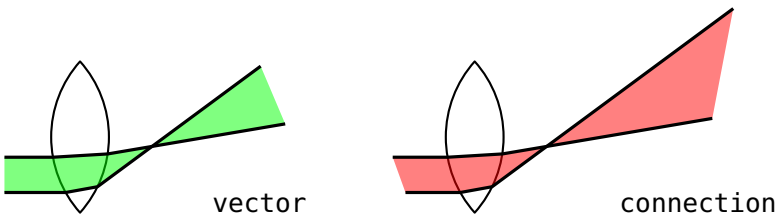
```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=0.5](A)(B)
  \addtopsstyle{Beam}{linecolor=black, fillstyle=solid,
      beamwidth=0.2, fillcolor=red, opacity=0.5, beaminside=false}
  \multido{\r=-0.3+0.3}{3}{\drawwidebeam[beampos=\r](A){}(B)}
\end{pspicture}
```

For a value `vector`, the plane is aligned vertical to the incoming beam vector. For wide beams the mean beam vector is used (see `\oeBeamVecMedian`). Examples where this is very convenient are Ex. 8.9 and Ex. 11.



```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \lens[abspos=0.5](A)(B)
  \addtopsstyle{Beam}{linecolor=black, fillstyle=solid,
      beamwidth=0.2, beamnodealign=vector, fillcolor=green,
      opacity=0.5, beaminside=false}
  \multido{\r=-0.3+0.3}{3}{\drawwidebeam[beampos=\r](A){}(B)}
\end{pspicture}
```

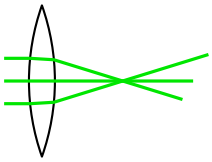
The automatic alignment is very handy in many cases, but does not allow to rotate the node planes to a fixed angle and can give again problems with shifted and rotated components (see Sec. 3.3). In the following example the green beam is with `beamnodealign` set to `vector`, for the red beam it is set to `connection`:



For more flexibility, the package provides an “invisible” plane which can be used for this purpose.

\optplane(⟨center⟩)

The plane is defined by a single node only, the rotation around this node is defined with angle. The angle is with respect to the default alignment, which is vertical.



```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(2.5,1){B}
  \lens[lens=3 3 2, n=2.5, abspos=0.5](A)(B)
  \optplane[angle=-30](B)
  \multido{\r=-0.3+0.3}{3}{%
    \drawbeam[beampos=\r](A){-}}
\end{pspicture}
```

9.2.6. Automatic connections

Beam can also be drawn at definition time of a component, which can be handy, but allows only very elemental beams.

beam=true, false default: false

Can be used as option for every component, is equivalent to the statement

```
\drawbeam[raytrace=false](\oenodeRefA{}){}(\oenodeRefB{})
```

which draws a beam from one reference node via the component to the other reference node.

9.2.7. Beam appearance

Beam ⟨psstyle⟩ default: linecolor=green!90!black, linejoin=1

The appearance of beams is controlled with this style. The optional argument of \draw*beam can also be used, these options can overwrite settings from the Beam style.

addtoBeam=⟨list⟩

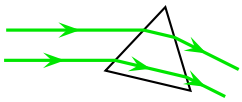
The Beam style is extended by locally adding the parameters contained in ⟨list⟩. The ⟨list⟩ must be surrounded by curly braces.

newBeam=⟨*list*⟩

Similar to addtoBeam, but an existing Beam style is overwritten with the new parameter set.

ArrowInsideMinLength=⟨*psnum*⟩ default: 0.2

\draw∗beam uses also the option ArrowInside from pstricks-add. In this case the arrows would be drawn for every beam segment, which is often undesired for the internal, short beam paths. ArrowInsideMinLength sets a minimum segment length, below which no internal arrows are drawn. In the following example the value is selected such that only the lower internal beam has an arrow (see also Ex. 11).

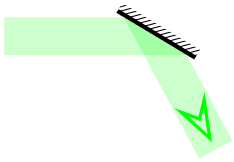


```
\begin{pspicture}(3,2)
  \optprism(0,1)(2,1)(3,0.5)
  \drawwidebeam[beamwidth=0.4, ArrowInside=->,
      ArrowInsideMinLength=0.5, arrowscale=1.5,
      raytrace=false](0,1){}(3,0.5)
\end{pspicture}
```

ArrowInsideMaxLength=⟨*psnum*⟩ default: -1

Like ArrowInsideMaxLength, but sets the maximum length of a segment for which the arrows are drawn. If the value is negative, no upper limit is defined.

ArrowInside can be used also without a linestyle.



```
\begin{pspicture}(0,0)(3.5,2)
  \pnodes(0,1.5){A}(2,1.5){B}(2.8,0){C}
  \mirror[mirrorwidth=1.2, mirrortype=extended](A)(B)(C)
  \newpsstyle{Beam}{linecolor=green, linestyle=none,
      fillstyle=solid, fillcolor=green, opacity=0.2}
  \drawwidebeam[beamwidth=0.5](A){}(C)
  \drawbeam[ArrowInside=->, arrowscale=2.5, ArrowFill=false,
      linewidth=1.5\pslinewidth, ArrowInsidePos=0.9,
      ArrowInsideMaxLength=1.8](A){}(C)
\end{pspicture}
```

fade ⟨*linestyle*⟩

Fade from the current linecolor to another color, defined by fadeto. The line style currently works only for line, not for curves. The complete path is divided in fadepoints−1 segments.

```
\begin{pspicture}(3,1)
  \psline[linewidth=0.2, linecolor=red, linestyle=fade,
      fadepoints=50](0,0.5)(3,0.5)
\end{pspicture}
```

fadeto=white, black, transparency default: white

The color at the fade end, depends on fadefuncname and fadefunc.

```
\begin{pspicture}(3,1)
  \psset{linewidth=0.2, linecolor=red, linestyle=fade, fadepoints=50}
  \psline[fadeto=white](0,0.9)(3,0.9)
  \psline[fadeto=black](0,0.5)(3,0.5)
  \psline[fadeto=transparency](0,0.1)(3,0.1)
\end{pspicture}
```

fadepoints=⟨*num*⟩ default: 200

Number of points used for lines with style fade.

fadefuncname=gauss, linear, squared, exp, custom default: linear

Some predefined fade functions, which determine the color gradient. If set to custom, the Postscript code in fadefunc is used.

fadefunc=⟨*PS code*⟩

Define a custom gradient for line style fade. The ⟨*PS code*⟩ must evaluate a number on the stack in the range $[0,1]$ (0: path start, 1: path end) to $[0,1]$ (0: fadeto color, 1: current line color). The function values are clipped to $[0,1]$.

```
\begin{pspicture}(3,1)
  \psset{linewidth=0.2, linecolor=red, linestyle=fade, fadepoints=50}
  \psline[fadefuncname=gauss](0,1)(3,1)
  \psline[fadefuncname=exp](0,0.7)(3,0.7)
  \psline[fadefuncname=custom, fadefunc={360 mul cos 1 add 0.5
      mul}](0,0.4)(3,0.4)
  \psline[fadefuncname=custom, fadefunc={neg 1 add 2
      mul}](0,0.1)(3,0.1)
\end{pspicture}
```

For a nice example see Ex. 11.

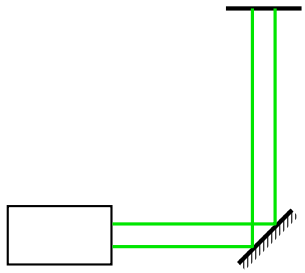
9.3. Drawing wide beams

All information about single beams (see Sec. 9.2) apply to wide beams as well. Here, you find all additional information which apply to wide beams only.

`\drawwidebeam`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

`beamwidth`=⟨*psnum*⟩ default: `0`

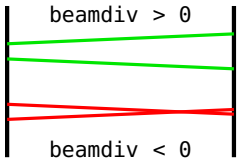
The initial beam width, measured at the initial position (defined with `beampos`) perpendicular to the beam direction.



```
\begin{pspicture}(4,4)
  \pnodes(1.5,0.5){A}(3.5,0.5){B}(3.5,3.5){C}
  \optbox[position=end](B)(A)
  \mirror[mirrortype=extended](A)(B)(C)
  \optplate[position=end](B)(C)
  \drawwidebeam[beamwidth=0.3]{-}
\end{pspicture}
```

`beamdiv`=⟨*psnum*⟩ default: `0`

The total beam divergence at the starting position, given in degree. Positive values give an expanding beam, negative values a contracting beam.



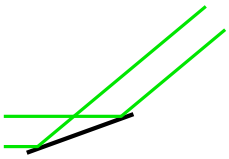

```
\begin{pspicture}(3,2)
  \pnodes(0,1){A}(3,1){B}
  \psset{plateheight=2}
  \optplate[position=start](A)(B)
  \optplate[position=end](A)(B)
  \addtopsstyle{Beam}{beamwidth=0.2}
  \drawwidebeam[beampos=0.4, beamdiv=5]{-}
  \drawwidebeam[beampos=-0.4, beamdiv=-5, linecolor=red]{-}
\end{pspicture}
```

9.3.1. Beam appearance

The appearance of wide beams can be controlled like that of single beams with the optional argument or the `Beam` style (see Sec. 9.2.7).

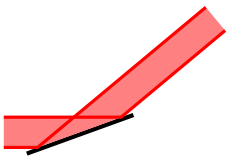
Wide beams have two marginal rays, which are drawn according to the line settings, and may be filled.

1. Only marginal rays.



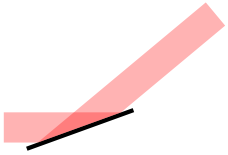
```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

2. Marginal rays, and with fill style.



```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \addtopsstyle{Beam}{fillstyle=solid,
      fillcolor=red!50!white, linecolor=red}
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

3. Only with fill style.

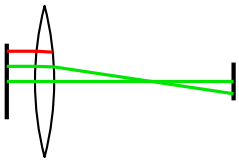


```
\begin{pspicture}(3,2)
  \pnodes(0,0.5){A}(1,0.5){B}(2.8,2){C}
  \mirror[mirrorwidth=1.5](A)(B)(C)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=red,
      opacity=0.3, linestyle=none}
  \drawwidebeam[beamwidth=0.4](A){}(C)
\end{pspicture}
```

9.4. Error handling

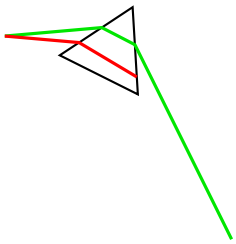
All beam are calculated and drawn directly in Postscript, so that a comprehensive error handling at compilation time is not possible. If a situation arises which is not supported by the drawing algorithm, the beam path is truncated at this point. This happens in the following cases:

1. A beam misses an interface. The red beam in the following example misses the last plane and is truncated at the last valid interface.



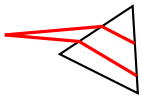
```
\begin{pspicture}(3,2.7)
  \pnodes(0,1){A}(3,1){B}
  \optplate[position=start](A)(B)
  \lens[lens=4 4 2, abspos=0.5, n=2.5](A)(B)
  \optplate[position=end, plateheight=0.5](A)(B)
  \drawbeam{-}
  \drawbeam[beampos=0.2]{-}
  \drawbeam[beampos=0.4, linecolor=red]{-}
\end{pspicture}
```

2. Total internal reflection occurs at an interface which is not set up for this. The red beam in the following example would be reflected, but this is not supported and the beam is truncated at the interface where the total internal reflection occurs.



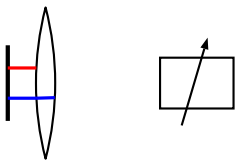
```
\begin{pspicture}(3,2.5)
  \pnodes(0,2){A}(3,0){B}
  \optplane(A)
  \optprism[n=1.8](A)([Xnodesep=1.5]A)(B)
  \optplane(B)
  \drawbeam[beamangle=5]{-}
  \drawbeam[beamangle=-5, linecolor=red]{-}
\end{pspicture}
```

These conditions apply to wide beams (see Sec. 9.3) as well, the beam path is interrupted as soon as one of the marginal rays has an error.



```
\begin{pspicture}(3,2.5)
  \pnodes(0,2){A}(3,0){B}
  \optplane(A)
  \optprism[n=1.8](A)([Xnodesep=1.5]A)(B)
  \optplane(B)
  \drawwidebeam[beamdiv=10, linecolor=red]{-}
\end{pspicture}
```

3. A component is to be connected, which does not support free-ray connections. In this case, the wrong component and all following ones are ignored.



```
\begin{pspicture}(3,2.7)
  \pnodes(0,1){A}(2,1){B}
  \optplate[position=start](A)(B)
  \lens[lens=4 4 2, abspos=0.5, n=2.5](A)(B)
  \fiberdelayline[fdlsize=1 0.7, position=end, fiber=none](A)(B)
  \drawbeam[beampos=0.2, linecolor=red]{-}
  \drawbeam[beampos=-0.2, linecolor=blue, beaminsidelast]{-}
\end{pspicture}
```

pswarning=true, false default: false

To simplify debugging, you can have a message printed out if any of these situations arises. The message will be printed to standard output, when a Postscript interpreter reads the file, e.g. when you view the file, or you convert it with ps2pdf or similar.

9.5. Custom beams

In principal, all kind of beam paths can be realized with an appropriate choice of refractive index and component parameters (e.g. lens curvature). This can become very cumbersome, so that the package provides the means to simplify this. The end points of one beam path can be used as starting points for the next beam, which allows piecewise definition of the beam divergence, angle, and direction.

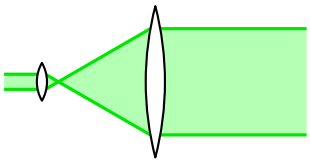
savebeampoints=true, false, ⟨*int*⟩ default: true

Saves the end points of a \draw*beam macro, previous points are overwritten. \drawbeam and \drawwidebeam are treated separately. If a number ⟨*int*⟩ > 0 is given, you can save as many end points as you like, see e.g. Ex. 11. But in most cases enabling (1) and disabling (0) this option is sufficient.

loadbeampoints=true, false, ⟨*int*⟩ default: false

Use the end points of a previous beam as starting points for the next beam. If no number ⟨*int*⟩ is specified, 1 is used.

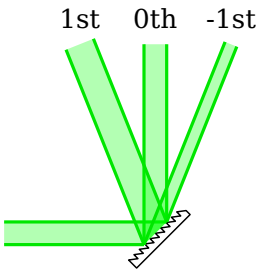
As example we consider a telescope which has no divergence at the input and output. Usually we would need to tweak the refractive index and the lens parameters to get an output beam without divergence. Instead, we draw the three beam parts separately using the end points of the previous beams as new starting points and specifying the divergence for each beam part explicitely.



```
\begin{pspicture}(4,2)
  \pnodes(0,1){A}(4,1){B}
  \begin{optexp}
    \lens[lens=0.5 0.5 0.5, abspos=0.5](A)(B)
    \lens[lens=4 4 2, abspos=2](A)(B)
    \addtopsstyle{Beam}{%
      fillstyle=solid, fillcolor=green, opacity=0.3}
    \psset{loadbeampoints}
    \drawwidebeam[beamwidth=0.2, stopinside](A){1}
    \drawwidebeam[beamdiv=-60]{1}{2}
    \drawwidebeam{2}(B)
  \end{optexp}
\end{pspicture}
```

The next example shows diffraction from a grating in the zeroth and first diffraction order. First we draw the beam to the grating, its end points are used for the beams into both diffraction orders.

Ex. 9.1




```
\begin{pspicture}(3.5,3.5)
  \pnodes(0,0.5){A}(2,0.5){B}(2,3){C}
  \nodexn{(C)-(1,0)}{C'}
  \nodexn{(C)(1,0)}{C''}
  \optgrating(A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[beamwidth=0.3](A){1}
  \psset{savebeampoints=false, loadbeampoints}
  \drawwidebeam{1}(C)
  \drawwidebeam{1}(C')
  \drawwidebeam{1}(C'')
\end{pspicture}
```

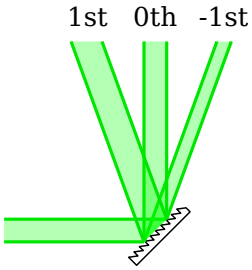
savebeam=true, false, ⟨*int*⟩ default: true

Saves the end conditions, i.e. end points and output vectors, of each \draw∗beam macro, previous values are overwritten. \drawbeam and \drawwidebeam are treated separately. If a number ⟨*int*⟩ > 0 is given, you can save as many end points as you like, but in most use cases enabling and disabling this option is sufficient.

loadbeam=true, false, ⟨*int*⟩ default: false

Use the end condition (points and vectors) of a previous beam as starting conditions for the next beam. The parameter beamangle can be used to change the direction relative to the loaded one (see following example).

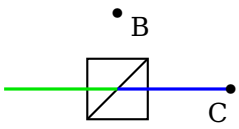
Ex. 9.2



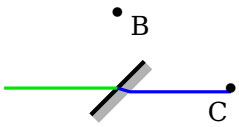

```
\begin{pspicture}(3.5,3.5)
  \pnodes(0,0.5){A}(2,0.5){B}(2,3){C}
  \optgrating(A)(B)(C)
  \addtopsstyle{Beam}{%
    fillstyle=solid, fillcolor=green, opacity=0.3}
  \drawwidebeam[savebeam, beamwidth=0.3](A){1}
  \psset{savebeam=false, loadbeam}
  \drawwidebeam[beamangle=-20]{1}(C)
  \drawwidebeam{1}(C)
  \drawwidebeam[beamangle=20]{1}(C)
\end{pspicture}
```

Please note, that loadbeam and savebeam do not always work, if the beam path ends inside a semitransparent mirror or a beamsplitter. In these cases only the transmitted ray vector is saved and continuing the beam via the reflective path is not possible.

Accordingly, in the following examples the red beam is not drawn, which should continue the green beam from the component to node ⟨B⟩.




```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.5){A}(1.5,1.5){B}(3,0.5){C}
  \beamsplitter(A)(1.5,0.5)(B)
  \drawbeam[beaminsidelast, savebeam](A){}
  \drawbeam[linecolor=red, beaminsidefirst, loadbeam]{}(B)
  \drawbeam[linecolor=blue, beaminsidefirst, loadbeam]{}(C)
\end{pspicture}
```



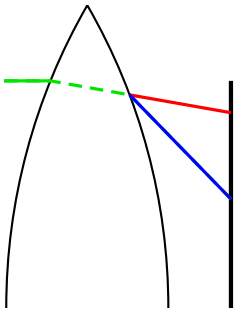

```
\begin{pspicture}(3,1.5)
  \pnodes(0,0.5){A}(1.5,1.5){B}(3,0.5){C}
  \mirror[mirrortype=semitrans](A)(1.5,0.5)(B)
  \drawbeam[beaminsidelast, savebeam](A){}
  \drawbeam[linecolor=red, beaminsidefirst, loadbeam]{}(B)
  \drawbeam[linecolor=blue, beaminsidefirst, loadbeam]{}(C)
\end{pspicture}
```

startinside=true, false default: false

The parameter beaminsidefirst controls if the internal beams are drawn inside the first component. However, for custom beams it can also be necessary to trace the internal beam of the first component without drawing it, e.g. if the previous beam ended at the first interface.

In the following example the red beam is the wrong continuation of the incoming green beam because it assumes that the loaded beam conditions refer to the output

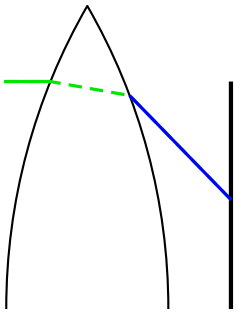
interface. But the previous beam stopped at the input interface, so one needs to set startinside in order to continue the beam properly (see blue beam). The dashed line helps to understand the correct beam path through the lens.



```
\begin{pspicture*}(0,2)(3.05,6)
  \optplane(0,2)
  \lens[lens=8 8 8, abspos=0.6, n=1.8](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{savebeam=false}
  \drawbeam[beampos=3, linestyle=dashed]{-}
  \drawbeam[beampos=3, savebeam]{1-2}
  \drawbeam[loadbeam, linecolor=red]{2-3}
  \drawbeam[loadbeam, startinside, linecolor=blue]{2-3}
\end{pspicture*}
```

stopinside=true, false default: false

This option is equivalent to startinside but refers to the last component.



```
\begin{pspicture*}(0,2)(3.05,6)
  \optplane(0,2)
  \lens[lens=8 8 8, abspos=0.6, n=1.8](0.5,2)(3,2)
  \optplate[position=1, plateheight=6](0.5,2)(3,2)
  \psset{savebeam=false}
  \drawbeam[beampos=3, linestyle=dashed]{-}
  \drawbeam[beampos=3, savebeam, stopinside]{1-2}
  \drawbeam[loadbeam, linecolor=blue]{2-3}
\end{pspicture*}
```

9.6. Drawing fibers

All available components can be connected with fiber connections, the fiber components in Sec. 5 are connected automatically.

\drawfiber[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

This macro draws fiber connections and takes a variable number of arguments (minimum of two) which can be either an ID, an ID range, a compname, or a PSTricks node (see Sec. 9.1). To distinguish between nodes and components, nodes must be either enclosed in parenthesis within the brackets, or enclosed only in parenthesis: \drawfiber{(node)}{comp}, or \drawfiber(node){comp}. If you specify more than two arguments, it draws the connections $1 \to 2$, $2 \to 3$, and so on.

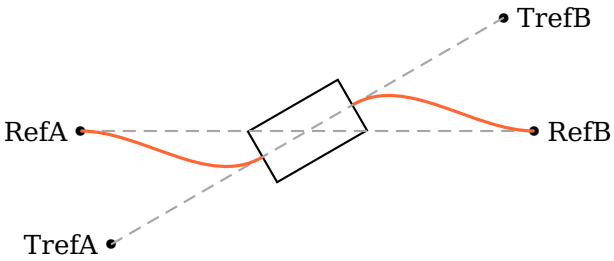
The automatic fiber connections are drawn internally with \drawfiber commands from each reference node to the component.

9.6.1. Fiber angles

The fibers are usually drawn as \nccurve connections, the angles at both curve ends are automatically determined by the component orientation and depend on whether you connect a component with a component, or a component with a node.

Component with node

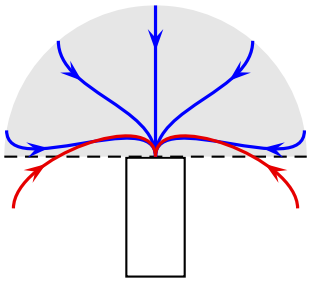
A connection of a component with a node is what is used for automatic connections (see Sec. 9.8). Consider the following example for clarification of the curve alignment:




```
\begin{pspicture}(-1,-0.7)(7,2.7)
  \pnodes(0,1){A}(6,1){B}
  \optbox[angle=30](A)(B)
  \drawfiber(A){}(B)
\end{pspicture}
```

The angle at the component itself agrees with the transformed reference line. The angle at the node corresponds to the angle of the original reference line of the component which it gets connected to. The fiber is always directed to the component node.

Considering the following example, this means that the start angles of the red fibers are rotated by 180° with respect to the blue fibers. The angles at the component do not change.

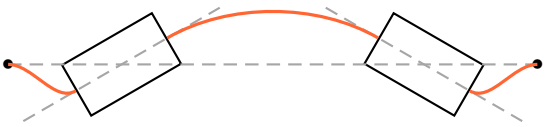


```
\begin{pspicture}(-2,-1.6)(2,2)
  \optbox[position=end](0,1)(0,0)
  \addtopsstyle{Fiber}{linecolor=blue}
  \multido{\i=10+40}{5}{%
    \drawfiber(2;\i){}
  }
  \addtopsstyle{Fiber}{linecolor=red!90!black}
  \drawfiber(2;-20){}
  \drawfiber(2;200){}
\end{pspicture}
```

Component with component

This variant is very helpful to achieve smoother connections between components which are not aligned on the same reference line but are arranged maybe in a loop or similar, see Ex. 4 and Ex. 9.6.2.

In this case, the curve angles are determined by the orientation of the component which the respective curve end is connected to. The fibers are always directed away from the component center.



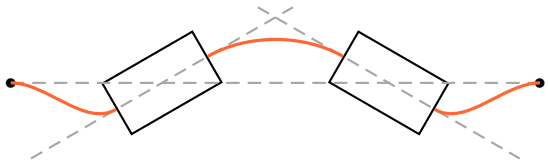
```
\begin{pspicture}(0,-0.8)(7,0.8)
  \pnodes(0,0){A}(3,0){B}(4,0){C}(7,0){D}
  \optbox[angle=30](A)(B)
  \optbox[angle=-30](C)(D)
  \drawfiber(A){1}{2}(D)
\end{pspicture}
```

These automatically calculated start and stop angles can be changed in a variety of ways, the available parameters are listed in the following.

`fiberalign`=`rel`, `relative`, `center`, `abs`, `absolute` default: `relative`

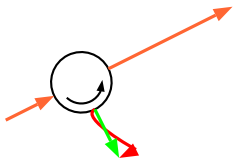
Set the reference for the fiber angles.

`relative` The fiber angles are relative to the component. When connecting a node to a component, the angle at the node is that of the component reference line. The angle at the component agrees with that of its transformed reference line.



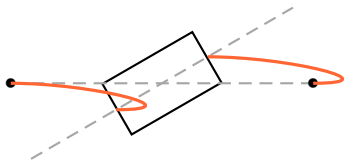
```
\begin{pspicture}(1,-1.7)(7,1.7)
  \pnodes(0,0){A}(4,0){B}(3,0){C}(7,0){D}
  \optbox[angle=30](A)(B)
  \optbox[angle=-30](C)(D)
  \drawfiber(A){1}{2}(D)
\end{pspicture}
```

center The fiber angles both at the node and the component are relative to the connection between the involved interface node and the center node of the component. In most cases this is equivalent to `relative`, but necessary e.g. for \optcirculator and \elecmixer. In the example, the green fiber shows the correct alignment, while the red one is with `relative` alignment.



```
\begin{pspicture}(1,0)(3,2)
  \addtopsstyle{Fiber}{arrowscale=1.3, arrows=->, arrowinset=0}
  \optcirculator[fiber=t](0,0.5)(3,2)(1.5,0)
  \drawfiber[linecolor=red]{}(1.5,0)
  \drawfiber[fiberalign=center, linecolor=green]{}(1.5,0)
\end{pspicture}
```

absolute The fiber angles are absolute, no automatic values are added.



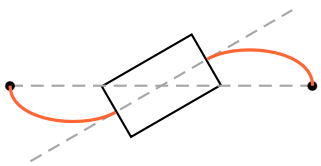
```
\begin{pspicture}(1,-1)(4.5,1)
  \pnodes(0,0){A}(4,0){B}
  \optbox[angle=30](A)(B)
  \drawfiber[fiberalign=abs, ncurv=1.5](A){}(B)
\end{pspicture}
```

fiberangleA=⟨*num*⟩ default: 0

An additive value for the starting angle, the total angle depends on fiberalign. If this parameter is set to `absolute`, then fiberangleA is the total angle, otherwise its value is added to the calculated angle.

fiberangleB=⟨*num*⟩ default: 0

Like fiberangleA, but for the end angle.



```
\begin{pspicture}(0,-1)(4,1)
  \pnodes(0,0){A}(4,0){B}
  \optbox[angle=30](A)(B)
  \drawfiber[fiberangleA=-90](A){}
  \drawfiber[fiberangleB=-90]{}(B)
\end{pspicture}
```

startnode=auto, N, 1, 2, ... default: auto

The connections with \drawfiber are drawn between the two nodes which are nearest to each other. This fits most of the time, but not always. With this option you can choose the start node explicitely.

stopnode=auto, N, 1, 2, ... default: auto

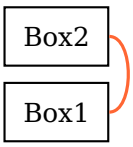
Like startnode, but for the stop node.

The following examples show some use cases for startnode and stopnode.

1. This example shows a situation, where choosing the nearest nodes is correct.

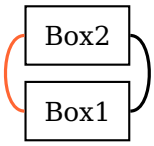
```
\begin{pspicture}(3,2)
  \pnodes(-0.5,1){A}(1.5,1){B}(3.5,1){C}
  \optbox(A)(B){Box1}\optbox(B)(C){Box2}
  \drawfiber{1}{2}
\end{pspicture}
```

2. In this case, the nearest connection is not well defined because two connections have the same length.




```
\begin{pspicture}(2,2)
  \pnodes(0,0.5){A}(2,0.5){B}([offset=1]B){C}(A|C){D}
  \optbox(A)(B){Box1}\optbox(C)(D){Box2}
  \drawfiber{1}{2}
\end{pspicture}
```

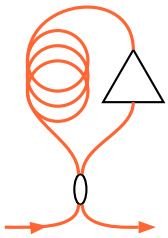
3. To select the desired connection it is often enough to set either startnode or stopnode. With this you fix one node, the other is the one nearest to it. In rare cases it might be necessary to fix both nodes.




```
\begin{pspicture}(2,2)
  \psset{label=0}
  \pnodes(0,0.5){A}(2,0.5){B}([offset=1]B){C}(A|C){D}
  \optbox(A)(B){Box1}\optbox(C)(D){Box2}
  \drawfiber[startnode=1]{1}{2}
  \drawfiber[startnode=N, linecolor=black]{1}{2}
\end{pspicture}
```

4. Another example where you need to specify the `startnode`.

Ex. 9.3



```
\begin{pspicture}(2.1,3)
  \pnodes(0,0){A}(2,0){B}(0.3,3){C}(1.7,3){D}
  \psset{fiber=none, arrowscale=1.2, arrowinset=0}
  \optcoupler[fiber=l,
              addtoFiberIn1={angleA=0, ArrowInside=->},
              addtoFiberIn2={angleA=180, arrows=<-},
              abspos=0.5, compname=Cpl](A)(B)(C)(D)
  \optfiber[compname=Hnlf, abspos=1](C)(C|A)
  \optamp[abspos=2, compname=Amp](D|B)(D)
  \drawfiber{Cpl}{Hnlf}
  \drawfiber[startnode=1, ncurv=1.2]{Hnlf}{Amp}
  \drawfiber{Cpl}{Amp}
\end{pspicture}
```

9.6.2. Fiber appearance

`Fiber` ⟨*psstyle*⟩

The appearance of fibers is controlled with this style. The optional argument of `\drawfiber` can be used as well, these options can overwrite settings from the `Fiber` style.

`addtoFiber`=⟨*list*⟩

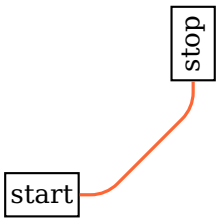
The `Fiber` style is extended by locally adding the parameters contained in ⟨*list*⟩. The ⟨*list*⟩ must be surrounded by curly braces. See for example Ex. 9.6.2.

`newFiber`=⟨*list*⟩

Similar to `addtoFiber`, but an existing `Fiber` style is overwritten with the new parameter set.

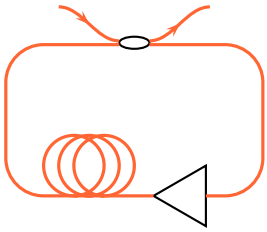
`fiberstyle`=⟨*string*⟩ default: `curve`

The fibers are usually drawn as `\nccurve`, but any type of node connection `\nc`⟨*string*⟩ from `pst-node` can be selected with this parameter, see the `pst-node` documentation[3] for possible values. The following examples show possible use cases, Ex. 11 contains another typical use case.




```
\begin{pspicture}(3,3)
  \pnodes(1, 0.5){A}(2.5,0.5){B}(2.5,2){C}
  \psset{optboxsize=1 0.6, innerlabel}
  \optbox[position=start](A)(B){start}
  \optbox[position=end](B)(C){stop}
  \drawfiber[fiberstyle=diag, linearc=0.5]{1}{2}
\end{pspicture}
```

Ex. 9.4



```
\begin{pspicture}(4,2.5)
  \pnodes(1,2.5){In}(3,2.5){Out}(0,2){A}(4,2){B}(4,0){C}(0,0){D}
  \optcoupler[fiber=t, addtoFiber={ArrowInside=->}, ↩
     coupleralign=b](In)(A)(Out)(B)
  \psset{fiber=none}
  \optamp[position=0.35](C)(D)
  \optfiber[position=0.35](D)(C)
  \drawfiber{2}{3}
  \addtopsstyle{Fiber}{fiberstyle=bar, linearc=0.5, armA=1.5}
  \drawfiber[stopnode=1]{1}{2}
  \drawfiber[stopnode=1]{1}{3}
\end{pspicture}
```

9.7. Drawing wires

All available components can be connected with wire connections, the wire components in Sec. 6 are connected automatically. The `wire`∗ parameters are equivalent to their `fiber`∗ counterparts, see their respective documentation for details.

`\drawwire`[⟨*options*⟩]{⟨obj_1⟩}{⟨obj_2⟩}...

This command can take a variable number of arguments (minimum of two) which can be either an ID, an ID range, a `compname`, or a PSTricks nodes (see Sec. 9.1). To distinguish between nodes and components, nodes must be either enclosed in parenthesis within the brackets, or enclosed only in parenthesis:

[3] http://mirror.ctan.org/help/Catalogue/entries/pst-node.html

\drawwire{(node)}{comp}, or \drawwire(node){comp}. If you specify more than two arguments, you get connections $1 \to 2$, $2 \to 3$, and so on.

The automatic connections are drawn internally with \drawwire commands from each reference node to the component.

wirealign=rel, relative, center, abs, absolute default: relative

See fiberalign.

wireangleA=⟨*num*⟩ default: 0

See fiberangleA.

wireangleB=⟨*num*⟩ default: 0

See fiberangleB.

wirestyle=⟨*string*⟩ default: angle

See fiberstyle, the default value is angle.

addtoWire=⟨*list*⟩

The Wire style is extended by locally adding the parameters contained in ⟨*list*⟩. The ⟨*list*⟩ must be surrounded by curly braces.

newWire=⟨*list*⟩

Similar to addtoWire, but an existing Wire style is overwritten with the new parameter set.

Wire ⟨*psstyle*⟩

The appearance of wires is controlled with this style, compare with Sec. 9.6.2. The optional argument of \drawwire can be used as well, these options can overwrite settings from the Wire style.

9.8. Automatic fiber and wire connections

All fiber components described in Sec. 5 are connected automatically to their reference nodes with `\drawfiber`, their behaviour is descibed in Sec. 9.6.1. The electrical components (Sec. 6) are accordingly connected with `\drawwire`. The connections can also be controlled in a number of ways, without explicitely using these macros. The style of each connection can be configure independently, see Sec. 9.8.1.

`fiber`=[∗+]`none`, `all`, `i`, `o`, ⟨*refpoint*⟩ default: `all`

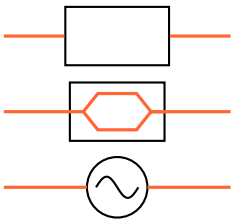
This parameter selects which fiber connections are drawn. The value of this parameter can affect either only the fiber components (if the value is prefixed with ∗), only free-ray and electrical components (prefixed with +), or all component types otherwise. Typical free-ray components which are often used with fibers as well are `\optbox` and `\optdetector`. Please note, that electrical components cannot be connected automatically with fibers, in these cases you must use `\drawfiber` explicitely.

You can select to connect `all` nodes, `none`, or select distinct connections to draw. Value `i` (`l` is equivalent) draws all input fibers, `o` (`r` is equivalent) draws all output fibers. These values are exclusive, that means `io` does not select all fibers, but none.

Components which have more than one input or output fiber (couplers and `\optcirculator`) allow also values `t` and `b`, possibly in combination with `i`, `o`, `r`, or `l`.

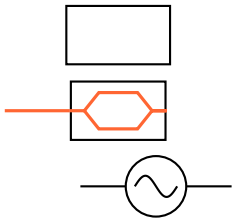
See the following examples for some variants.

1. Enable all fiber connections all components.



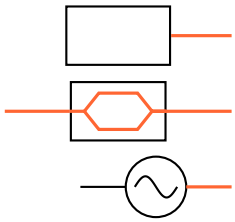
```
\begin{pspicture}(3,3)
  \psset{fiber}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

2. Draw only the input fiber of a fiber component (`\optmzm`), the other components are not affected.



```
\begin{pspicture}(3,3)
  \psset{fiber=*i}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](1,0.5)(3,0.5)
\end{pspicture}
```

3. All connections of the fiber component are drawn, the other components get only the output fiber.



```
\begin{pspicture}(3,3)
  \psset{fiber=+o}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](1,0.5)(3,0.5)
\end{pspicture}
```

4. The values `t` and `b` affect only components which have more than two fibers, i.e `\optcirculator` and the couplers (Sec. 5.10).


```
\begin{pspicture}(3,1)
  \optcoupler[fiber=t](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```

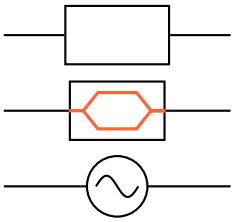
```
\begin{pspicture}(3,1)
  \wdmsplitter[fiber=br](0,0.5)(3,1)(3,0)
\end{pspicture}
```

```
\begin{pspicture}(3,1)
  \optcirculator[fiber=b](0,0.5)(3,1)(1,0)
\end{pspicture}
```

`wire`=[*+]`none`, `all`, `i`, `o`, ⟨*refpoint*⟩ default: `all`

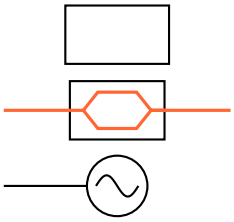
This parameter selects which wire connections are drawn. The value of this parameter can affect either only the electrical components (if the value is prefixed with *), only free-ray and fiber components (prefixed with +), or all components otherwise.

1. Enable all wire connections for all components.



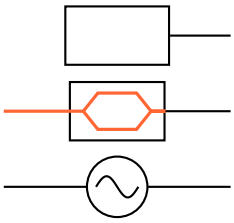
```
\begin{pspicture}(3,3)
  \psset{wire}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

2. Draw only the input wire of an electrical component (`\elecsynthesizer`), the other components are not affected.



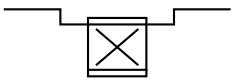
```
\begin{pspicture}(3,3)
  \psset{wire=*i}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

3. Draw only the output wire of the free-ray and fiber components, but all connections of the wire component.

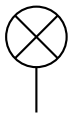


```
\begin{pspicture}(3,3)
  \psset{wire=+o}
  \optbox(0,2.5)(3,2.5)
  \optmzm(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

4. The values `t` and `b` affect only components which have more than two connections, i.e `\elecmixer` and `\eleccoupler`.



```
\begin{pspicture}(3,1)
  \eleccoupler[wire=t](0,1)(0,0)(3,1)(3,0)
\end{pspicture}
```



```
\begin{pspicture}(3,1.5)
  \elecmixer[wire=b](0,1)(3,1)(1,0)
\end{pspicture}
```

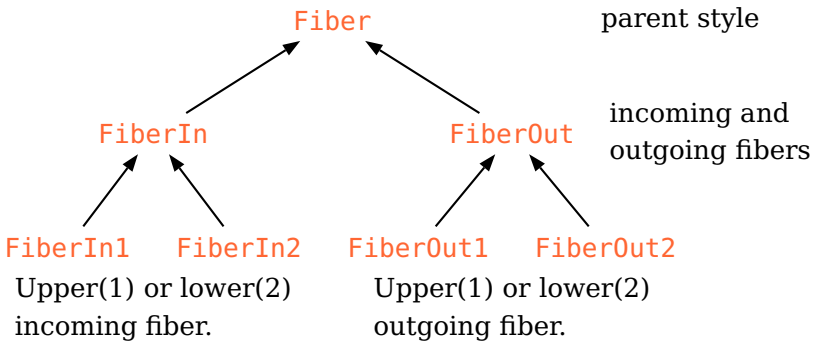


Figure 9.1.: Inheritance diagram for the psstyles used for the automatic fiber connections. These styles should be changed with `\addtopsstyle` or the respective `addto`⟨*style*⟩ options to preserve the inheritance. An identical inheritance tree applies to the respective `Wire...` styles.

9.8.1. Appearance of automatic connections

The styles of all automatic fiber connections can be configured independently, they are all derived from the basic `Fiber` style which is described in Sec. 9.6.2.

`FiberIn` `FiberIn1` `FiberIn2` `FiberOut` `FiberOut1` `FiberOut2`

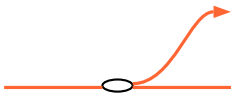
Style `FiberIn` applies to all input fibers, it inherits from `Fiber`. If the component has more than one input fiber they use the styles `FiberIn1` and `FiberIn2` which inherit from `FiberIn`. All output fibers use the style `FiberOut`, in case of two output fibers they use the styles `FiberOut1` and `FiberOut2` which inherit from `FiberOut`. See Fig. 9.1 for an inheritance diagram of the styles. You must have in mind, that this inheritance is effective only if you use `\addtopsstyle` to change the styles (see also below).

`WireIn` `WireIn1` `WireIn2` `WireOut` `WireOut1` `WireOut2`

Style `WireIn` applies to all input wires, it inherits from `Wire`. If the component has more than one input wire they use the styles `WireIn1` and `WireIn2` which inherit from `WireIn`. All output wires use the style `WireOut`, in case of two output wires they use the styles `WireOut1` and `WireOut2` which inherit from `WireOut`. See Fig. 9.1 for an inheritance diagram of the styles. You must have in mind, that this inheritance is effective only if you use `\addtopsstyle` to change the styles (see also below).

`new`⟨*style*⟩ `addto`⟨*style*⟩

For every style two appropriate keys `new`⟨*style*⟩ and `addto`⟨*style*⟩ are provided which can be used to change the styles for single objects. This can be used to define own components (Sec. 10.1) with respectively changed connections or to avoid explicit grouping.

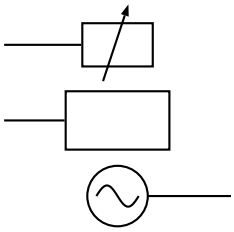


99/1

```
\begin{pspicture}(3,2)
  \newpsobject{tapcoupler}{wdmsplitter}{%
    coupleralign=b,
    addtoFiberOut1={arrows=->, arrowscale=1.2, arrowinset=0}}
  \tapcoupler(0,0.5)(3,1.5)(3,0.5){99/1}
\end{pspicture}
```

9.8.2. Known limitations

The combination of `fiber` and `wire` does not allow any arbitrary combination of all parameter values in the current implementation (version v6.1). Most combinations do work, but the following examples shows a variant, which does not work correctly (the `\elecsynthesizer` should have an input fiber):



```
\begin{pspicture}(3,3)
  \psset[optexp]{fiber=i, wire=+i}
  \fiberdelayline(0,2.5)(3,2.5)
  \optbox(0,1.5)(3,1.5)
  \elecsynthesizer[position=0.5](0,0.5)(3,0.5)
\end{pspicture}
```

In such special cases one must resort to manual connections.

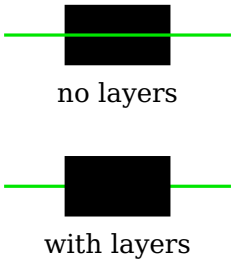
9.9. Layers

The necessary order of constructing a drawing is to define the components first before you can connect them. This implies that all connections are drawn on top of the components, which might be unwanted.

To circumvent this, we provide the `optexp` environment, inside which all connections are drawn behind the components.

```
\begin{optexp}

  \end{optexp}
```




```
\begin{pspicture}(3,4)
  \psset{beam, fillstyle=solid, fillcolor=black}
  \optbox(0,3.5)(3,3.5){no layers}
  \begin{optexp}
    \optbox(0,1.5)(3,1.5){with layers}
  \end{optexp}
\end{pspicture}
```

Beware, that this is achieved by executing twice all code included in the environment. This works fine for all `pst-optexp` commands which take care of not drawing stuff twice but only in a changed order. All other text and graphics are drawn twice which leads to ugly results because of altered anti-aliasing in the viewer.

To avoid this, you can either move all other code outside the environment, or use the following two commands to define code parts which are executed only once.

`\backlayer`{⟨*code*⟩}

Execute ⟨*code*⟩ only once in the first pass.

`\frontlayer`{⟨*code*⟩}

Execute ⟨*code*⟩ only once in the second pass.

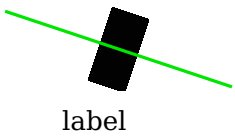
In this example, the text is put on top of the frame although the text is defined first:

```
\begin{pspicture}(3,1)
\begin{optexp}
  \frontlayer{\rput(1.5,0.5){front}}
  \backlayer{\psframe*[linecolor=DOrange!50](1,0)(2,1)}
\end{optexp}
\end{pspicture}
```

10. Custom components

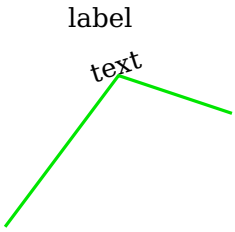
The `pst-optexp` package provides two commands which can use anything as optical components. This includes e.g. external images or your own drawings.

`\optdipole`[⟨*options*⟩](⟨*in*⟩)(⟨*out*⟩){⟨*comp*⟩}{⟨*label*⟩}




```
\begin{pspicture}(3,3)
  \optdipole[labeloffset=1, beam](0,2)(3,1){%
    \rput(0,0){\rule{0.5cm}{1cm}}%
  }{label}
\end{pspicture}
```

`\opttripole`[⟨*options*⟩](⟨*in*⟩)(⟨*center*⟩)(⟨*out*⟩){⟨*comp*⟩}{⟨*label*⟩}

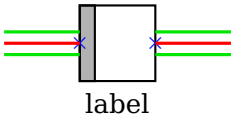



```
\begin{pspicture}(3,3)
  \opttripole[beam](3,1.5)(1.5,2)(0,0){%
    \rput[b](0,0){text}%
  }{label}
\end{pspicture}
```

These components can also be connected with beams and fibers, but do not provide full access to all options. `\opttripole` defines a single reflective interface, whereas `\optdipole` can have two transmittive interfaces:

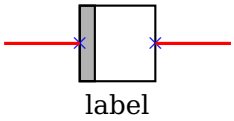
`optdipolesize`=⟨*width*⟩[⟨*height*⟩] default: `0 0`

With this option you can define two transmittive interfaces for `\optdipole`, which are placed at $(\pm 0.5\langle width\rangle, 0)$.




```
\begin{pspicture}(3,2)
\pnodes(0,1){A}(3,1){B}
\optdipole[optdipolesize=1, showifcnodes](A)(B){%
  \psframe[fillstyle=solid,fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}{label}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\drawbeam[beaminside=false, linecolor=red](A){}(B)
\end{pspicture}
```

If also a ⟨*height*⟩ greater than zero is set, this is used as size of the numerical aperture for ray tracing (see useNA). In the following example the height is set to `0.2` which is smaller than the wide beam which is not drawn, in contrast to the previous example.

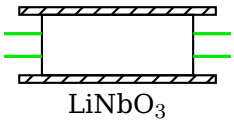



```
\begin{pspicture}(3,2)
\pnodes(0,1){A}(3,1){B}
\optdipole[optdipolesize=1 0.2, showifcnodes](A)(B){%
  \psframe[fillstyle=solid,fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}{label}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\drawbeam[beaminside=false, linecolor=red](A){}(B)
\end{pspicture}
```

Further examples are Ex. 11 and Ex. 11.

optdipolecomp=⟨*macros*⟩

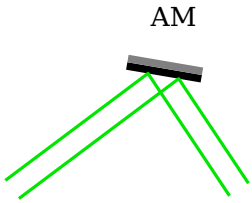
This option defines the drawing of a component, and can be used to define a new component based in \optdipole with \newpsobject.




```
\begin{pspicture}(3,2)
\def\phasemodulatorcomp{%
  \psframe(-1,-0.4)(1,0.4)
  \psframe[fillstyle=hlines](-1.3,-0.5)(1.3,-0.4)
  \psframe[fillstyle=hlines](-1.3,0.5)(1.3,0.4)}
\newpsobject{phasemodulator}{optdipole}{optdipolesize=2 0.8,
    optdipolecomp={\phasemodulatorcomp}}
\pnodes(0,1){A}(3,1){B}
\phasemodulator(A)(B){LiNbO$_3$}
\drawwidebeam[beamwidth=0.3, beaminside=false](A){}(B)
\end{pspicture}
```

opttripolecomp=⟨*macros*⟩

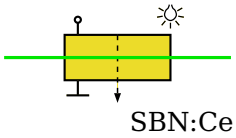
This option defines the drawing of a component, and can be used to define a new component based in \opttripole with \newpsobject.



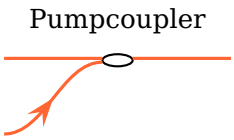

```
\begin{pspicture}(3,2)
\def\ampmodcomp{%
  \psframe*[linestyle=none](-0.5,0)(0.5,0.1)
  \psframe*[linestyle=none, linecolor=gray](-0.5,0.1)(0.5,0.2)}
\newpsobject{ampmod}{opttripole}{opttripolecomp={\ampmodcomp}}
\pnodes(0,0){A}(2,1.5){B}(3,0){C}
\ampmod(A)(B)(C){AM}
\drawwidebeam[beamwidth=0.3](A){}(C)
\end{pspicture}
```

10.1. Customized versions of existing components

You can define a customized version of an existing component using the `\newpsobject` macro. With this you can define a new component using predefined objects with a set of options. These options serve only as default values and can be overridden when calling the macro.

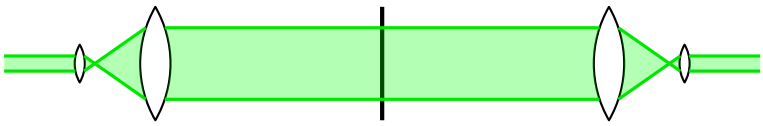



```
\begin{pspicture}(3,1.7)
\newpsobject{sbn}{crystal}{%
  voltage, lamp, %
  fillstyle=solid, fillcolor=yellow!90!black}
\sbn[label=1.2 45, beam](0,1)(3,1){SBN:Ce}
\end{pspicture}
```




```
\begin{pspicture}(3,1.7)
\newpsobject{pumpcoupler}{wdmcoupler}{%
  coupleralign=top, addtoFiberIn2={ArrowInside=->, arrowscale=2}}
\pumpcoupler[label=0.5 180](0,1)(0,0)(3,1){Pumpcoupler}
\end{pspicture}
```

Ex. 10.1



```
\begin{pspicture}(10,2)
  \newpsobject{MOLensIn}{lens}{lens=0.5 0.5 0.5}
  \newpsobject{MOLensOut}{lens}{lens=1.5 1.5 1.5}
  \pnodes(0,1){A}(10,1){B}
  \MOLensIn[abspos=1](A)(B)\MOLensOut[abspos=2](A)(B)
  \optplate[plateheight=1.5](A)(B)
  \MOLensOut[abspos=8](A)(B)\MOLensIn[abspos=9](A)(B)
  \addtopsstyle{Beam}{n=1, fillstyle=solid, fillcolor=green, opacity=0.3}
  \psset{loadbeampoints}
  \drawwidebeam[beamwidth=0.2, stopinside](A){1}
  \drawwidebeam[beamdiv=-70]{1}{2}
  \drawwidebeam[stopinside]{2-4}
  \drawwidebeam[beamdiv=-70]{4}{5}
  \drawwidebeam{5}(B)
\end{pspicture}
```

10.2. Defining new objects

pst-optexp provides some high-level macros to allow very convenient definition of your own custom components.

Note, that this part of the documentation is not complete. It roughly describes the steps to define own components since version 3.0. Some parts may be subject to change which are not backward compatible.

\newOptexpDipole[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

\newOptexpTripole[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

\newOptexpFiberDipole[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

\newOptexpElecDipole[⟨*fixopt*⟩]{⟨*name*⟩}{⟨*dftopt*⟩}

These macros generate all organizing code for the component \⟨*name*⟩ which handle the positioning, label placement, rotation and shifting, and the layering. In the simplest case you only have to define the actual drawing of the component outline.

With the argument ⟨*fixopt*⟩ you can specify argument settings which will not be allowed to be overwritten, with ⟨*dftopt*⟩ you can specify a set of default arguments which may be overwritten with the components optional argument. Both arguments ⟨*fixopt*⟩ and ⟨*dftopt*⟩ are optional and may be omitted.

The macros `\newOptexpFiberDipole` and `\newOptexpElecDipole` only differ from the generic `\newOptexpDipole` in the connection settings, like the components in Sec. 5 and Sec. 6, respectively.

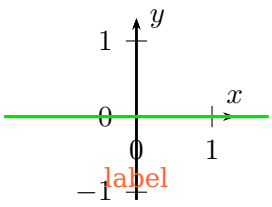
The actual process of creating a new component is split up in different parts, depending on the component complexity and user requirements:

1. The component drawing (see Sec. 10.2.1), this is the only required part.
2. Support for raytracing (optional)
3. Optional support for `rotateref`, `extnode` and `position=start|end` (optional).

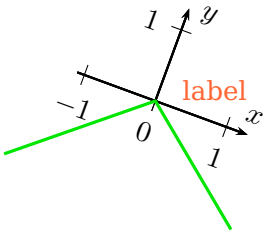
The following section show the development of a simple example component `\testcomp`.

10.2.1. The component drawing

You must define a macro which is called e.g. `\...@comp`, which contains the component drawings. See the following two examples for dipole and tripole which show the orientation and position of the component coordinate system.

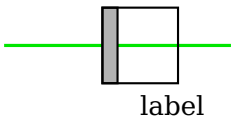



```
\begin{pspicture}(3.5,2.6)
\newOptexpDipole{mydipole}
\makeatletter
\def\mydipole@comp{%
  \psaxes[arrows=->](0,0)(0,-1.1)(1.3,1.3)[$x$,90][$y$,0]
}%
\makeatother
\mydipole(0,1)(3.5,1){\color{spot}label}
\drawbeam(0,1){}(3.5,1)
\end{pspicture}
```




```
\begin{pspicture}(3.5,3.7)
\newOptexpTripole{mytripole}{labelangle=-60}
\makeatletter
\def\mytripole@comp{%
  \psaxes[arrows=->](0,0)(-1.1,0)(1.3,1.3)[$x$,90][$y$,0]
}%
\makeatother
\mytripole(0,1)(2,1.7)(3,0){\color{spot}label}
\drawbeam(0,1){}(3,0)
\end{pspicture}
```

Our example component and its drawing are defined as follows:




```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)
}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\end{optexp}
\end{pspicture}
```

The positioning of the component and the label, and the layering is working already.

10.2.2. Support for raytracing

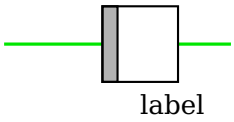
To add support for ray-tracing, you must define the interface with their properties. The definitions are nested in a macro \⟨*name*⟩@nodes in order to be accessible in the different layers.

The definitions are done with the macro \newOptexpComp

```
\def\testcomp@nodes{%
  \newOptexpComp{%
    % definition of the first interface (IF)
    {-0.5 0} % x- and y-coordinate of the IF center
    {0 1}   % direction vector of the IF (x and y)
```

```
    -0.5 0.5 % limits use for the NA as factor for IF vector.
    trans  % behavior of the IF, can be 'trans', 'refl' or 'abs'
    {PlainIfc} % type of IF
    % the same for the right IF, only the center is different
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    \POE@key@n % refractive index, could be any number
  }%
}%
```

So, in condensed form, our example component looks like this, and the beam is not drawn inside the component any more:



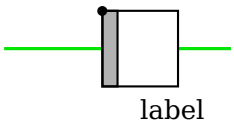

```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}%
\def\testcomp@nodes{%
  \newOptexpComp{%
    {-0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc} \POE@key@n }}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\end{optexp}
\end{pspicture}
```

10.2.3. Support for external nodes

To enable, as last step, support for external nodes with `extnode`, positioning at the start or end with `position=start|end`, and setting the rotation reference node with `rotateref`, we need to provide the external dimensions of the component. For this we need to define a new macro \⟨*name*⟩`@ref`, which defines at Postscript level the values `@@x` (half width), `@@y` (half height), and optionally `@@x0` and `@@y0` (offset of the center).

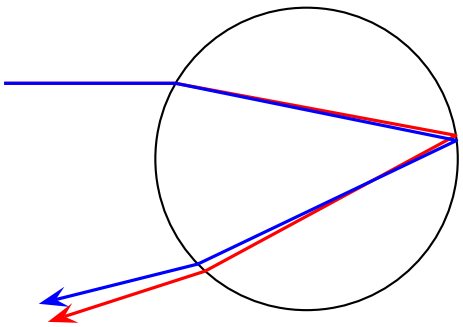
```
\def\testcomp@ref{%
  \POE@setref{ /@@x 0.5 def /@@y 0.5 def }%
}%
```

This completes the code for our example component \testcomp, which now looks like this:




```
\begin{pspicture}(3, 2)
\newOptexpDipole{testcomp}
\makeatletter
\def\testcomp@comp{%
  \psframe[fillstyle=solid, fillcolor=black!30](-0.5,-0.5)(-0.3,0.5)
  \psframe(-0.5,-0.5)(0.5,0.5)}%
\def\testcomp@nodes{%
  \newOptexpComp{%
    {-0.5 0} {0 1} -0.5 0.5 trans {PlainIfc}
    {0.5 0} {0 1} -0.5 0.5 trans {PlainIfc} \POE@key@n }}%
\def\testcomp@ref{%
  \POE@setref{ /@@x 0.5 def /@@y 0.5 def }}%
\makeatother
\begin{optexp}
\testcomp[position=0.6, labelalign=l, extnode=tl](0,1)(3,1){label}
\drawbeam[beaminside=false](0,1){}(3,1)
\psdot(\oenodeExt{})
\end{optexp}
\end{pspicture}
```

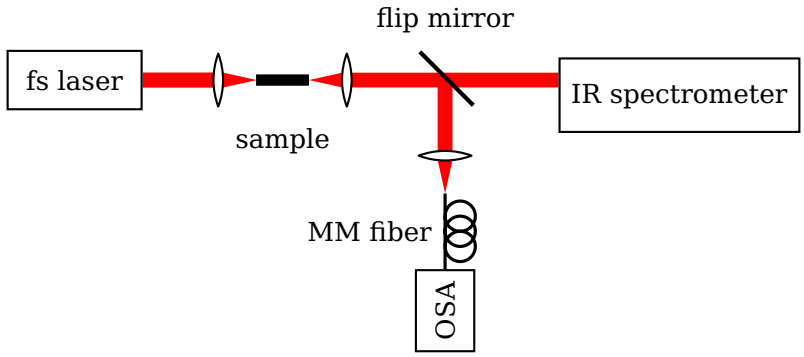
10.2.4. Examples of custom components



```
\begin{pspicture}(6,4)
  \newOptexpTripole{raindrop}
  \makeatletter
  \def\raindrop@nodes{%
    \newOptexpComp{%
      {0 -2} { 0 2 } -2 2 trans {CurvedIfc}
      {0 2} {0 -2} -2 2 refl {CurvedIfc}
      {0 -2} { 0 2 } -2 2 trans {CurvedIfc}
      \POE@key@n }}%
  \def\raindrop@comp{\pscircle(0,0){2}}%
  \raindrop(0,4)(4,2)(0,0)
  \psset{beamalign=abs, arrows=->, arrowscale=2}
  \drawbeam[linecolor=red](0,3){}(0,1)
  \drawbeam[n=*n*1.05, linecolor=blue](0,3){}(0,1)
\end{pspicture}
```

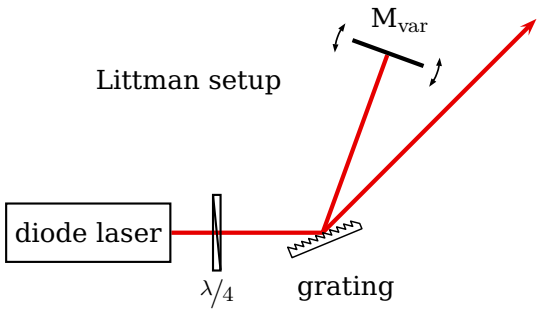
11. Examples

Ex. 11.1



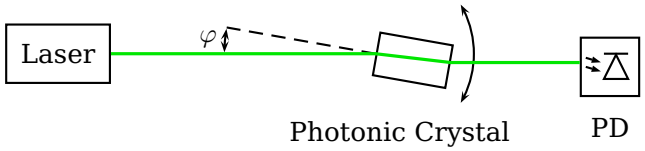

```
\begin{pspicture}(0.1,0.4)(10,4.5)
  \psset[optexp]{lens=1 1 0.7, loadbeampoints}
  \pnodes(2,4){L}([Xnodesep=4]L){M}([Xnodesep=1.5]M){IRSpec}([offset=-2.5]M){OSA}
  \begin{optexp}
  \optbox[position=start, innerlabel, optboxwidth=1.8](L)(M){fs laser}
  \lens[abspos=1](L)(M)
  \optbox[abspos=1.85, optboxsize=0.7 0.15, fillstyle=solid,
      fillcolor=black](L)(M){sample}
  \lens[abspos=2.7](L)(M)
  \mirror[labelangle=45](L)(M)(OSA){flip mirror}
  \optbox[position=end, optboxsize=3.2 1, innerlabel, compshift=-0.2](M)(IRSpec){IR
      spectrometer}
  \lens[abspos=1](M)(OSA)
  \optplane[angle=90]([offset=-1.5]M)
  \optfiber[fiberloopradius=0.2, label=1,
      newFiber={linewidth=1.5\pslinewidth}]([offset=-1.5]M)(OSA){MM fiber}
  \optbox[position=end, innerlabel, optboxwidth=1.1](M)(OSA){OSA}
  \newpsstyle{Beam}{linestyle=none, fillcolor=red, fillstyle=solid, raytrace=false}
  \drawwidebeam[beamwidth=0.2, stopinside]{1-2}
  \drawwidebeam[beamdiv=-25]{2-3}
  \drawwidebeam[loadbeampoints=false, beamdiv=25]{3-4}
  \drawwidebeam[startinside]{4-5}
  \drawwidebeam[savebeampoints=false, beamalign=abs]{5-6}
  \drawwidebeam[stopinside]{5}{7}
  \drawwidebeam[beamdiv=-25]{7-8}
\end{optexp}
\end{pspicture}
```

Ex. 11.2



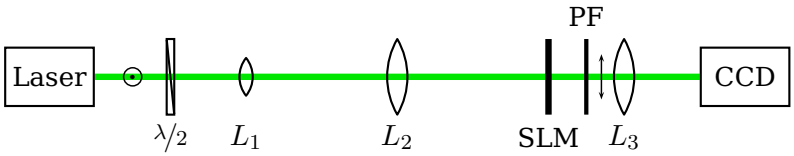

```
\begin{pspicture}(-4.2,-1)(3,3)
\pnodes(-2,0){LOut}(0,0){Grat}(4;45){Out}(2.5;70){Mvar}
\newpsstyle{Beam}{linewidth=2\pslinewidth, linecolor=red!90!black}
\begin{optexp}
  \optbox[optboxwidth=2.2, labeloffset=0, position=start](LOut)(Grat){diode laser}
  \mirror[variable, label=0.5](Grat)(Mvar)(Grat){M$_\mathrm{var}$}
  \optretplate[position=0.3](LOut)(Grat){$\nicefrac{\lambda}{4}$}
  \optgrating(LOut)(Grat)(Out){grating}
  \drawbeam[arrows=->]{1}{3}{4}(Out)
  \drawbeam{2}{4}
\end{optexp}
\rput[l](-3,2){Littman setup}
\end{pspicture}
```

Ex. 11.3



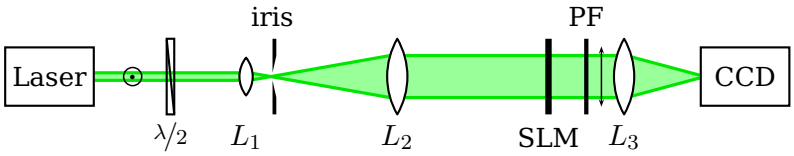

```
\begin{pspicture}(8.5,1.9)
\pnodes(1.4,1.3){Laser}(7.6,1.3){Diode}
\optbox[position=start, labeloffset=0](Laser)(Diode){Laser}%
\optbox[abspos=4, optboxsize=1 0.6, labeloffset=1, n=3,
       compname=PC, angle=-10, rotateref=l](Laser)(Diode){Photonic Crystal}
\optdetector[dettype=diode](\oenodeOut{PC})(Diode|\oenodeOut{PC}){PD}
\nodexn{(\oenodeIn{PC}) + (2;170)}{Angle1}
\psline[linestyle=dashed](\oenodeIn{PC})(Angle1)
\psarc{<->}(\oenodeIn{PC}){1.3}{330}{30}
\psarc[arcsep=1pt]{<->}(\oenodeIn{PC}){2}{170}{180}
\uput{2.1}[175](\oenodeIn{PC}){\small $\varphi$}
\drawbeam{-}
\end{pspicture}
```

Ex. 11.4



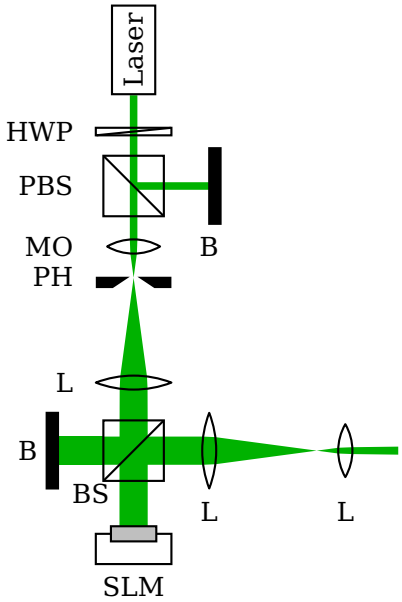

```
\begin{pspicture}(10.4,1.7)
\pnodes(1.2,1){Start}(9.2,1){CCD}
\begin{optexp}
\optbox[position=end, label=0, optboxwidth=1.2](CCD)(Start){Laser}
\optbox[position=end, label=0, optboxwidth=1.2](Start)(CCD){CCD}
\polarization[poltype=perp,abspos=0.5](Start)(CCD)
\optretplate[abspos=1](Start)(CCD){$\nicefrac{\lambda}{2}$}
\lens[lens=0.4 0.4 0.5,abspos=2](Start)(CCD){$L_1$}
\lens[abspos=4](Start)(CCD){$L_2$}
\optplate[abspos=6, platelinewidth=3\pslinewidth](Start)(CCD){SLM}
\optplate[abspos=6.5, labelangle=180](Start)(CCD){PF}
\polarization[abspos=6.7](Start)(CCD)
\lens[abspos=7](Start)(CCD){$L_3$}
\drawbeam[linewidth=2\pslinewidth]{1}{2}
\end{optexp}
\end{pspicture}
```

Ex. 11.5



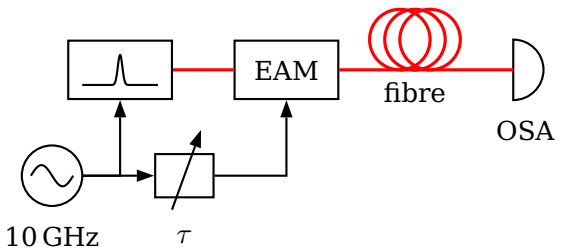

```
\begin{pspicture}(10.4,1.7)
\pnodes(1.2,1){Start}(9.2,1){CCD}
\begin{optexp}
\optbox[position=start, label=0, optboxwidth=1.2](Start)(CCD){Laser}
\polarization[poltype=perp,abspos=0.5](Start)(CCD)
\optretplate[abspos=1](Start)(CCD){$\nicefrac{\lambda}{2}$}
\lens[lens=0.4 0.4 0.5,abspos=2](Start)(CCD){$L_1$}
\pinhole[phwidth=0.05, abspos=2.35, labelangle=180](Start)(CCD){iris}
\lens[abspos=4](Start)(CCD){$L_2$}
\optplate[abspos=6,platelinewidth=3\pslinewidth](Start)(CCD){SLM}
\optplate[abspos=6.5,labelangle=180](Start)(CCD){PF}
\lens[abspos=7](Start)(CCD){$L_3$}
\optbox[position=end, label=0, optboxwidth=1.2](Start)(CCD){CCD}
\polarization[abspos=6.7, polsize=0.8](Start)(CCD)
\addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40!white, loadbeampoints}
\drawwidebeam[beamwidth=0.1, stopinside]{1}{4}
\drawwidebeam[beamdiv=-20]{4}{6}
\drawwidebeam[stopinside]{6-9}
\drawwidebeam[beamdiv=-30]{9-10}
\end{optexp}
\end{pspicture}
```

Ex. 11.6: Adapted from arXiv:1112.5270v2



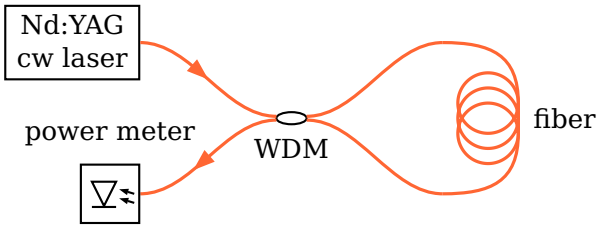

```
\begin{pspicture}(0.3,0.4)(5.5,8.3)
\pnodes(2,7){L}(2,5.8){Pbs}([Xnodesep=1]Pbs){Blk1}(2,2.3){Bs}
\pnodes([Xnodesep=-1]Bs){Blk2}([Xnodesep=3.5]Bs){Out}(2,1.3){Slm}
\begin{optexp}
  \optbox[position=end, innerlabel, optboxsize=1.2 0.6](Pbs)(L){Laser}
  \psset{labelalign=r, mirrortype=extended}
  \newpsstyle{ExtendedMirror}{fillstyle=solid, fillcolor=black}
  \optretplate[position=0.4](L)(Pbs){HWP}
  \beamsplitter[labelangle=-90](L)(Pbs)(Blk1){PBS}
  \mirror[label=. -90 c](Pbs)(Blk1)(Pbs){B}
  \lens[abspos=0.8, lens=0.7 0.7 0.7, n=2](Pbs)(Bs){MO}
  \pinhole[phwidth=0.15, abspos=1.2](Pbs)(Bs){PH}
  \lens[abspos=2.6, lensradius=1.4](Pbs)(Bs){L}
  \psset{labelalign=c}
  \beamsplitter[labelangle=-135](Pbs)(Bs)(Blk2){BS}
  \mirror[labeloffset=0.4](Bs)(Blk2)(Bs){B}
  \lens[abspos=1, lensradius=1.4](Bs)(Out){L}
  \lens[abspos=2.8, lens=0.7 0.7 0.7, n=2](Bs)(Out){L}
  \opttripole(Bs)(Slm)(Bs){%
    \psframe[fillstyle=solid,fillcolor=gray!50](-0.3,0)(0.3,0.2)
    \psline(-0.3,0.1)(-0.5,0.1)(-0.5,0.5)(0.5,0.5)(0.5,0.1)(0.3,0.1)}{SLM}
  \addtopsstyle{Beam}{linestyle=none, fillstyle=solid, fillcolor=green!70!black}
  \drawwidebeam[beamwidth=0.1]{1-4}
  \drawwidebeam[beamwidth=0.1, beaminsidefirst, beaminsidelast]{3}{5-8}
  \psset{savebeampoints=false, loadbeampoints}
  \drawwidebeam{8-11}(Out)\drawwidebeam[beaminsidefirst]{8}{12}
\end{optexp}
\end{pspicture}
```

Ex. 11.7



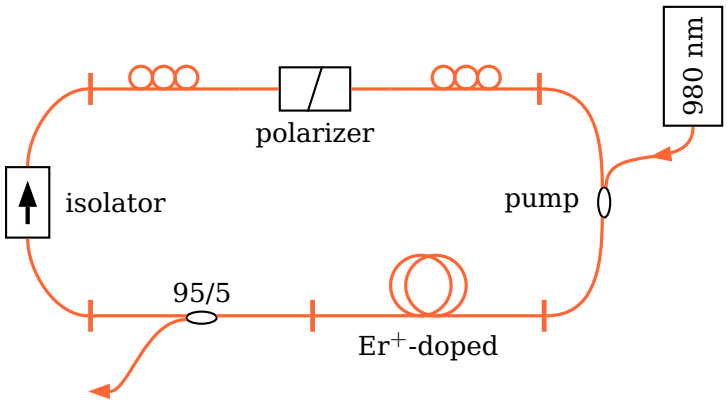

```
\begin{pspicture}(6.4,3.2)
\addtopsstyle{Fiber}{linecolor=red}\psset{fiber=none}
\pnodes(2.3,2.3){Lin}([Xnodesep=4.5]Lin){Det}
\optbox[label=0.2, position=start, compname=L, extnode=b](Lin)(Det){%
  \psGauss[yunit=0.03,sigma=0.03]{-0.5}{0.5}}
\optbox[label=0, compname=EAM, extnode=b, abspos=1.5](Lin)(Det){EAM}
\optfiber[labeloffset=0.3, abspos=3.2](Lin)(Det){fibre}
\optdetector(Lin)(Det){OSA}
\pnode([Xnodesep=-0.5,offset=-1]\oenodeExt{L}){Osc}
\elecsynthesizer[wire=none](Osc)(\oenodeExt{EAM}|Osc){10\,GHz}
\optarrowcomp[arrowcompsize=0.8 0.6, arrowscale=1.5](Osc)(\oenodeExt{EAM}|Osc){$\tau$}
\drawfiber{1-4}
\addtopsstyle{Wire}{arrows=->, arrowinset=0, arrowscale=1.5}
\drawwire{5}{6}\psset[optexp]{wireangleB=90}
\drawwire{6}(\oenodeExt{EAM})\drawwire{5}(\oenodeExt{L})
\end{pspicture}
```

Ex. 11.8



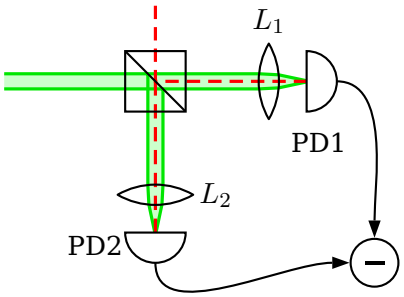

```
\begin{pspicture}(0.2,0.6)(8.2,3.5)
\pnodes(2,3){Laser}(2,1){PwMeter}(6,3){CplTop}(6,1){CplBot}
\psset{arrowscale=1.5, arrowinset=0}
\optbox[position=start, optboxsize=1.8 1, labeloffset=0](Laser)([Xnodesep=0.1]Laser){%
  \begin{tabular}{@{}c@{}}Nd:YAG\\[-0.4ex]cw laser\end{tabular}}
\optcoupler[addtoFiberIn1={ArrowInside=->}, addtoFiberIn2={ArrowInside=-<},
          labeloffset=0.4](Laser)(PwMeter)(CplTop)(CplBot){WDM}
\optfiber[addtoFiberOut={ncurv=1, angleB=0}, addtoFiberIn={ncurv=1, angleA=0},
        compshift=-1, label=0.2 . l](CplBot)(CplTop){fiber}
\optdetector[dettype=diode]([Xnodesep=0.1]PwMeter)(PwMeter){power meter}
\end{pspicture}
```

Ex. 11.9



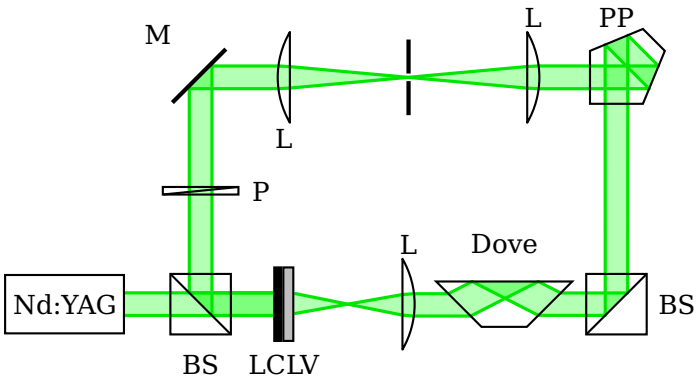

```
\begin{pspicture}(0.9,0.9)(10.4,6.1)
\psset{arrowscale=1.5, arrowinset=0}
\pnodes(2,5){PC1in}(4,5){PC1out}(6,5){PC2in}(8,5){PC2out}(2,2){CplSig}
\pnodes(5,2){CplIn}(2,1){CplOut}(10,4.5){Pump}(8,2){PumpSig}
\optisolator[compshift=0.8, addtoFiberIn={angleA=180},
           addtoFiberOut={angleB=180}, label=0.5 . l]%
           (CplSig)(PC1in){isolator}
\polcontrol[addtoFiberIn={arrows=|-}](PC1in)(PC1out)
\optfiberpolarizer[labeloffset=0.6](PC1out)(PC2in){polarizer}
\polcontrol[addtoFiberOut={arrows=-|}](PC2in)(PC2out)
\wdmsplitter[labeloffset=0.3, coupleralign=bottom, addtoFiberIn={arrows=|-},
           addtoFiberOut1={arrows=->}, addtoFiberOut2={arrows=-|}]%
           (CplIn)(CplOut)(CplSig){95/5}
\wdmcoupler[addtoFiberIn1={ArrowInside=->}, addtoFiberIn2={angleA=0},
          addtoFiberOut={angleB=0,arrows=-|}, ncurv=0.9,
          coupleralign=bottom, compshift=0.8](Pump)(PC2out)(PumpSig){pump}
\optbox[position=start, innerlabel, optboxwidth=1.6](Pump)([offset=-0.1]Pump){980~nm}
\optfiber[fiberloops=2, labeloffset=0.4](CplIn)(PumpSig){Er$^+$-doped}
\end{pspicture}
```

Ex. 11.10



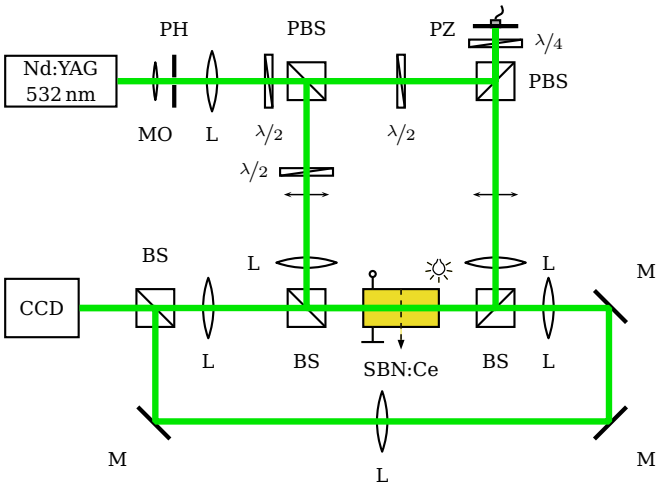

```
\begin{pspicture}(1,1.2)(6.2,5)
\pnodes(1,4){SigIn}(3,4){BS}(3,5){LO}(5,4){Det1}(3,2){Det2}
\begin{optexp}
  \optplane[compname=LO, angle=90](LO)
  \beamsplitter[compname=BS](SigIn)(BS)(Det2)
  \lens[abspos=0.5, compname=L2, n=2.1](Det2)(BS){$L_2$}
  \lens[abspos=0.5, compname=L1, n=2.1](Det1)(BS){$L_1$}
  \psset[optexp]{extnodealign=rel, extnode=r}
  \optdetector[compname=Det1](BS)(Det1){PD1}
  \optdetector[compname=Det2](BS)(Det2){PD2}
  \addtopsstyle{Beam}{beamwidth=0.2, fillstyle=solid,
                      fillcolor=green, opacity=0.2}
  \drawwidebeam(SigIn){BS}{L2}{Det2}
  \drawwidebeam[beaminsidefirst]{BS}{L1}{Det1}
  \newpsstyle{Beam}{linecolor=red, linestyle=dashed, linewidth=1.5\pslinewidth}
  \drawbeam{LO}{BS}{L1}{Det1}
  \drawbeam[beaminsidefirst]{BS}{L2}{Det2}
\end{optexp}
\cnodeput[framesep=5pt]([Xnodesep=0.5,
    offset=-0.5]\oenodeExt{Det1}|\oenodeExt{Det2}){M}{\rule{2.5mm}{1pt}}
\psset{arrows=<-, arrowscale=1.5, arrowinset=0}
\nccurve[angleA=90]{M}{\oenodeExt{Det1}}
\nccurve[angleA=180, angleB=-90]{M}{\oenodeExt{Det2}}
\end{pspicture}
```

Ex. 11.11



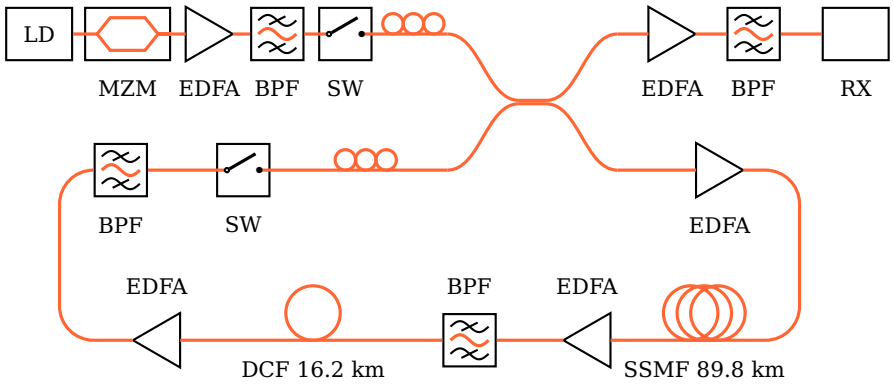

```
\begin{pspicture}(-0.2,0)(9,5)
\psset[optexp]{lens=1.2 0 1.2, n=1.72}
\addtopsstyle{Beam}{fillstyle=solid, fillcolor=green, opacity=0.3}
\pnodes(2.4,1){BS1}([offset=3]BS1){M1}([Xnodesep=5.5]M1){PP}(PP|BS1){BS2}
\begin{optexp}
  \optbox[label=0, position=start, optboxwidth=1.6]([Xnodesep=-1]BS1)(BS1){Nd:YAG}
  \beamsplitter[compname=BS](BS2)(BS1)(M1){BS}
  \optdipole[position=0.2, compname=LCLV, optdipolesize=0.28 1](BS1)(BS2){%
    \psframe[fillstyle=solid,fillcolor=black,dimen=outer](-0.14,-0.5)(-0.02,0.5)
    \psframe[fillstyle=solid,fillcolor=gray!50,dimen=outer](-0.02,-0.5)(0.14,0.5)
  }{LCLV}
  \optretplate(BS1)(M1){P}
  \mirror(BS1)(M1)(PP){M}
  \lens[position=0.2](M1)(PP){L}
  \pinhole(M1)(PP)
  \lens[position=0.2](PP)(M1){L}
  \pentaprism(M1)(PP)(BS2){PP}
  \beamsplitter(PP)(BS2)(BS1){BS}
  \doveprism[compname=Dove, position=0.27, n=2.3](BS2)(BS1){Dove}
  \lens[n=2.5](BS2)(BS1){L}
  \drawwidebeam[beamwidth=0.3]{1-3}
  \drawwidebeam[loadbeampoints]{3}{2}{4-}{3}
\end{optexp}
\end{pspicture}
```

Ex. 11.12



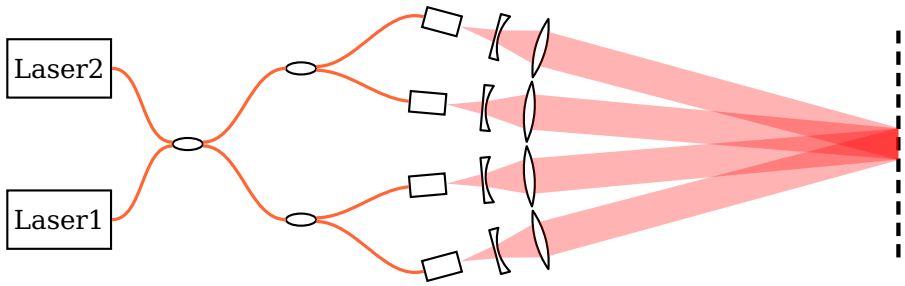

```
\begin{pspicture}(0,-0.2)(8.6,5.6)
\pnodes(1.5,5){Laser}(4,5){PBS}(6.5,5){PBS2}(6.5,5.7){piezo}(4,2){BSFwd}
\pnodes(6.5,2){BSBwd}(2,2){BS4f}(2,0.5){M4f3}(8,2){M4f1}(8,0.5){M4f2}(1,2){CCD}
\psset{mirrorwidth=0.6, plateheight=0.7, outerheight=0.7, labeloffset=0.7,
    labelstyle=\scriptsize, lens=1.2 1.2 0.8, bssize=0.5}
\optbox[position=start, optboxsize=1.5 0.7, innerlabel]%
  (Laser)(PBS){\parbox{1.5cm}{\centering Nd:YAG\\ 532\,nm}}
\lens[lensheight=0.5, position=0.2](Laser)(PBS){MO}
\pinhole[position=0.3,labelangle=180](Laser)(PBS){PH}
\lens[position=0.5](Laser)(PBS){L}
\optretplate[position=0.8](Laser)(PBS){$\nicefrac{\lambda}{2}$}
\beamsplitter(Laser)(PBS)(BSFwd){PBS}
\optretplate[position=0.4](PBS)(BSFwd){$\nicefrac{\lambda}{2}$}
\polarization(PBS)(BSFwd)\polarization(PBS2)(BSBwd)
\lens[position=0.8](PBS)(BSFwd){L}
\optretplate(PBS)(PBS2){$\nicefrac{\lambda}{2}$}
\beamsplitter(PBS)(PBS2)(piezo){PBS}
\optretplate[abspos=0.5](PBS2)(piezo){$\nicefrac{\lambda}{4}$}
\mirror[mirrortype=piezo,labelangle=90](PBS2)(piezo)(PBS2){PZ}
\lens[position=0.8,labelangle=180](PBS2)(BSBwd){L}
\crystal[crystalsize=1 0.5, voltage, lamp, fillstyle=solid,
    fillcolor=yellow!90!black, labeloffset=0.8](BSFwd)(BSBwd){SBN:Ce}
\beamsplitter(PBS)(BSFwd)(BSBwd){BS}
\beamsplitter[labelangle=-90](PBS2)(BSBwd)(BSFwd){BS}
\mirror(BSBwd)(M4f1)(M4f2){M}\mirror(M4f1)(M4f2)(M4f3){M}
\lens[labelangle=180](M4f2)(M4f3){L}\mirror(M4f2)(M4f3)(BS4f){M}
\beamsplitter(M4f3)(BS4f)(CCD){BS}
\optbox[position=end, label=0, optboxwidth=1](BS4f)(CCD){CCD}
\lens[abspos=0.7](BS4f)(BSFwd){L}\lens[abspos=0.7](BSBwd)(M4f1){L}
\addtopsstyle{Beam}{linewidth=2\pslinewidth, linejoin=2}
\drawbeam{1-6}{11-14}{13}{12}{15}{18}{16}{17}{25}{23}{24}
\drawbeam[beaminsidefirst]{6}{7}{10}{17}{16}{18}{26}{19-24}
\end{pspicture}
```

Ex. 11.13



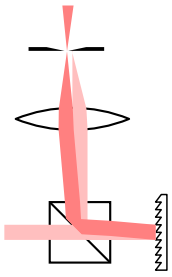

```
\begin{pspicture}(13,5.5)
\psset{optboxwidth=1, labelstyle=\footnotesize, fiber=none}
\pnodes(1,5){LD}([Xnodesep=5.5]LD){CPLin1}([offset=-2]CPLin1){CPLin2}
\pnodes([Xnodesep=2.5]CPLin1){CPLout1}([Xnodesep=2.5]CPLin2){CPLout2}
\pnode([Xnodesep=3]CPLout1){RX}
\optbox[position=start, label=0](LD)(CPLin1){LD}
\optmzm[abspos=0.8](LD)(CPLin1){MZM}
\optamp[abspos=2](LD)(CPLin1){EDFA}
\optfilter[abspos=3](LD)(CPLin1){BPF}
\optswitch[abspos=4](LD)(CPLin1){SW}
\polcontrol[abspos=5, fiber=out](LD)(CPLin1)
\drawfiber{1-6}
\optcoupler[couplertype=none, fiber](CPLin1)(CPLin2)(CPLout1)(CPLout2)
\optamp[abspos=0.8, fiber=in](CPLout1)(RX){EDFA}
\optfilter[abspos=2](CPLout1)(RX){BPF}
\optbox[position=end](CPLout1)(RX){RX}
\drawfiber{8-10}
\pnodes([Xnodesep=-0.5]LD|CPLin2){TL}(RX|TL){TR}([offset=-2.5]TR){BR}(TL|BR){BL}
\optamp[fiber=i](CPLout2)(TR){EDFA}
\optfiber[label=0.3 . t, position=0.85](BL)(BR){SSMF 89.8~km}
\drawfiber[fiberstyle=angle, arm=1.2, linearc=0.5, startnode=N]{11}{12}
\optamp[position=0.3](BR)(BL){EDFA}
\optfilter[position=0.55, labelangle=180](BL)(BR){BPF}
\optfiber[fiberloops=1, label=0.3 . t, position=0.35](BL)(BR){DCF 16.2~km}
\optamp[position=0.85](BR)(BL){EDFA}
\optfilter[position=0.2](TL)(CPLin2){BPF}
\optswitch(TL)(CPLin2){SW}
\polcontrol[position=0.8, fiber=out](TL)(CPLin2)
\drawfiber{12-16}\drawfiber{17-19}
\drawfiber[fiberstyle=angle, arm=0.5, linearc=0.5, stopnode=1]{16}{17}
\end{pspicture}
```

Ex. 11.14



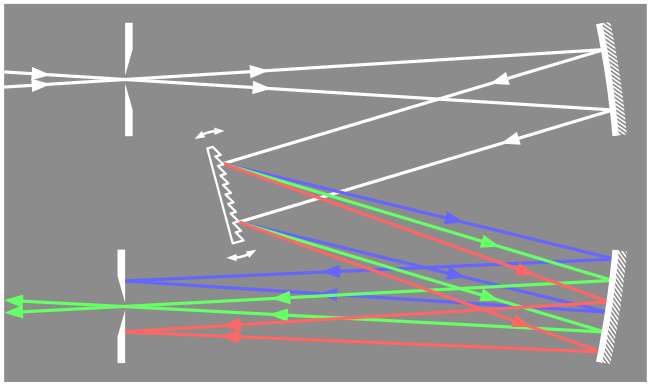

```
\begin{pspicture}(12, 4)
  \psset{labeloffset=0.3, fiber=none}
  \pnodes(1.6,2){In}(12, 2){Ref}
  \optplate[linestyle=dashed, plateheight=3, position=1, compname=RefPlane](In)(Ref)
  \addtopsstyle{Beam}{linestyle=none, beamdiv=20, beaminside=false, fillstyle=solid,
      fillcolor=red, opacity=0.3}
  \multido{\i=1+1, \ii=165+10}{4}{%
    \rput(Ref){\pnode(6;\ii){A\i}}
    \optbox[optboxsize=0.5 0.3, position=end, compname=Box\i, extnode=l](Ref)(A\i)
    \lens[lens=0 -0.6 0.6, abspos=0.5, compname=L\i](A\i)(Ref)
    \lens[lens=1.2 1.2 0.8, abspos=1.1, n=1.65, compname=LC\i](A\i)(Ref)
    \drawwidebeam{Box\i}{L\i}{LC\i}{RefPlane}
  }%
  \optcoupler[abspos=1](In)(In)(Ref)(Ref)
  \wdmsplitter[abspos=2.5, compshift=1](In)(Ref)(Ref)
  \wdmsplitter[abspos=2.5, compshift=-1](In)(Ref)(Ref)
  \optbox[compshift=-1, position=start, label=0](In)(Ref){Laser1}
  \optbox[compshift=1, position=start, label=0](In)(Ref){Laser2}
  \drawfiber{17}{14}{15}{Box1}\drawfiber{15}{Box2}
  \drawfiber{18}{14}{16}{Box3}\drawfiber{16}{Box4}
\end{pspicture}
```

Ex. 11.15



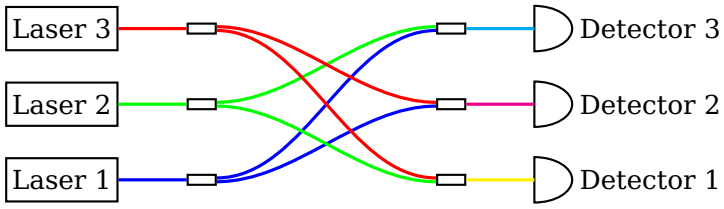
```
\begin{pspicture}(0.5,0)(2.2,4)
  \pnodes(0,1){A}(1,1){BS}(2,1){G}(1,4){B}
  \beamsplitter(G)(BS)(B) \optgrating(BS)(G)(BS)
  \lens[lens=2 2 1.5, compshift=0.1, n=2.25](BS)(B)
  \pinhole[phwidth=0.05, innerheight=0.05, position=0.8,
      compshift=0.18](BS)(B)
  \optplane[angle=90](B)
  \addtopsstyle{Beam}{linestyle=none, beamwidth=0.2, fillstyle=solid}
  \drawwidebeam[fillcolor=red!25!white](A){1-2}{1}{3-5}
  \drawwidebeam[fillcolor=red!50!white, beamangle=-5]{2}{1}{3-5}
\end{pspicture}
```

Ex. 11.16: Adapted from Wikipedia



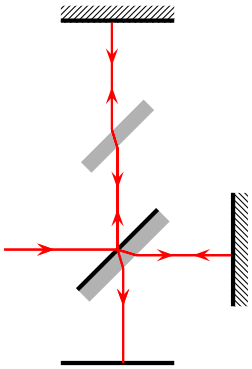
```
\begin{pspicture}(0,-1)(8.5,4)
\psframe[fillstyle=solid,fillcolor=gray!90,linestyle=none](0,-1)(8.5,4)
\addtopsstyle{OptComp}{linecolor=white}
\newpsstyle{Beam}{ArrowInside=->, linejoin=2, arrowscale=1.3, arrowinset=0}
\addtopsstyle{ExtendedMirror}{hatchcolor=white, hatchsep=0.5\pslinewidth}
\pnodes(0,3){A}(8,3){B}(3,1.5){C}(8,0){D}(0,0){E}%
\psset{linewidth=1.5\pslinewidth, mirrorradius=13, mirrorwidth=1.5,
    gratingwidth=1.3, mirrortype=extended, phwidth=0.1, outerheight=1.5}%
\begin{optexp}
  \pinhole[position=0.2](A)(B)%
  \mirror(A)(B)(C)
  \optgrating[reverse, angle=15, variable](B)(C)(D)
  \mirror(C)(D)(E)
  \pinhole[position=0.8](D)(E)
  \drawwidebeam[ArrowInsidePos=0.3, beamwidth=0.2, linecolor=white,
      beamdiv=-7.15](A){1-3}
  \psset{loadbeampoints, savebeampoints=false}%
  \drawwidebeam[ArrowInsidePos=0.6, beamangle=3, linecolor=blue!60!white]{3-5}(E)
  \drawwidebeam[ArrowInsidePos=0.7, linecolor=green!60!white, arrows=->,
      ArrowInsideMinLength=2]{3-5}(E)
  \drawwidebeam[ArrowInsidePos=0.8, beamangle=-3, linecolor=red!60!white]{3-5}(E)
\end{optexp}
\end{pspicture}
```

Ex. 11.17



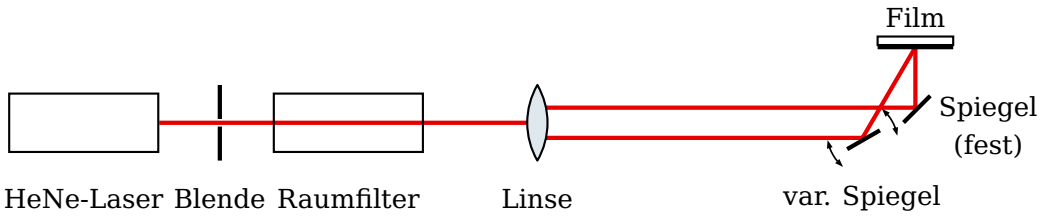

```
\begin{pspicture}(0,0.7)(9.4,3.3)
  \psset[optexp]{couplertype=rectangle, fiber=none, detsize=0.5 0.6}
  \multido{\i=1+1}{3}{%
    \optbox[optboxsize=1.5 0.6, innerlabel,position=start](1.5,\i)(7,\i){Laser \i}
    \wdmsplitter[position=0.2](1.5,\i)(7,\i)(7,\i)
    \wdmcoupler[position=0.8](1.5,\i)(1.5,\i)(7,\i)
    \optdetector[label=0.4 90 l](1.5,\i)(7,\i){Detector \i}
  }%
  \drawfiber[linecolor=blue, startnode=N]{1}{2}{7}
  \drawfiber[linecolor=blue]{2}{11}
  \drawfiber[linecolor=green, stopnode=1]{5}{6}{11}
  \drawfiber[linecolor=green, stopnode=2]{6}{3}
  \drawfiber[linecolor=red]{9}{10}{3}
  \drawfiber[linecolor=red, startnode=2]{10}{7}
  \drawfiber[linecolor=cyan]{11}{12}
  \drawfiber[linecolor=magenta]{7}{8}
  \drawfiber[linecolor=yellow]{3}{4}
\end{pspicture}
```

Ex. 11.18



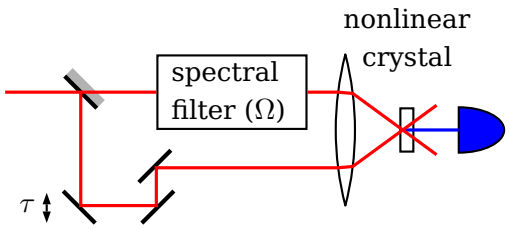
```
\psset{unit=1.5}
\begin{pspicture}(0,1)(2.2,4.2)
  \pnodes(0,2){In}(1,2){G}(1,4){M1}(2,2){M2}(1,1){S}
  \mirror[mirrortype=semitrans](In)(G)(M1)
  \optbox[optboxwidth=0.15, angle=45, style=SemitransMirror](G)(M1)
  \mirror[mirrortype=extended](G)(M1)(G)
  \mirror[mirrortype=extended](G)(M2)(G)
  \optplate[position=end](G)(S)
  \newpsstyle{Beam}{linecolor=red, ArrowInside=->, ❯
      ArrowInsidePos=0.4, arrowscale=1.5}
  \drawbeam(In){1}{4}{1}{5}
  \drawbeam(In){1-3}{2}{1}{5}
\end{pspicture}
```

Ex. 11.19: Adapted from semibyte.de



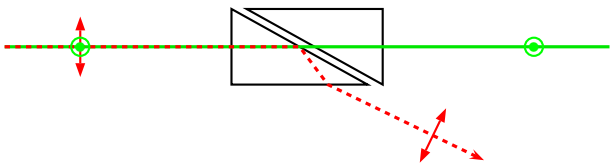

```
\begin{pspicture}(13.6,2.6)
  \psset[optexp]{labeloffset=1, mirrorwidth=0.5, optboxwidth=2}
  \pnodes(2,1){Laser}(12,2){Film}(Film|Laser){Ref}
  \begin{optexp}
    \optbox[position=start](Laser)(Ref){HeNe-Laser}
    \pinhole[abspos=0.8](Laser)(Ref){Blende}
    \optbox[abspos=2.5](Laser)(Ref){Raumfilter}
    \lens[fillstyle=solid, fillcolor={[rgb]{0.87,0.91,0.93}}](Laser)(Ref){Linse}
    \mirror[label=1 30]([offset=0.2]Laser)([offset=0.2]Ref)(Film){%
      \begin{tabular}{c}Spiegel\\(fest)\end{tabular}}
    \opttripole[label=0.4](Ref)(Film)(Ref){%
      \psframe(0.5,1\pslinewidth)(-0.5,5\pslinewidth)
      \psline[linewidth=2\pslinewidth](0.5,0)(-0.5,0)
    }{Film}
    \newpsstyle{Beam}{linewidth=2\pslinewidth, linecolor=red!90!black,
       beamalign=absolute}
    \drawbeam{1-4}
    \drawbeam[beampos=0.2]{4-6}
    \mirror[variable, label=0.8 -30]([offset=-0.2]Laser)([offset=-0.2,
       Xnodesep=-0.7]Ref)(Film){var. Spiegel}
    \drawbeam[beampos=-0.2]{4}{7}{6}
  \end{optexp}
\end{pspicture}
```

Ex. 11.20



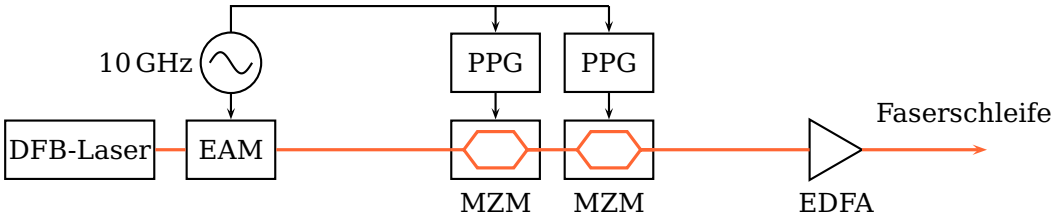

```
\begin{pspicture}(0,0.2)(6.6,3.1)
  \psset{mirrorwidth=0.6, useNA=false, beaminsidefirst}
  \pnodes(0,2){In}(1,2){BS}(1,0.5){M1}(2,1){M2}(6,1.5){Det}
  \mirror[mirrortype=semitrans, n=1](In)(BS)(M1)
  \mirror[label=0.6 180 . absolute, compname=M](BS)(M1)(M2|M1)
  \mirror(M1)(M2|M1)(M2)
  \mirror(M2|M1)(M2)([Xnodesep=1]M2)
  \optbox[optboxsize=2 1, abspos=2, label=0, ↵
     allowbeaminside=false](BS)([offset=0.5]Det){%
    \parbox{1.6\psxunit}{spectral\\filter ($\Omega$)}}
  \lens[lens=4 4 2, n=3.3]([Xnodesep=-3]Det)(Det)
  \optbox[optboxsize=0.2 0.6, n=1, label=-1.2](\oenodeOut{6})(Det){%
    \begin{tabular}{@{}c@{}}nonlinear\\ crystal\end{tabular}}
  \optplate[linestyle=none](\oenodeOut{7})(Det)
  \optdetector[detsize=0.6 0.6, fillstyle=solid, fillcolor=blue](\oenodeOut{6})(Det)
  \newpsstyle{Beam}{linewidth=1.5\pslinewidth}
  \drawbeam[linecolor=blue]{7-9}
  \addtopsstyle{Beam}{linecolor=red}
  \drawbeam(In){1-4}{6-8}
  \drawbeam{1}{5-8}
  \rput[r](\oenodeLabel{M}){$\tau$
    \psline[arrows=<->, arrowinset=0](0.15,0.25)(0.15,-0.15)}
\end{pspicture}
```

Ex. 11.21



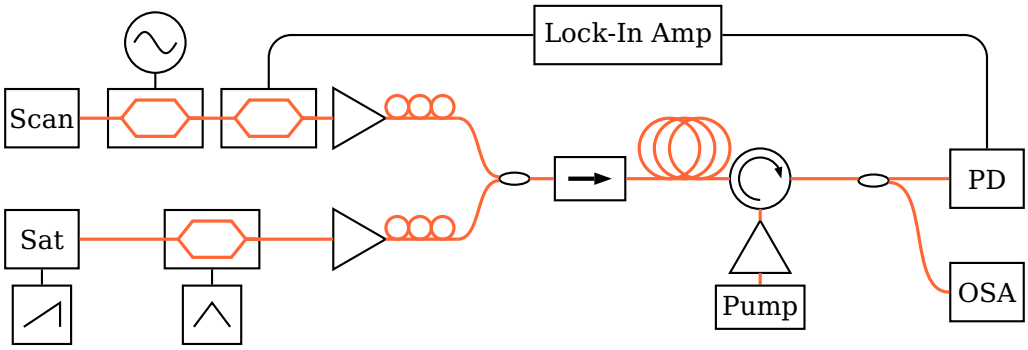
```
\begin{pspicture}(-1,-0.5)(8,1.5)
  \newpsobject{perppol}{polarization}{poltype=perp, linecolor=green, polsize=1}
  \newpsobject{parpol}{polarization}{poltype=parallel, linecolor=red, polsize=0.8}
  \newpsstyle{Polarization}{arrowinset=0, arrowscale=1.2}
  \pnodes(0,1){A}(8,1){B}(4,-0.5){C}
  \glanthompson[glanthompsongap=0.2, glanthompsonsize=2 1](A)(B)
  \optplane(B)\optplane[angle=90](C)
  \drawbeam(A){1-2}
  \drawbeam[linestyle=dashed, dash=2pt 2pt, linecolor=red, arrows=->](A){1}{3}
  \parpol[abspos=1](A)(B)\perppol[abspos=1](A)(B)
  \nodexn{(\oenodeBeam{})-1.5(!\oeBeamVec{})}{D}
  \perppol[abspos=7](A)(B)\parpol(D)(\oenodeBeam{})
\end{pspicture}
```

Ex. 11.22



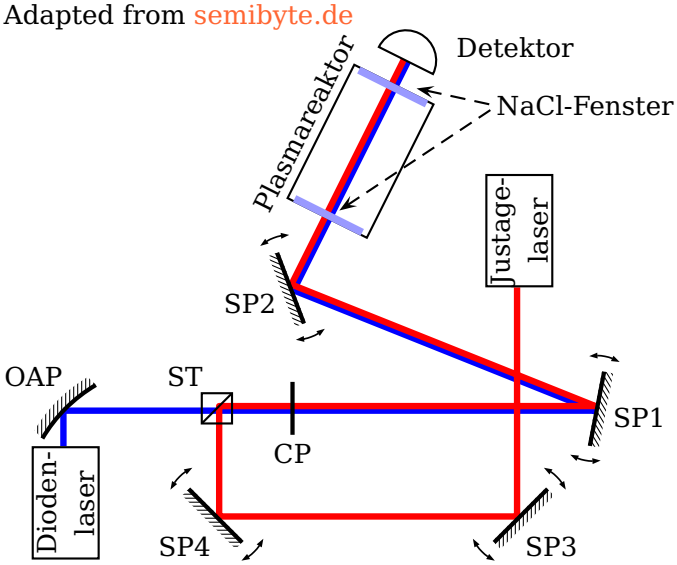

```
\begin{pspicture}(13.7,2.9)
\newpsstyle{Wire}{arm=0.1, arrows=->}
\psset[optexp]{optboxsize=1.2 0.8, optmzmsize=1.2 0.8, fiber=none, label=0.7}
\pnodes(2,1){L}([Xnodesep=11]L){A}
\optbox[position=start, optboxwidth=2, label=0](L)(A){DFB-Laser}
\optbox[abspos=1, label=0, compname=Eam, extnode=t](L)(A){EAM}
\optmzm[abspos=4.5, compname=Mzm1, extnode=t](L)(A){MZM}
\optmzm[abspos=6, compname=Mzm2, extnode=t](L)(A){MZM}
\optamp[abspos=9](L)(A){EDFA}
\drawfiber{1-5}\drawfiber[arrows=->]{}(A)
\pnode([offset=1.5]\oenodeExt{Eam}){Osci}
\elecsynthesizer[compname=Syn, position=0.5, wire=o, label=0.5 . ↩
    r](Osci)(\oenodeExt{Eam}){10\,GHz}
\psset[optexp]{label=0, wire, optboxsize=0.8 1.2}
\optbox[compname=Ppg1](\oenodeExt{Mzm1}|Osci)(\oenodeExt{Mzm1}){PPG}
\optbox[compname=Ppg2](\oenodeExt{Mzm2}|Osci)(\oenodeExt{Mzm2}){PPG}
\drawwire[arrows=-, wireangleB=-90]{Syn}(\oenodeExt{Mzm2}|Osci)
\rput([offset=0.5, Xnodesep=-0.3]A){Faserschleife}
\end{pspicture}
```

Ex. 11.23: Adapted from DOI:10.1364/OL.37.000930



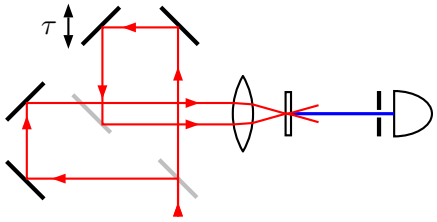

```
\begin{pspicture}(0,0.2)(13.5,4.7)
\psset[optexp]{optboxsize=1 0.8, label=0, fiber=none}
\pnodes(1,3.2){Scan}\pnode([offset=-1.6]Scan){Sat}
\pnode([Xnodesep=5]Scan){ScanCpl}\pnode(ScanCpl|Sat){SatCpl}
\pnode([Xnodesep=1.5,offset=-0.8]ScanCpl){Cpl}\pnode([Xnodesep=5]Cpl){PD}
\pnode([Xnodesep=2.5, offset=-1.4]Cpl){Pump}
\optbox[position=start](Scan)(ScanCpl){Scan}
\optmzm[abspos=1, compname=M1, extnode=t](Scan)(ScanCpl)
\optmzm[abspos=2.5, compname=M2, extnode=t](Scan)(ScanCpl)
\optamp[abspos=3.7](Scan)(ScanCpl)
\polcontrol[abspos=4.5, fiber=o](Scan)(ScanCpl)
\drawfiber{1-5}
\optbox[position=start, compname=S, extnode=b](Sat)(SatCpl){Sat}
\optmzm[abspos=1.75, compname=M3, extnode=b](Sat)(SatCpl)
\optamp[abspos=3.7](Sat)(SatCpl)
\polcontrol[abspos=4.5, fiber=o](Sat)(SatCpl)
\drawfiber{6-9}
\wdmcoupler[fiber=i](ScanCpl)(SatCpl)(Cpl)
\optisolator[position=0.1](Cpl)(Pump|Cpl)
\optfiber[position=0.6](Cpl)(Pump|Cpl)
\addtopsstyle{OptCircArrow}{arrows=<-}
\optcirculator[optcircangle=0 -90](Cpl)(PD)(Pump)
\wdmsplitter[position=0.8, coupleralign=t](Cpl)(PD)
\optbox[position=end, compname=Pd, extnode=t](Cpl)(PD){PD}
\optbox[position=end, compshift=-1.5](Cpl)(PD){OSA}
\optamp[fiber](Pump)(\oenodeIfc{2}{13})
\optbox[position=start, optboxsize=0.6 1.2](Pump)(Pump|Cpl){Pump}
\drawfiber{10-15}\drawfiber{14}{16}
\optbox[compshift=0.7, optboxsize=2.5
    0.8](\oenodeExt{M2})(\oenodeExt{Pd}|\oenodeExt{M2}){Lock-In Amp}
\addtopsstyle{Wire}{arrm=0.1}
\drawwire[wireangleA=90, linearc=0.3](\oenodeExt{M2}){}
\drawwire[wireangleB=90, linearc=0.3]{}(\oenodeExt{Pd})
\elecsynthesizer([offset=0.2]\oenodeExt{M1})(\oenodeExt{M1})
\psset[optexp]{synthshape=rectangle}
\elecsynthesizer[synthtype=sawtooth](\oenodeExt{S}|Pump)(\oenodeExt{S})
\elecsynthesizer[synthtype=triangle](\oenodeExt{M3}|Pump)(\oenodeExt{M3})
\end{pspicture}
```

Ex. 11.24: Adapted from semibyte.de



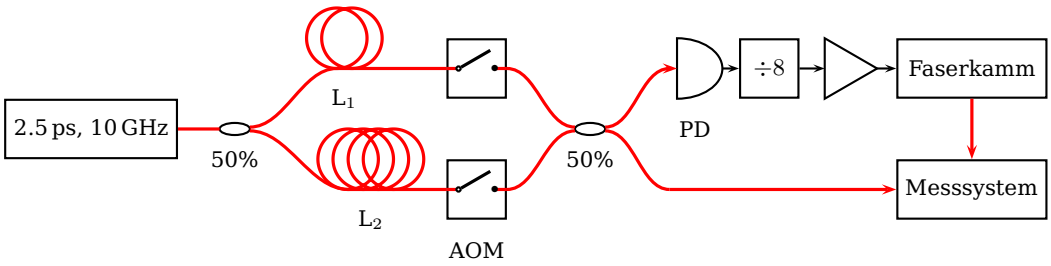

```
\begin{pspicture}(0.2,0.5)(9,7.4)
  \pnode(1,2){DL}\pnode([offset=0.5]DL){OAP}\pnode([Xnodesep=2]OAP){ST}
  \pnode([offset=-1.4]ST){SP4}\pnode([Xnodesep=4]SP4){SP3}
  \pnode([offset=3]SP3){JL}\pnode([Xnodesep=1]SP3|ST){SP1}
  \pnode([Xnodesep=1]ST|JL){SP2}\pnode([Xnodesep=1.5,offset=3]SP2){Det}
  \psset[optexp]{optboxsize=1.5 0.9, labeloffset=0.6}
  \begin{optexp}
    \optbox[position=start, innerlabel](DL)(OAP){\parbox{1.5\psunit}{\centering
       Dioden-\\laser}}
    \psset[optexp]{mirrortype=extended}
    \mirror[mirrorradius=2](DL)(OAP)(ST){OAP}
    \psset[optexp]{variable}
    \beamsplitter[bssize=0.4, labelangle=-45](SP4)(ST)(SP1){ST}
    \optplate[abspos=1, plateheight=0.7](ST)(SP1){CP}
    \mirror(ST)(SP1)(SP2){SP1}\mirror(SP1)(SP2)(Det){SP2}
    \optdipole[label=-0.7 . b relative, optdipolesize=2, position=0.6](SP2)(Det){%
      \psframe[dimen=outer](-1,-0.6)(1,0.6)
      \psset{fillstyle=solid, fillcolor=blue!40, linestyle=none}
      \psframe(-1,-0.5)(-0.9,0.5)\psframe(0.9,-0.5)(1,0.5)}{Plasmareaktor}
    \optdetector[label=. 30 l](SP2)(Det){Detektor}
    \optbox[position=end, innerlabel](SP3)(JL){\parbox{1.5\psunit}{\centering
       Justage-\\laser}}
    \mirror(SP4)(SP3)(JL){SP3}\mirror(ST)(SP4)(SP3){SP4}
    \psset{linewidth=2\pslinewidth}
    \drawbeam[linecolor=blue, beampos=0.03]{1-2}
    \drawbeam[beamalign=abs, loadbeampoints, linecolor=blue]{2-8}
    \drawbeam[linecolor=red, beampos=-0.03]{9-11}{3-8}
  \end{optexp}
  \pnode([Xnodesep=1.2,offset=-0.6]\oenodeIn{8}){NaCl}
  \psset{linestyle=dashed, arrows=->, arrowscale=1.5}
  \ncline[nodesepA=0.1, nodesepB=0.2]{NaCl}{\oenodeIn{7}}
  \ncline[nodesepA=0.1, nodesepB=0.3]{NaCl}{\oenodeOut{7}}
  \rput[l](NaCl){NaCl-Fenster}
\end{pspicture}
```

Ex. 11.25: Adapted from DOI:10.1364/AOP.1.000308, Fig. 8



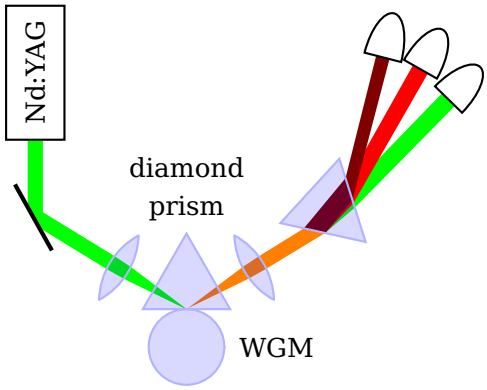

```
\begin{pspicture}(5.7,2.8)
\psset{mirrorwidth=0.7, bsstyle=plate, bssize=0.5, arrowinset=0, arrowscale=1.2}
\newpsobject{splitterplate}{beamsplitter}{linewidth=2\pslinewidth, linecolor=gray!50}
\pnodes(2.3,0){In}([offset=0.5]In){STM1}([Xnodesep=-2]STM1){M1}
\pnodes([offset=1]M1){M2}([offset=2]STM1){M3}([Xnodesep=-1]M3){M4}(M4|M2){STM2}
\splitterplate(In)(STM1)(M1)
\mirror(STM1)(M1)(M2)
\mirror(M1)(M2)(M3|M2)
\mirror(STM1)(M3)(M4)
\mirror[label=0.6 180 . absolute](M3)(M4)(STM2)
\rput[r](\oenodeLabel{}){$\tau$
  \psline[arrows=<->](0.15, 0.4)(0.15, -0.2)}
\splitterplate[compoffset=0.2, compname=STM2](M4)(STM2)([Xnodesep=1]STM2)
\pnode(\oenodeCenter{STM2}){STM2C}
\pnode([Xnodesep=4]STM2C){Out}
\lens[n=2](STM2C)(Out)
\optbox[position=0.65, optboxsize=0.1 0.6](STM2C)(Out)
\optplate[linestyle=none, position=0.75](STM2C)(Out)
\pinhole[position=0.95, outerheight=0.6](STM2C)(Out)
\optdetector[detsize=0.5 0.6](STM2C)(Out)
\drawbeam[linecolor=blue]{8-}
\newpsstyle{Beam}{ArrowInside=->, linecolor=red, ArrowInsideMinLength=1, arrows=>-}
\drawbeam[ArrowInsidePos=0.835](In){1-3}{7-9}
\drawbeam[ArrowInsidePos=0.74](In){1}{4-9}
\end{pspicture}
```

Ex. 11.26



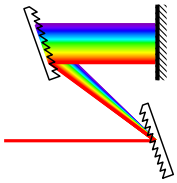

```
\begin{pspicture}(13.8,3.8)
  \pnode(2.3,2){Signal}
  \pnode([Xnodesep=1.5,offset=0.8]Signal){UpIn}
  \pnode([Xnodesep=1.5, offset=-0.8]Signal){LowIn}
  \pnode([Xnodesep=3.5]UpIn){UpOut}
  \pnode(UpOut|LowIn){LowOut}
  \pnode([Xnodesep=1.5]UpOut){UpIn2}
  \pnode([Xnodesep=3]UpIn2){UpOut2}
  \pnode([Xnodesep=1.5]LowOut){LowIn2}
  \pnode(UpOut2|LowIn2){LowOut2}
  \psset{labelstyle=\footnotesize}
  \psset{fiber=none}\addtopsstyle{Fiber}{linecolor=red}
  \psset{wire=none}\addtopsstyle{Wire}{arm=0.1, arrows=->}
  \optbox[position=start, label=0, optboxsize=2.3 ↵
      0.8](Signal)([Xnodesep=0.1]Signal){2.5\,ps, 10\,GHz}
  \wdmsplitter[label=0.4](Signal)(UpIn)(LowIn){50\%}
  \optfiber[fiberloops=4, label=0.4, position=0.3](LowIn)(LowOut){L$_2$}
  \optswitch[position=0.7](LowIn)(LowOut){AOM}
  \optfiber[fiberloops=2, label=0.4, position=0.2](UpIn)(UpOut){L$_1$}
  \optswitch[position=0.7](UpIn)(UpOut)
  \optcoupler[label=0.4, position=0.3](UpOut)(LowOut)(UpIn2)(LowIn2){50\%}
  \optbox[optboxsize=2 0.8, position=end, label=0, ↵
      extnode=t](LowIn2)(LowOut2){Messsystem}
  \drawfiber{1-4}{7}(LowIn2)\drawfiber[arrows=->](LowIn2){8}
  \optdetector[abspos=0.4, detsize=0.6 0.8](UpIn2)(UpOut2){PD}
  \optbox[optboxsize=0.8 0.8, label=0, position=0.44](UpIn2)(UpOut2){$\div 8$}
  \optamp[position=0.8](UpIn2)(UpOut2)
  \optbox[position=end, extnode=b, label=0, optboxsize=2 0.8](UpIn2)(UpOut2){Faserkamm}
  \drawfiber{2}{5-7}(UpIn2)\drawfiber[arrows=->](UpIn2){9}
  \drawwire{9-12}
  \drawfiber[arrows=->](\oenodeExt{12})(\oenodeExt{8})
\end{pspicture}
```

Ex. 11.27: Adapted from DOI:10.1103/PhysRevLett.105.263904



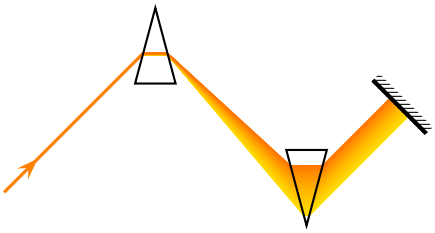

```
\begin{pspicture}(0.6,2.8)(6.8,7.8)
\pnodes(1,6){Laser}(1,5){M}(3,4){PR}(5,5){PT}(6,7){D1}
\newpsstyle{Glass}{fillcolor=blue, fillstyle=solid, opacity=0.15, ⤸
    linecolor=blue!30!white}
\begin{optexp}
  \optbox[position=start, innerlabel, optboxwidth=1.8](Laser)(M){Nd:YAG}
  \optprism[style=Glass, compname=P, n=1, prismtype=reflective, prismangle=60, ⤸
      labeloffset=1.2](M)(PR)(PT){\begin{tabular}{@{}c@{}}diamond\\ prism\end{tabular}}
  \mirror(Laser)(M)(\oenodeIfc{2}{P})%
  \lens[abspos=1.05, style=Glass](\oenodeIfc{2}{P})(M)%
  \optprism[compname=PDet, style=Glass](\oenodeIfc{2}{P})(PT)(D1)%
  \lens[abspos=1.05, style=Glass](\oenodeIfc{2}{P})(\oenodeOut{PDet})%
  \newpsstyle{Beam}{linestyle=none, fillstyle=solid}%
  \drawwidebeam[fillcolor=green, beamwidth=0.2, beaminsidelast, ⤸
      beampathcount=5]{1}{3}{4}{2}%
  \drawwidebeam[fillcolor=orange, beaminsidefirst, startinsidecount=1, loadbeam, ⤸
      beaminsidelast]{2}{6}%
  \psset{savebeam=false, loadbeam, beamnodealign=vector}%
  \drawwidebeam[fillcolor=orange]{6}{5}%
  \drawwidebeam[fillcolor=green, n=1.2, beampathskip=1, savebeam=2]{6}{5}(D1)
  \drawwidebeam[fillcolor=red, beampathskip=1, n=1.4, savebeam=3]{6}{5}(D1)
  \drawwidebeam[fillcolor=red!50!black, beampathskip=1, n=1.57, savebeam=4]{6}{5}(D1)
  \multido{\i=2+1}{3}{%
    \backlayer{%
      \pnode(!\oeBeamCenter{\i}){Det\i}
      \nodexn{(Det\i)-(!\oeBeamVecMedian{\i})}{Det\i'}}
    \optdetector[detsize=0.6 0.5](Det\i')(Det\i)}
\end{optexp}
\pscircle[style=Glass]([offset=-0.5]\oenodeIfc{2}{P}){0.5}
\rput[l]([offset=-0.5,Xnodesep=0.7]\oenodeIfc{2}{P}){WGM}
\end{pspicture}
```

Ex. 11.28



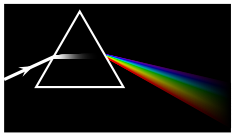
```
\begin{pspicture}(2.2,2.2)
\definecolor[ps]{bl}{rgb}{tx@addDict begin Red Green Blue end}%
\pnode(0,0.5){In}\pnode(2,0.5){G1}
\pnode([Xnodesep=-1.5, offset=1.3]G1){G2}\pnode(G1|G2){MG}
\begin{optexp}
  \optgrating[angle=40, gratingalign=c](In)(G1)(G2)
  \optgrating[reverse, angle=40, gratingalign=c](G1)(G2)(MG)
  \mirror[mirrorwidth=1, mirrortype=extended](G2)(MG)(G2)
  \drawbeam[linecolor=red](In){1}
  \multido{\i=0+1}{40}{%
    \pstVerb{\i\space 650 400 sub 39 div mul 400 add
           tx@addDict begin wavelengthToRGB end }%
    \drawbeam[linecolor=bl, beamangle=\i\space 16 sub 0.2 mul]{1-2}%
    \drawbeam[linecolor=bl, loadbeampoints]{2-3}
  }%
\end{optexp}
\end{pspicture}
```

Ex. 11.29



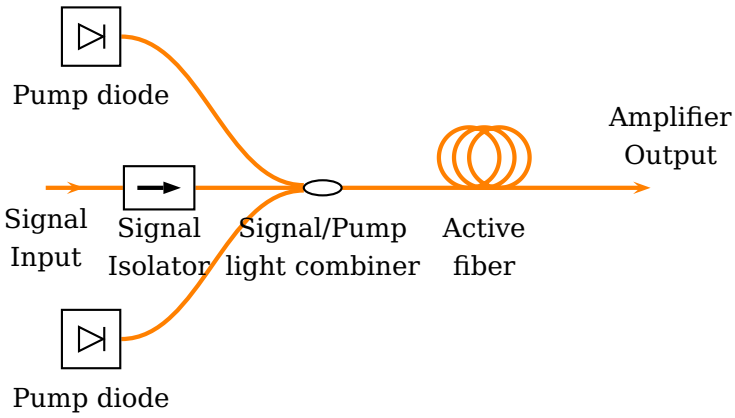
```
\begin{pspicture}(5.6,3)
\pnodes(0,0.5){S1}(2,2.5){S2}(3,1.5){S3}(4,0.5){S4}(4.5,1){S5}
\psset{prismangle=30, mirrortype=extended}
\definecolor[ps]{bl}{rgb}{tx@addDict begin Red Green Blue end}%
\begin{optexp}
  \optprism(S1)(S2)(S3)\optprism(S3)(S4)(S5)
  \mirror[compshift=-0.07](\oenodeIn{2})([offset=1,Xnodesep=1]\oenodeIn{2})(\oenodeIn{2})
  \drawbeam[linecolor=orange, ArrowInside=->, ArrowInsidePos=0.2, arrowscale=1.5](S1){1}
  \addtopsstyle{Beam}{linecolor=bl, linewidth=0.35\pslinewidth, beampathskip=1}
  \multido{\r=0+0.5}{120}{%
    \pstVerb{\r\space 620 590 sub 59 div mul 590 add
           tx@addDict begin wavelengthToRGB end }%
    \drawbeam[n=3.42 -0.002 \r\space mul add, beampos=\r\space 60 sub 0.0002
        mul](S1){1-3}
  }%
\end{optexp}
\end{pspicture}
```

Ex. 11.30



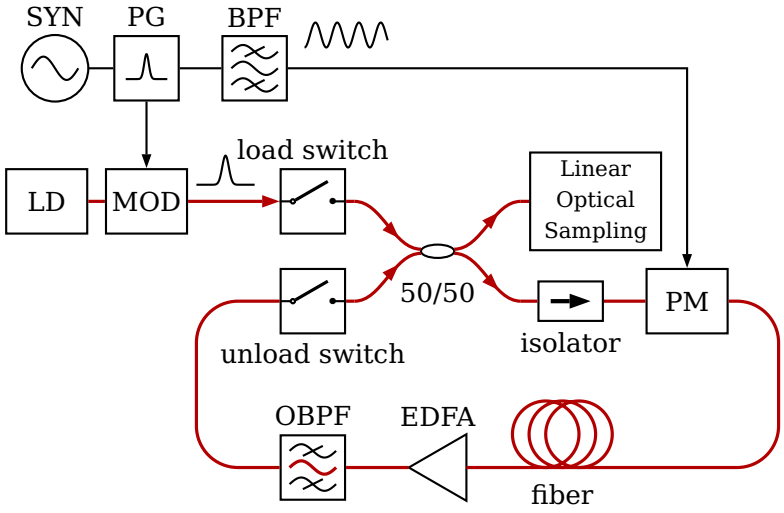
```
\begin{pspicture}(3,1.7)
\pnodes(-1,0){A}(1,1){B}(3,0){C}
\psframe*(0,0)(3,1.7)
\begin{optexp}
  \optplane(0,0.7)
  \optprism[prismalign=center, linecolor=white](A)(B)(C)
  \optplane(C)
  \addtopsstyle{Beam}{linecolor=white}
  \drawbeam[ArrowInside=->]{1-2}
  \definecolor[ps]{bl}{rgb}{%
    tx@addDict begin Red Green Blue end}%
  \addtopsstyle{Beam}{linestyle=fade, beaminsidelast, fadeto=black}
  \multido{\i=0+1}{60}{%
    \drawbeam[n=1.5 \i\space 0.003 mul add, linecolor=white, beampathskip=1,
        fadefuncname=gauss]{1-2}
    \pstVerb{\i\space 650 400 sub 59 div mul 400 add
      tx@addDict begin wavelengthToRGB end }%
    \drawbeam[n=1.5 \i\space 0.003 mul add, linecolor=bl, beampathskip=2,
        linewidth=0.3\pslinewidth]{-}%
  }%
\end{optexp}
\end{pspicture}
```

Ex. 11.31

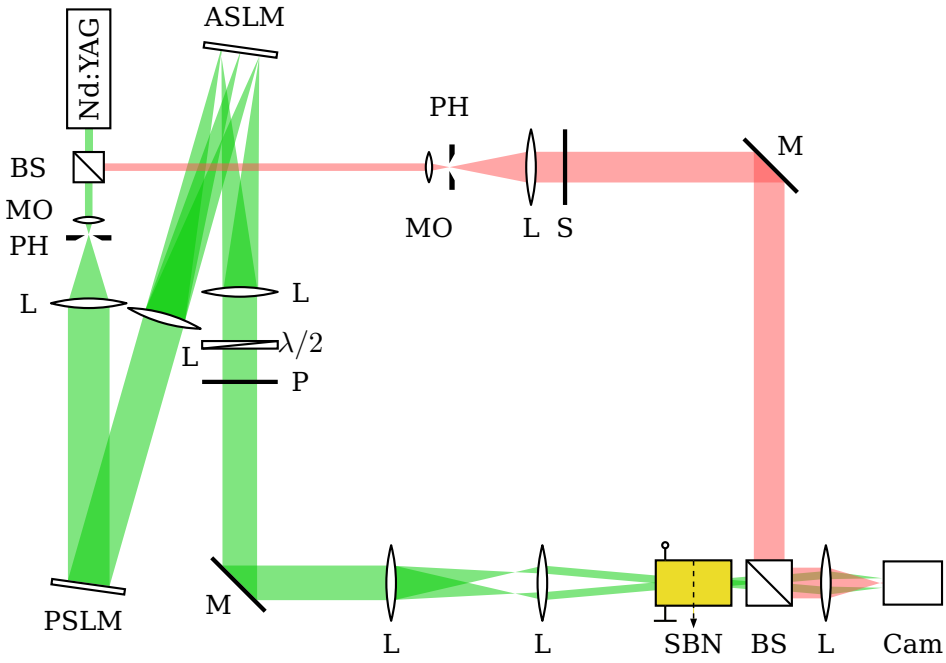



```
\begin{pspicture}(10,6)
  \begin{optexp}
    \psset[optexp]{usefiberstyle, couplersize=0.25, couplersep=0.07}
    \newpsstyle{Fiber}{linecolor=orange, linewidth=2\pslinewidth}
    \pnodes(2,1){PumpDiode}(2,5){UpperPumpDiode}(1,3){SignalIsolatorIn}(4, ↩
        3){SignalIsolatorOut}(1,3){SignalIn}(10,3){SignalCombinerOut}
    \pnodes(7,3){FiberIn}(9,3){FiberOut}(9,3){AmpOut}
    \optdiode[compname=PumpDiode, ↩
        position=start](PumpDiode)([Xnodesep=1]PumpDiode){Pump diode}
    \optdiode[compname=UpperPumpDiode, ↩
        position=start](UpperPumpDiode)([Xnodesep=1]UpperPumpDiode){Pump diode}
    \optisolator[compname=SignalIsolator, ↩
        fiber=none](SignalIsolatorIn)(SignalIsolatorOut)%
        {\begin{tabular}{@{}c@{}}Signal\\Isolator\end{tabular}}
    \wdmcoupler[compname=SignalPumpCombiner](UpperPumpDiode)(\oenodeOut{SignalIsolator})(PumpDi
        {\begin{tabular}{@{}c@{}}Signal/Pump\\light combiner\end{tabular}}
    \optfiber[
        compname=ActiveFiber,
        position=start,
        addtoFiberOut={arrows=->}](FiberIn)(FiberOut)%
        {\begin{tabular}{@{}c@{}}Active\\fiber\end{tabular}}
    \drawfiber[ArrowInside=->](SignalIn){SignalIsolator}
  \end{optexp}
  \nput{75}{AmpOut}{\begin{tabular}{@{}c@{}}Amplifier\\Output\end{tabular}}
  \nput{-90}{SignalIn}{\begin{tabular}{@{}c@{}}Signal\\Input\end{tabular}}
\end{pspicture}
```

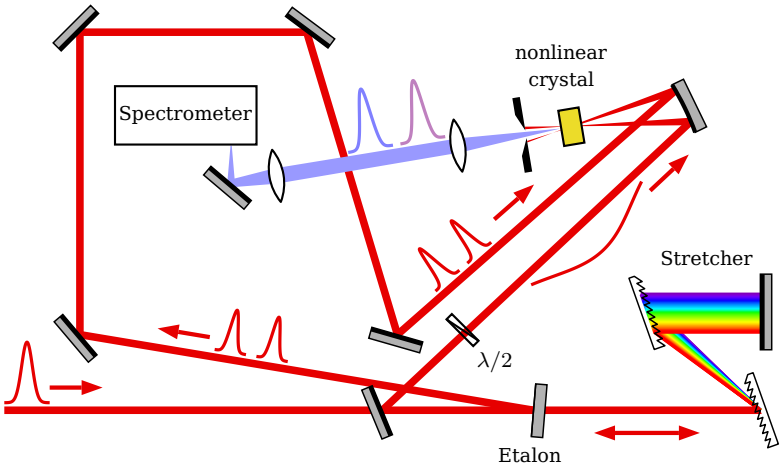
Ex. 11.32



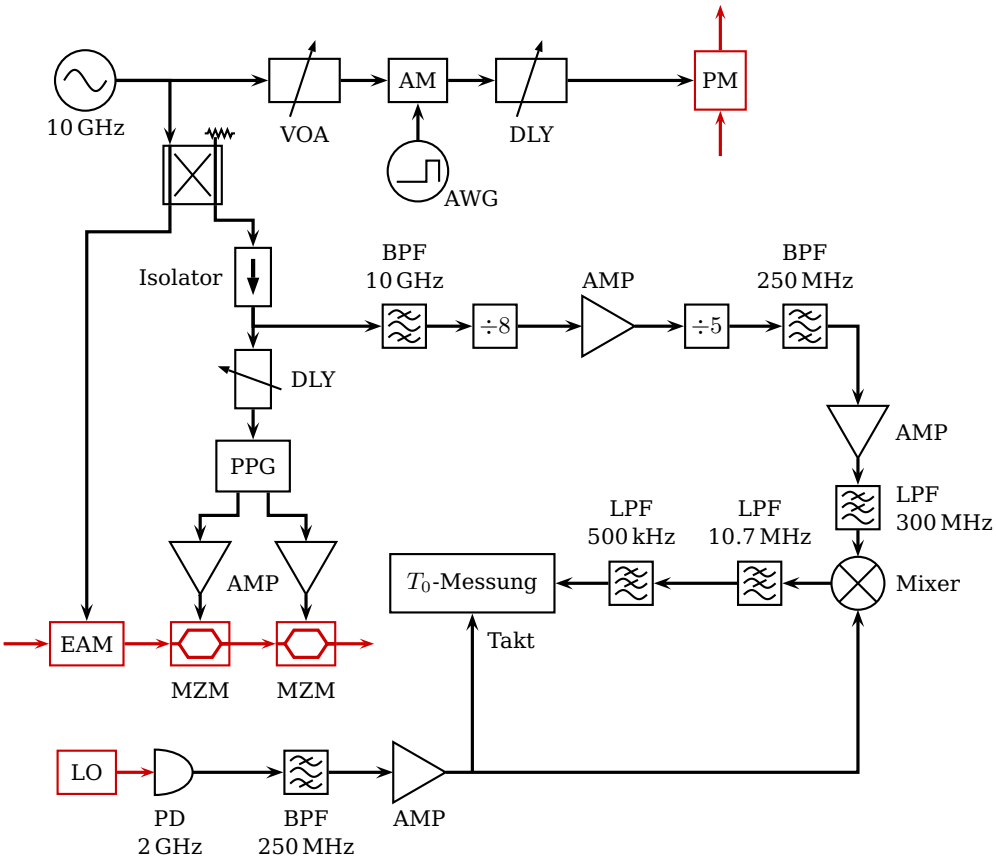
Ex. 11.33: Adapted from DOI:10.1364/OL.37.000797



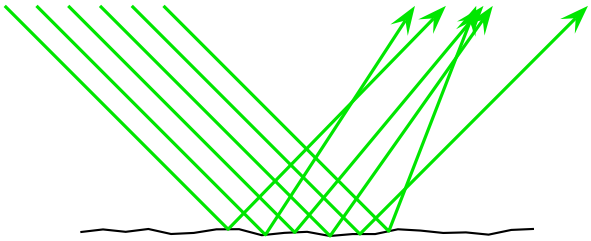
Ex. 11.34: Adapted from DOI:10.1364/AOP.1.000308, Fig. 37



Ex. 11.35



Ex. 11.36



```
\begin{pspicture}(10,3)
\makeatletter
\newOptexpTripole{roughsurface}
\def\roughsurface@nodes{%
  \pssavepath[linestyle=none, arrows=-,ArrowInside=-]{\oenode@Path{A}}{%
    \code{0 srand}
    \moveto(!-3 Rand 0.5 sub 0.1 mul)
    \multido{\r=-2.7+0.3}{20}{%
      \lineto(!\r\space Rand 0.5 sub 0.1 mul)}%
  }%
  \newOptexpComp{%
    {0 0} tx@IntersectDict /\PIT@name{\oenode@Path{A}} get 0 0 refl {PathIfc}
    1 }%
}%
\def\roughsurface@comp{%
  \pstracecurve{\oenode@Path{A}}
}%
\makeatother
\optplane[angle=90](1,3)
\roughsurface(0,3)(5,0)(10,3)
\addtopsstyle{Beam}{beamalign=abs, beamangle=-45, arrows=->, arrowscale=2}
\multido{\r=0+0.3}{6}{%
\drawbeam[beampos=\r]{1}{2}{1}}
\end{pspicture}
```

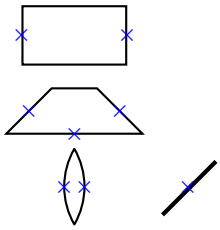
12. Additional information

12.1. Internal component structure

showifcnodes=true, false default: false

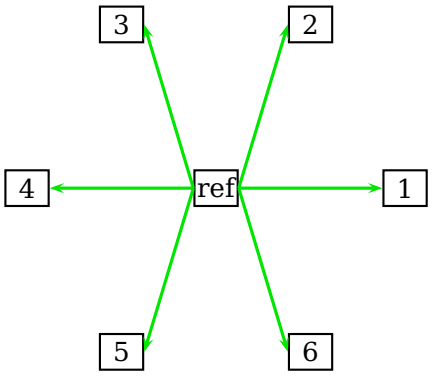
IfcNodeStyle⟨*psstyle*⟩ default: dotstyle=x, dotscale=1.5, linecolor=blue

Every component consists internally of interfaces which are defined by a node on the optical axis, a plane vector or a curvature radius, and by its optical characteristics (reflective or transmittive, or both e.g. for \beamsplitter). The interface nodes can be visualized for each component with the showifcnodes parameter.



```
\begin{pspicture}(4,3)
  \psset{showifcnodes}
  \optbox(0,2.5)(4,2.5)
  \doveprism(0,1.5)(4,1.5)
  \lens(0,0.5)(4,0.5)
  \mirror(0,0.5)(3.5,0.5)(3.5,3)
\end{pspicture}
```

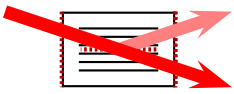
\draw*beam calculates the distance between the interface nodes of two components and connects the two nearest to each other.



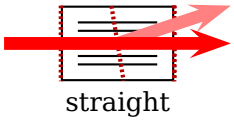

```
\begin{pspicture}(-3,-3)(3,3)
  \psset{optboxsize=0.6 0.5, labeloffset=0}
  \multido{\i=0+60,\ii=1+1}{6}{%
    \pnode(2.5;\i){A}
    \optbox([Xnodesep=-1]A)([Xnodesep=1]A){\ii}
  }
  \optbox[compname=ref](-1,0)(1,0){ref}
  \addtopsstyle{Beam}{arrows=->}
  \multido{\i=1+1}{6}{\drawbeam{ref}{\i}}
\end{pspicture}
```

showinterfaces=true, false default: false

Visualizes all interfaces of a component.




```
\begin{pspicture}(3,1.5)
  \psset[optexp]{showinterfaces, aomgratingcount=6}
  \addtopsstyle{IfcStyle}{linewidth=2\pslinewidth}
  \pnodes(0,1.5){A}(3,0.5){B}(3,1.5){C}
  \optaom[aomalign=symmetric](A)(B)(C){symmetric}
  \addtopsstyle{Beam}{arrows=->, linewidth=4\pslinewidth}
  \drawbeam[linecolor=red!50](A){}(C)
  \drawbeam[linecolor=red](A){}(B)
\end{pspicture}
```




```
\begin{pspicture}(3,1.5)
  \psset[optexp]{showinterfaces, aomgratingcount=6}
  \addtopsstyle{IfcStyle}{linewidth=2\pslinewidth}
  \pnodes(0,1){A}(3,1){B}(3,1.5){C}
  \optaom[aomalign=straight](A)(B)(C){straight}
  \addtopsstyle{Beam}{arrows=->, linewidth=4\pslinewidth}
  \drawbeam[linecolor=red!50](A){}(C)
  \drawbeam[linecolor=red](A){}(B)
\end{pspicture}
```

IfcStyle⟨*psstyle*⟩ default: linestyle=dashed, dash=1pt 1pt, linecolor=red!70!black

Determines the appearance of the interfaces, which are shown with showinterfaces.

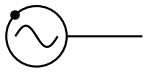
12.2. Overview of special nodes

In this section you find overviews of all special nodes which each component provides. The possible positions for external and rotation reference nodes, and the interface nodes.

12.2.1. External and rotation reference nodes

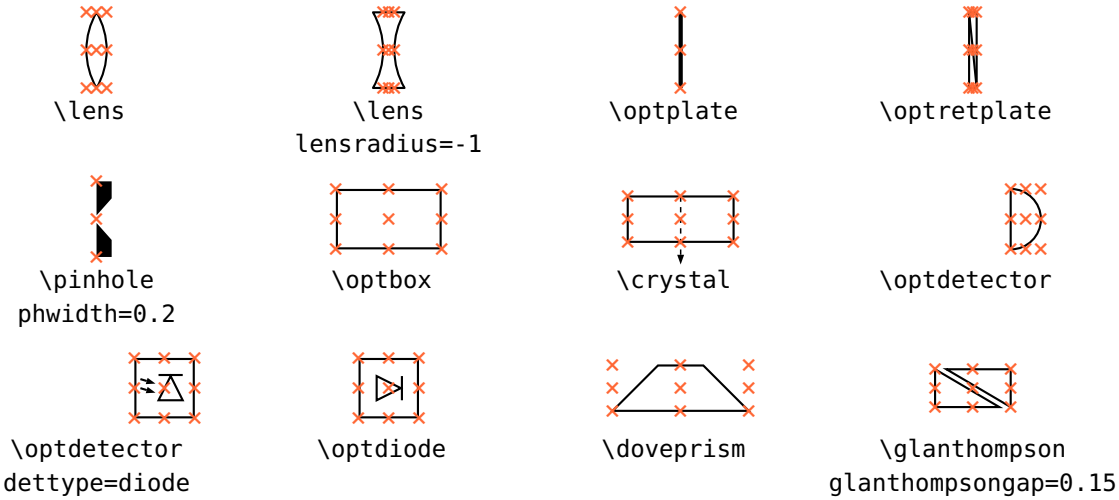
Here, you find all possible external (Sec. 8.5) and rotation reference nodes (Sec. 8.7) for all components. Displayed are all possible combinations of `t`, `c`, or `b` with `l`, `c`, or `r`. Not all components support all combinations, e.g. a plain `\mirror` ignores `t` and `b` and an `\optplate` ignores `l` and `r`.

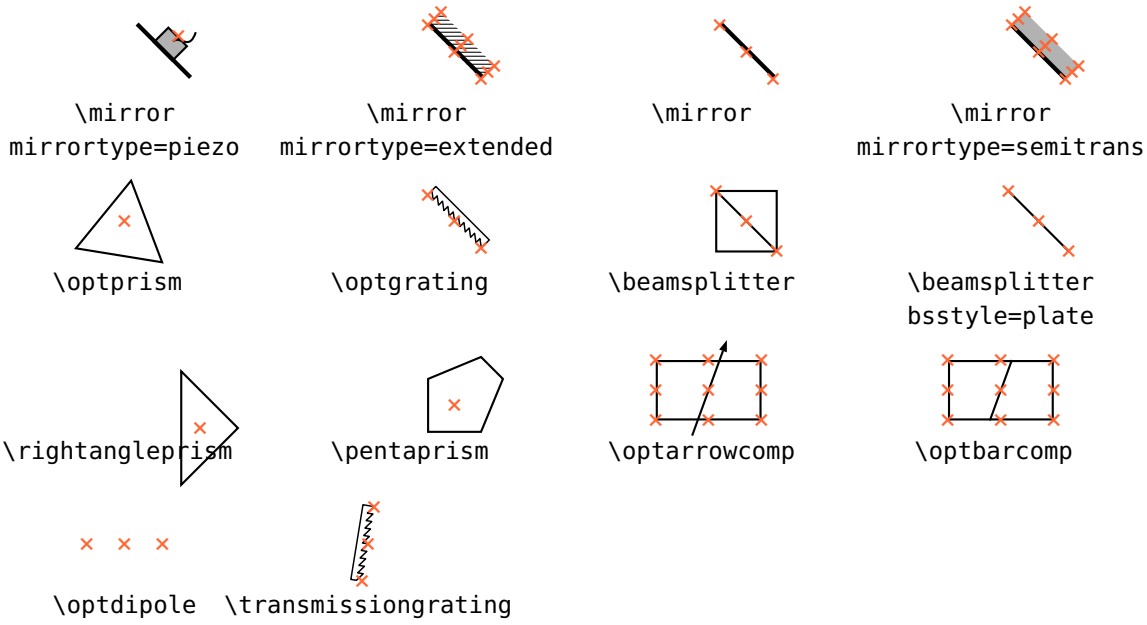
Instead of these values also fractional parts can be used: `t` corresponds to $+1 \cdot \Delta y$, `b` corresponds to $-1 \cdot \Delta y$, `l` corresponds to $-1 \cdot \Delta x$, and `r` corresponds to $+1 \cdot \Delta x$. This can be especially useful for round outer shapes, as the following examples shows.



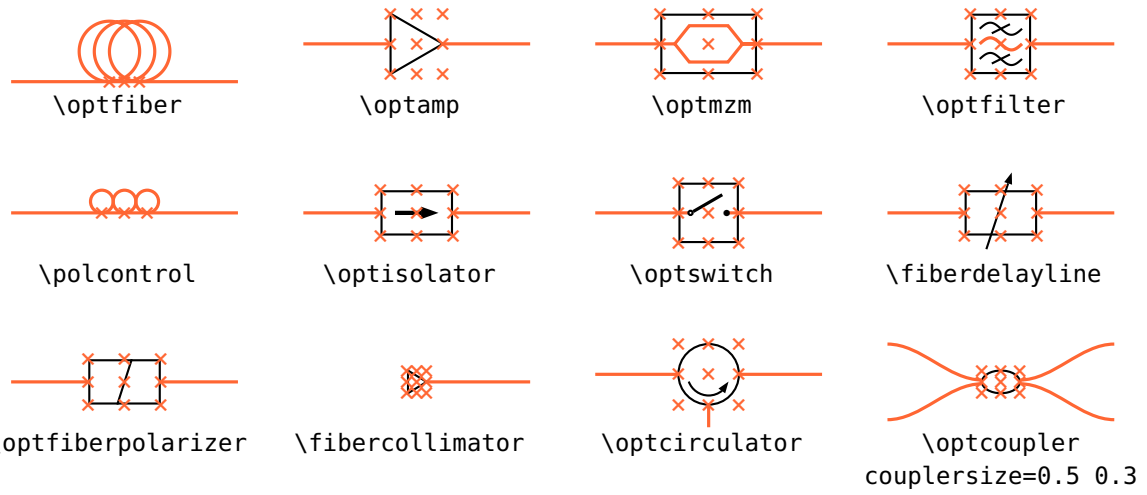
```
\begin{pspicture}(2,1)
  \elecsynthesizer[extnode={-0.707,0.707}](1,0.5)(2,0.5)
  \psdot(\oenodeExt{})
\end{pspicture}
```

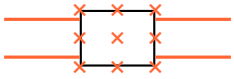
Free-ray components





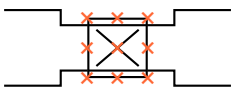
Fiber components





\fiberbox

Electrical components



\eleccoupler

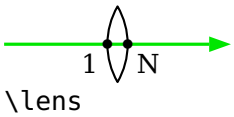
\elecmixer

\elecsynthesizer

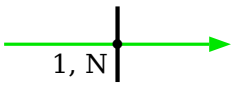
12.2.2. Interface nodes

Here, all available components and their interface nodes are listed. If a node is labeled as “1, N”, it means that both nodes are equal.

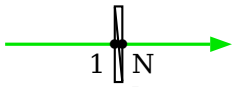
Free-ray components



\lens

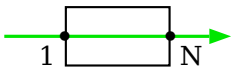


\optplate

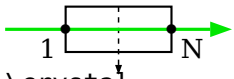


\optretplate

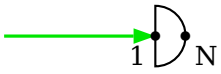
\pinhole



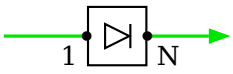
\optbox



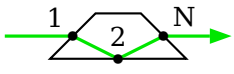
\crystal



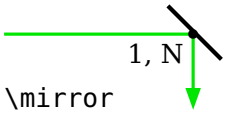
\optdetector



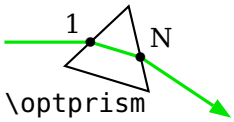
\optdiode



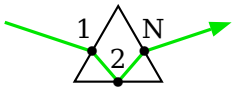
\doveprism



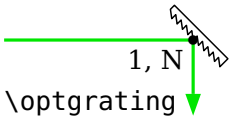
\mirror



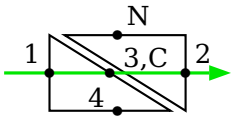
\optprism



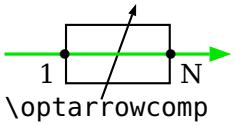
\optprism
prismtype=reflective



\optgrating



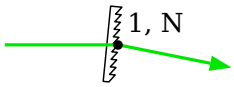
\glanthompson



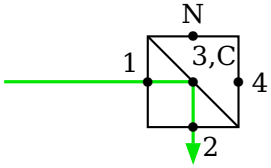
\optarrowcomp

\optbarcomp

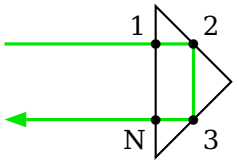
\optdipole



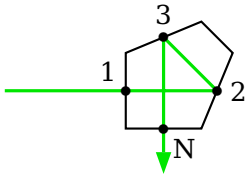
\transmissiongrating



\beamsplitter

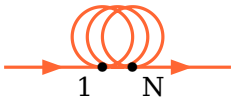


\rightangleprism

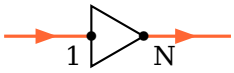


\pentaprism

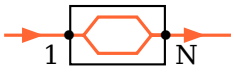
Fiber components



\optfiber



\optamp



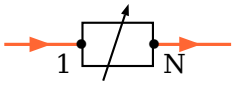
\optmzm

\optfilter

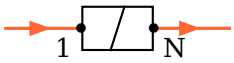
\polcontrol

\optisolator

\optswitch

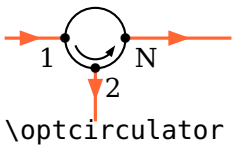


\fiberdelayline

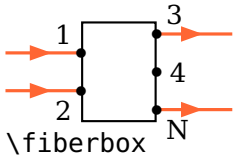


\optfiberpolarizer

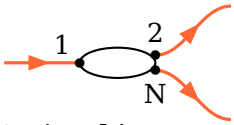
\fibercollimator



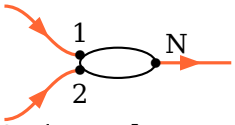
\optcirculator



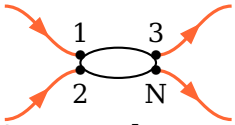
\fiberbox



\wdmsplitter

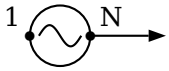


\wdmcoupler

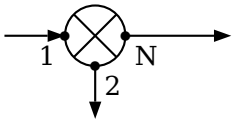


\optcoupler

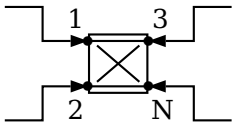
Electrical components



\elecsynthesizer



\elecmixer



\eleccoupler

12.3. Backward compatibility

In this section you find a comprehensive listing of the behaviour which changed in a version compared to the previous one and which is not backward compatible. Here you find also information on what you must note when upgrading and how you must change old documents to work correctly with the newer version.

12.3.1. Version 4.3a

`prismtype` The option `prismtype` was still not correctly implemented, die automatic alignment was somewhat arbitrary. This should be finally fixed, an example is shown for option `prismalign`.

`\optprism` The positioning of the label was not consistent with other components, the reference point was not the center point and did also depend on the component rotation. This is fixed now, but could mean little corrections for existing images.

12.3.2. Version 4.3

`\rightangleprism` The alignment of the `\rightangleprism` and the positioning of the label was wrong. This became apparent only if the input and output vectors had very different lenghts.

12.3.3. Version 4.2

prismtype

The option prismtype, introduced in version 4.1, always needed an additional angle rotation of 180°. This is not necessary anymore, the component behavior was corrected, but this may require adjustment of drawings created with version 4.1.

12.3.4. Version 4.0

The option conn and the old, buggy code for beam connections was finally removed. To update older documents, see Sec. 12.3.6. With this change also option connjoin became useless.

The appearance of \optfiber changed slightly, because the old implementation was buggy and the separation between the loops depended on the number of loops (see example below). Note, that to achieve a similar default look, the default value for fiberloopsep was changed from 0.3 to 0.2. Therefore, the interface nodes are a bit closer together.

The macro \newOptexpDipoleNolabel was removed and can be replaced by \newOptexpDipole without any further changes.

The old syntax of defining own components with \mycomp@iii is not supported anymore.

12.3.5. Version 3.3

\opttripole

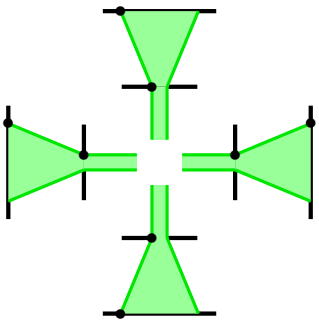
Changed the reference angle for the label of the \opttripole to be consistent with the other tripoles.

Beam nodes

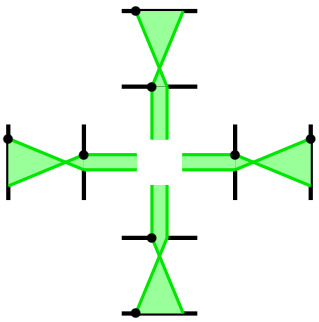
Before version 3.3 the beam nodes \oenodeBeamUp and \oenodeBeamLow were not swapped correctly and the orientation of “upper” and “lower” was only relative to the beam direction (equivalent to extnodealign=relative).

This could lead to problems when using loadbeampoints with beamdiv. In that case the sign of beamdiv was not unambiguously associated with contracting or expanding beam.

As of version 3.3 the beam nodes are arranged according to extnodealign=absolute and the association of negative beamdiv with a contracting and positive beamdiv with an expanding beam is always valid. In the following two examples all \oenodeBeamUp are marked.



```
\begin{pspicture}(-2,-2)(2,2)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40}
  \multido{\i=0+90,\ia=1+2,\ib=2+2}{4}{%
    \optplate[position=end](0.3;\i)(1;\i)
    \optplate[position=end, plateheight=1.5](1;\i)(2;\i)
    \drawwidebeam[beamwidth=0.2](0.3;\i){\ia}
    \pnode(\oenodeBeamUp{}){Up}
    \drawwidebeam[loadbeampoints, beamdiv=45]{\ia-\ib}
    \psdot(Up)\psdot(\oenodeBeamUp{})
  }
\end{pspicture}
```



```
\begin{pspicture}(-2,-2)(2,2)
  \addtopsstyle{Beam}{fillstyle=solid, fillcolor=green!40}
  \multido{\i=0+90,\ia=1+2,\ib=2+2}{4}{%
    \optplate[position=end](0.3;\i)(1;\i)
    \optplate[position=end](1;\i)(2;\i)
    \drawwidebeam[beamwidth=0.2](0.3;\i){\ia}
    \pnode(\oenodeBeamUp{}){Up}
    \drawwidebeam[loadbeampoints, beamdiv=-45]{\ia-\ib}
    \psdot(Up)\psdot(\oenodeBeamUp{})
  }
\end{pspicture}
```

12.3.6. Version 3.0

The changes in version 3.0 compared to version 2.1 which are not backward compatible.

General parameters

`namingscheme`=old, new default: `new`

In version 2.1 special component nodes had to be accessed by their explicit name. If you need the old naming scheme, because you accessed internal nodes directly, then you must set `namingscheme=old`. Since version 3.0 explicit macros are provided to access all special component nodes, so that the actual naming scheme does not matter, see Sec. 8.

`endbox` `angle` `rotateref` These options now affect all components.

`extnode` Some components now provide more possible positions for `extnode`. `\optdetector` had only one possible external node, which was accessible for any value of `extnode`, now this node can be accessed only with `extnode=r`. All other values give different node positions.

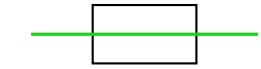
`\optbox` An `\optbox` does always have the input and output nodes at the respective interfaces, whereas in version 2.1 the input and output nodes where at the same position for `endbox=true`.

`\fibercollimator` The `\fibercollimator` has only a fiber connection by default, the beam must be drawn manually or with `beam` option.

Beam connections

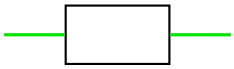
In version 2.1, beam connections could be internal, that is the connections were specified together with the compent (`beam` or `conn`), or external with an additional call of `\drawbeam`. The main reason for internal connections was the aesthetic aspect of drawing the beam behind the component, but also convenience played a role. Drawing the beam behind the component is possible with layering (Sec. 9.9) since 3.0.

Some of the variants of internal beam connections can be used also in 3.0:



`conn=o-i, conn=i-o`

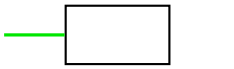
```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox[beam](A)(B)
\end{pspicture}
```



`conn=o-o`

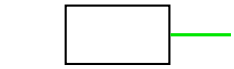
```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox[beam, beaminside=false](A)(B)
\end{pspicture}
```

Other internal connections need an additional `\drawbeam` call, and if you want to move the beam behind the component, the code must be wrapped in a `optexp` environment. Except for this, external connection are equivalent:



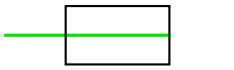
`conn=o-, conn=a-`

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox(A)(B)
  \drawbeam(A){}
\end{pspicture}
```



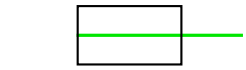
`conn=-o, conn=-b`

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \optbox(A)(B)
  \drawbeam{}(B)
\end{pspicture}
```



`conn=i-, A-`

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox(A)(B)
    \drawbeam[beaminsidelast](A){}
  \end{optexp}
\end{pspicture}
```



conn=-i, conn=-B

```
\begin{pspicture}(3,2)
  \pnode(0,1){A}\pnode(3,1){B}
  \begin{optexp}
    \optbox(A)(B)
    \drawbeam[beaminsidefirst]{}(B)
  \end{optexp}
\end{pspicture}
```

Documentation index

A

abspos, 9, 10, 13–17, 22, **22**, 30, 40, 78, 85, 87, 94–96, 98, 101–103, 108, 110, 112, 117, 129, 134–138, 140, 142–144, 147–151, 153, 154
addtoBeam, **103**, 104
addtoFiber, 117, **117**, 118
addtoFiberIn, 123, 138, 139
addtoFiberIn1, 117, 123, 138, 139
addtoFiberIn2, 117, 123, 128, 138, 139
addtoFiberOut, 123, 138, 139, 157
addtoFiberOut1, 123, 124, 139
addtoFiberOut2, 123, 139
addtoOptComp, 25, 26, **26**
\addtopsstyle, 24, 123
addtoWire, 119, **119**
addtoWireIn, 123
addtoWireIn1, 123
addtoWireIn2, 123
addtoWireOut, 123
addtoWireOut1, 123
addtoWireOut2, 123
allowbeaminside, 38, 74, 100, **100**, 148
angle, 23, **23**, 40, 44, 50, 70, 77, 79, 83, 84, 86–88, 97, 98, 103, 113–116, 134, 135, 140, 144–146, 149, 155, 160
aomalign, 51, **51**, 162
aomcomp, 51, **52**
aomgratingcount, **51**, 162
aomheight, **50**, 51
aomreflalign, **51**, 52, 53
aomsize, **51**, 53
aomwidth, **50**, 51
arrowcompangle, **33**

arrowcompheight, 33, **33**
arrowcompshape, 33, **33**
arrowcompsize, **33**, 138
ArrowCompStyle, 34, **34**, 61
arrowcompwidth, 33, **33**
ArrowInside, 104
ArrowInsideMaxLength, 104, **104**
ArrowInsideMinLength, 104, **104**, 145, 152
aspherecoefficients, **29**, 30
asphereconstant, **29**, 30
asphereheight, **29**, 30
aspherereadiusleft, **29**, 30
aspherereadiusright, **29**, 30
aspherewidth, **29**, 30
\asphericlens, **29**, 30
\axicon, 41, **41**
axiconangle, **42**
axiconheight, **41**
axiconwidth, **42**

B

\backlayer, 88, 125, 154
barcompangle, 34, **34**, 35
barcompheight, 34, **34**
barcompshape, **34**, 35
barcompsize, **34**
BarCompStyle, 35, **35**
barcompwidth, 34, **34**
Beam, 14, 15, 17, 40, 46, 50–53, 74, 85, 88, 93–95, 97, 101–103, **103**, 104, 106, 107, 110, 111, 129, 134–137, 140–142, 144–148, 152, 154–156, 160, 162, 169
beam, 21, 25–28, 31, 32, 36–45, 47–50, 54–56, 74, 75, 77, **103**, 126, 128, 170, 171
beamalign, 35, 46, 86, 87, 95–97, **97**, 133, 134, 147, 151, 160
beamangle, 11, 35, 50, 92, 93, 95, 96, **96**, 97, 108, 110, 111, 144, 145, 155, 160
beamdiffractionorder, 52, 53, **53**
beamdiv, 35, 97, 106, **106**, 108, 110, 129, 134, 136, 144, 145, 169
beaminside, 11, 99, **99**, 100, 102, 127, 131–133, 144, 171
beaminsidefirst, 85, 96, 99, **99**, 100, 101, 111, 137, 140, 142, 148, 172
beaminsidelast, 84, 86, 87, **99**, 100, 101, 108, 137, 154, 171
beammode, 53, **101**

beamnodealign, 88, **101**, 102, 154
beampathcount, 98, **98**, 154
beampathskip, 88, **97**, 98, 154–156
beampos, 10, 11, 30, 40, 44, 46, 47, 86, 92, 94, 96, **96**, 99, 100, 102, 103, 106, 108, 112, 147, 151, 155, 160
\beamsplitter, 17, 20, 47, **47**, 50, 90, 101, 111, 137, 140–142, 144, 151, 152, 161
beamwidth, 11, 14, 15, 17, 35, 41, 46, 49, 51–54, 74, 85, 87, 88, 98, 101, 102, 104, 106, **106**, 107, 110, 111, 127–129, 134, 136, 137, 140, 141, 144, 145, 154, 169
bssize, **47**, 90, 142, 151, 152
bsstyle, 17, 47, **47**, 152

C

caxisinv, **36**
caxislength, 36, **36**, 37
compname, 17, 18, 78, **78**, 89–91, 94, 98, 101, 112, 117, 118, 135, 138, 140, 141, 144, 148–150, 152, 154, 157, 162
compoffset, **23**, 24, 152
components
- \aspheriClens, **29**, 30
- \axicon, 41, **41**
- \beamsplitter, 17, 20, 47, **47**, 50, 90, 101, 111, 137, 140–142, 144, 151, 152, 161
- \crystal, 36, **36**, 37, 128, 142, 148
- \doveprism, 38, **38**, 91, 92, 94, 141, 161
- \eleccoupler, 69, **69**, 70, 122
- \elecmixer, **71**, 72, 115, 122
- \elecsynthesizer, 24, 70, **70**, 71, 120–122, 124, 138, 149, 150, 163
- \fiberbox, 67, **67**, 68
- \fibercollimator, 73, 74, **74**, 75, 170
- \fiberdelayline, 33, 61, **61**, 108, 124
- \glanthompson, 39, **39**, 40, 98, 149
- \lens, **27**
- \mirror, 10, 17, 23–26, 42, **42**, 43–45, 47, 77, 81, 82, 85, 93, 96, 97, 104, 106, 107, 111, 134, 135, 137, 141, 142, 145–148, 151, 152, 154, 155, 161, 163
- \oapmirror, 46, **46**, 47
- \optamp, 58, **58**, 117, 118, 143, 149, 150, 153
- \optaom, 50, **50**, 51–53, 162
- \optarrowcomp, 33, **33**, 34, 138

– \optbarcomp, 34, **34**, 35
– \optbox, 10–17, 19–23, 32, **32**, 68, 78–82, 84, 86–88, 90, 96, 97, 99, 100, 106, 113–118, 120–122, 124, 125, 134–139, 141–144, 146–154, 161, 162, 170–172
– \optcirculator, 61, **61**, 62, 72, 80, 115, 120, 121, 150
– \optcoupler, 63, **63**, 64, 65, 70, 80, 117, 118, 121, 138, 143, 144, 153
– \optdetector, 13, 16, 17, 73, 75, **75**, 76, 80, 88, 120, 135, 138, 140, 146, 148, 151–154, 170
– \optdiode, 38, **38**, 100, 157
– \optdipole, 24, 126, **126**, 127, 141, 151
– \optfiber, 57, **57**, 58, 117, 118, 134, 138, 139, 143, 150, 153, 157, 168
– \optfiberpolarizer, 34, 61, **61**, 139
– \optfilter, 24, 57, 73, **73**, 74, 100, 143
– \optgrating, 26, **47**, 48–50, 95, 110, 111, 135, 144, 145, 155
– \optisolator, 59, **59**, 60, 139, 150, 157
– \optmzm, 15–17, 57, 59, **59**, 120–122, 143, 149, 150
– \optplane, 35, 38, 40, 46, 53, 87, 92, 94, 95, 98, 102, 103, 108, 112, 134, 140, 144, 149, 156, 160
– \optplate, 13, 31, **31**, 95–97, 106, 108, 112, 129, 136, 144, 146, 148, 151, 152, 163, 169
– \optprism, 54, **54**, 88, 91, 92, 95, 98, 101, 104, 108, 154–156
– \optretplate, 13, 31, **31**, 99, 100, 135–137, 141, 142
– \optsource, 35, **35**
– \optswitch, 60, **60**, 143, 153
– \opttripole, 126, **126**, 127, 128, 137, 147, 168
– \optwedge, 41, **41**
– \parabolicmirror, 45, **45**, 46
– \pentaprism, 56, **56**, 83, 99, 100, 141
– \pinhole, 32, **32**, 95, 136, 137, 141, 142, 144, 145, 147, 152
– \polarization, 40, **40**, 87, 98, 136, 142, 149
– \polcontrol, 59, **59**, 139, 143, 150
– \rightangleprism, 55, **55**, 167
– \transmissiongrating, 49, **49**, 50
– \wdmcoupler, 63, **63**, 65, 66, 128, 139, 146, 150, 157
– \wdmsplitter, 16, 17, 63, **63**, 65, 66, 121, 124, 139, 144, 146, 150, 153
compshift, 17, 23, **23**, 24, 79, 88, 134, 138, 139, 144, 150, 155
conn, 168, 170
connjoin, 168
coupleralign, 64, **64**, 65, 66, 80, 118, 124, 128, 139, 150
couplersep, 63, **63**, 64, 70, 80, 157

couplersize, 63, **63**, 64, 70, 80, 157
couplertype, 63, 64, **64**, 70, 80, 143, 146
\crystal, 36, **36**, 37, 128, 142, 148
CrystalCaxis, 37, **37**
crystalheight, 36, **36**
CrystalLamp, **37**
crystalsize, 36, **36**, 142
crystalwidth, 36, **36**

D
DetectorStyle, 76, **76**
detsize, **76**, 88, 146, 148, 152–154
dettype, 13, 17, 76, **76**, 135, 138
diffractionorders, **52**, 53
\doveprism, 38, **38**, 91, 92, 94, 141, 161
doveprismsize, 38, **38**, 92
\drawbeam, 7, 10, 11, 13, 20, 30, 35, 38, 40, 42–44, 46, 47, 50, 83, 84, 86, 87, 89–100, 102–104, 108–112, 127, 130–133, 135, 136, 140, 142, 146–149, 151, 152, 155, 156, 160–162, 170–172
\drawfiber, 12, 16, 17, 65, 112–118, 120, 138, 143, 144, 146, 149, 150, 153, 157
\drawwidebeam, 11, 14, 15, 17, 35, 41, 46, 49, 51–54, 74, 84, 85, 87, 88, 91, 93, 97, 98, 101, 102, 104, 106–111, 127–129, 134, 136, 137, 140, 141, 144, 145, 154, 161, 169
\drawwire, 118–120, 138, 149, 150, 153

E
\eleccoupler, 69, **69**, 70, 122
eleccouplerinput, **69**, 70
eleccouplersep, **69**
eleccouplersize, 69, **69**
eleccouplertype, 69, **69**, 70
\elecmixer, **71**, 72, 115, 122
elecmixersize, **72**
\elecsynthesizer, 24, 70, **70**, 71, 120–122, 124, 138, 149, 150, 163
endbox, **22**, 170
environ, 6
environments
  – optexp, 11, 18, 48, 124, 171
  – pspicture, 78, 85

ExtendedMirror, 24, 25, 43, **44**, 45, 81, 82, 137, 145
extnode, 43, 81, **81**, 82, 83, 88, 130, 132, 133, 138, 140, 144, 149, 150, 153, 163, 170
extnodealign, 81, **81**, 82, 140, 169
extnodes, 82, **82**

F

fade, **104**, 105
fadefunc, 105, **105**
fadefuncname, 105, **105**, 156
fadepoints, 104, 105, **105**
fadeto, 104, 105, **105**, 156
FdlArrow, **61**
fdlsize, **61**, 108
Fiber, 57, 61, 62, 70, 114, 115, 117, **117**, 118, 123, 138, 153, 157
fiber, 15–17, 25, 34, 35, 65, 68, 74, 108, 115, 117, 118, 120, **120**, 121, 124, 134, 138, 143, 144, 146, 149, 150, 153, 157
fiber*, 118
fiberalign, **114**, 115, 119
fiberangleA, 115, **115**, 116, 119
fiberangleB, **115**, 116, 119
\fiberbox, 67, **67**, 68
fiberboxcount, 67, 68, **68**
fiberboxheight, 68, **68**
fiberboxsepin, 67, **68**
fiberboxsepout, 67, 68, **68**
fiberboxsize, 68, **68**
fiberboxwidth, 68, **68**
\fibercollimator, 73, 74, **74**, 75, 170
fibercolsize, **75**
\fiberdelayline, 33, 61, **61**, 108, 124
FiberIn, 123, 157
FiberIn1, 123
FiberIn2, 123
fiberloopradius, 58, **58**, 134
fiberloops, **57**, 58, 139, 143, 153
fiberloopsep, 58, **58**, 168
FiberOut, 123, 157
FiberOut1, 61, 123

FiberOut2, 61, 123
fiberpolsize, **61**
fiberstyle, 12, 70, **117**, 118, 119, 143
fillstyle, 11
filterangle, 74, **74**
filtersize, **73**
FilterStyle, 74, **74**
filtertype, 73, **73**, 74
forcebeaminside, 100, **100**
\frontlayer, 125

G
\glanthompson, 39, **39**, 40, 98, 149
glanthompsongap, 39, **39**, 40, 149
glanthompsonheight, 39, **39**
glanthompsonsize, **39**, 40, 149
glanthompsonwidth, 39, **39**
gratingalign, **48**, 49, 155
gratingcount, 48, **48**
gratingdepth, 48, **48**
gratingheight, **48**
gratinglinewidth, **49**
gratingtype, 48, **48**
gratingwidth, **48**, 145

I
IfcNodeStyle, **161**
IfcStyle, 162, **162**
innercompalign, 24, **24**, 74
innerheight, **32**, 95, 144
innerlabel, **21**, 23, 81, 82, 118, 142, 151
IsolatorArrow, 60, **60**
isolatorsize, **59**

L
label, 15–17, 19, 21, **21**, 36, 37, 41, 57, 58, 68, 81, 82, 117, 126–128, 130–139, 141–144, 146–153
labelalign, 19, **19**, 21, 131–133, 137
labelangle, 10, **19**, 20, 21, 24, 131, 134, 136, 137, 142, 143, 151

labeloffset, 10, 13–15, 17, 19, **19**, 20, 21, 42, 49, 64, 80, 90, 126, 135, 137–139, 142, 144, 147, 151, 154, 162
labelref, 10, 20, **20**, 21, 81
labelstyle, 15–17, **19**, 142, 143, 153
lamp, **37**
\lens, **27**
lens, 9–11, 13–15, 17, 22–28, **28**, 30, 35, 54, 85, 87, 94, 96, 98, 101–103, 108, 110, 112, 129, 134, 136, 137, 140–142, 144, 147, 148, 152, 154, 161
lensheight, 11, 14, 15, 27, **27**, 142
lensradius, 11, 13–15, 28, **28**, 85, 137
lensradiusleft, 27, **27**, 28
lensradiusright, **27**, 28
lenswidth, 28, **28**
linestyle, 11
loadbeam, 85–87, 101, **110**, 111
loadbeampoints, **109**, 110, 129, 134, 136, 137, 141, 155, 169

M

macros
- \addtopsstyle, 24, 123
- \backlayer, 88, 125, 154
- \drawbeam, 7, 10, 11, 13, 20, 30, 35, 38, 40, 42–44, 46, 47, 50, 83, 84, 86, 87, 89–100, 102–104, 108–112, 127, 130–133, 135, 136, 140, 142, 146–149, 151, 152, 155, 156, 160–162, 170–172
- \drawfiber, 12, 16, 17, 65, 112–118, 120, 138, 143, 144, 146, 149, 150, 153, 157
- \drawwidebeam, 11, 14, 15, 17, 35, 41, 46, 49, 51–54, 74, 84, 85, 87, 88, 91, 93, 97, 98, 101, 102, 104, 106–111, 127–129, 134, 136, 137, 140, 141, 144, 145, 154, 161, 169
- \drawwire, 118–120, 138, 149, 150, 153
- \frontlayer, 125
- \nccurve, 12, 113, 118
- \ncput, 21
- \newOptexpComp, 131
- \newOptexpDipole, 129–133, 168
- \newOptexpDipoleNolabel, 168
- \newOptexpElecDipole, 129, 130
- \newOptexpFiberDipole, 129, 130
- \newOptexpTripole, 129, 131, 133, 160

– \newpsobject, 25, 127, 128
– \newpsstyle, 8, 24
– \nodexn, 87
– \oeBeamCenter, 88, 154
– \oeBeamVec, 86, 87, 98, 149
– \oeBeamVecLow, 87
– \oeBeamVecMedian, 88, 102, 154
– \oeBeamVecUp, 87
– \oenode, 52, 53, 82
– \oenodeBeam, 84, 86, 87, 98, 149
– \oenodeBeam*, 85
– \oenodeBeamLow, 87, 169
– \oenodeBeamUp, 87, 169
– \oenodeCenter, 78, 80, 152
– \oenodeExt, 81, 82, 133, 138, 140, 149, 150, 153, 163
– \oenodeIfc, 54, 65, 83, 98, 101, 150, 154
– \oenodeIn, 78, 83, 135, 151, 155
– \oenodeLabel, 148, 152
– \oenodeOut, 78, 83, 135, 148, 151, 154, 157
– \oenodeRefA, 21, 79, 83, 103
– \oenodeRefB, 21, 79, 103
– \oenodeRotref, 84
– \oenodeTrefA, 79
– \oenodeTrefB, 79
– \psbezier, 75
\mirror, 10, 17, 23–26, 42, **42**, 43–45, 47, 77, 81, 82, 85, 93, 96, 97, 104, 106, 107, 111, 134, 135, 137, 141, 142, 145–148, 151, 152, 154, 155, 161, 163
mirrordepth, 17, 43, 44, **44**, 47, 81, 82
mirrorlinewidth, **42**
mirrorradius, 42, **42**, 43, 96, 145, 151
mirrortype, 17, 25, 43, **43**, 44, 45, 47, 81, 82, 85, 96, 97, 104, 106, 111, 137, 142, 145, 146, 148, 151, 155
mirrorwidth, **42**, 85, 104, 107, 142, 145, 147, 148, 152, 155
multido, 6

N

n, 10, 11, 38, 85–88, 91–93, **93**, 94–96, 101–103, 108, 112, 129, 133, 135, 137, 140, 141, 144, 148, 152, 154–156
namingscheme, 170

\nccurve, 12, 113, 118
\ncput, 21
newBeam, **103**
newFiber, **117**, 134
newFiberIn, 123
newFiberIn1, 123
newFiberIn2, 123
newFiberOut, 123
newFiberOut1, 123
newFiberOut2, 123
newOptComp, 25, 26, **26**
\newOptexpComp, 131
\newOptexpDipole, 129–133, 168
\newOptexpDipoleNolabel, 168
\newOptexpElecDipole, 129, 130
\newOptexpFiberDipole, 129, 130
\newOptexpTripole, 129, 131, 133, 160
\newpsobject, 25, 127, 128
\newpsstyle, 8, 24
newWire, **119**
newWireIn, 123
newWireIn1, 123
newWireIn2, 123
newWireOut, 123
newWireOut1, 123
newWireOut2, 123
\nodexn, 87
npos, 21

O

\oapmirror, 46, **46**, 47
oapmirroraperture, 46, **46**, 47
\oeBeamCenter, 88, 154
\oeBeamVec, 86, 87, 98, 149
\oeBeamVecLow, 87
\oeBeamVecMedian, 88, 102, 154
\oeBeamVecUp, 87
\oenode, 52, 53, 82
\oenodeBeam, 84, 86, 87, 98, 149

\oenodeBeam*, 85
\oenodeBeamLow, 87, 169
\oenodeBeamUp, 87, 169
\oenodeCenter, 78, 80, 152
\oenodeExt, 81, 82, 133, 138, 140, 149, 150, 153, 163
\oenodeIfc, 54, 65, 83, 98, 101, 150, 154
\oenodeIn, 78, 83, 135, 151, 155
\oenodeLabel, 148, 152
\oenodeOut, 78, 83, 135, 148, 151, 154, 157
\oenodeRefA, 21, 79, 83, 103
\oenodeRefB, 21, 79, 103
\oenodeRotref, 84
\oenodeTrefA, 79
\oenodeTrefB, 79
\optamp, 58, **58**, 117, 118, 143, 149, 150, 153
optampsize, **58**
\optaom, 50, **50**, 51–53, 162
\optarrowcomp, 33, **33**, 34, 138
\optbarcomp, 34, **34**, 35
\optbox, 10–17, 19–23, 32, **32**, 68, 78–82, 84, 86–88, 90, 96, 97, 99, 100, 106, 113–118, 120–122, 124, 125, 134–139, 141–144, 146–154, 161, 162, 170–172
optboxheight, **32**, 33
optboxsize, 11, 12, 20–22, **33**, 68, 78, 81, 82, 86, 87, 90, 118, 134, 135, 137, 138, 142, 144, 146, 148–153, 162
optboxwidth, 13, 15–17, 20, **32**, 33, 99, 100, 134–136, 139, 141–143, 146, 147, 149, 154
optcircangle, 62, **62**, 150
optcircangleA, **62**
optcircangleB, **62**
OptCircArrow, 62, **62**, 150
optcircsize, **62**
\optcirculator, 61, **61**, 62, 72, 80, 115, 120, 121, 150
OptComp, 25, **25**, 26, 74, 76, 145
\optcoupler, 63, **63**, 64, 65, 70, 80, 117, 118, 121, 138, 143, 144, 153
\optdetector, 13, 16, 17, 73, 75, **75**, 76, 80, 88, 120, 135, 138, 140, 146, 148, 151–154, 170
\optdiode, 38, **38**, 100, 157
optdiodesize, **38**
\optdipole, 24, 126, **126**, 127, 141, 151

optdipolecomp, 127, **127**
optdipolesize, **126**, 127, 141, 151
optexp, 11, 18, 48, 124, 171
\optfiber, 57, **57**, 58, 117, 118, 134, 138, 139, 143, 150, 153, 157, 168
\optfiberpolarizer, 34, 61, **61**, 139
\optfilter, 24, 57, 73, **73**, 74, 100, 143
\optgrating, 26, **47**, 48–50, 95, 110, 111, 135, 144, 145, 155
optional, 25, 26, **26**
OptionalStyle, 25, **25**, 26
\optisolator, 59, **59**, 60, 139, 150, 157
\optmzm, 15–17, 57, 59, **59**, 120–122, 143, 149, 150
optmzmsize, **59**, 149
\optplane, 35, 38, 40, 46, 53, 87, 92, 94, 95, 98, 102, 103, 108, 112, 134, 140, 144, 149, 156, 160
\optplate, 13, 31, **31**, 95–97, 106, 108, 112, 129, 136, 144, 146, 148, 151, 152, 163, 169
\optprism, 54, **54**, 88, 91, 92, 95, 98, 101, 104, 108, 154–156
\optretplate, 13, 31, **31**, 99, 100, 135–137, 141, 142
\optsource, 35, **35**
\optswitch, 60, **60**, 143, 153
\opttripole, 126, **126**, 127, 128, 137, 147, 168
opttripolecomp, **127**, 128
\optwedge, 41, **41**
outerheight, **32**, 142, 145, 152

P

packages
- environ, 6
- multido, 6
- pst-eucl, 6
- pst-intersect, 6
- pst-node, 6, 21, 118
- pst-optexp, 6, 24, 89, 125
- pst-plot, 6
- pstricks, 6
- pstricks-add, 6, 104

\parabolicmirror, 45, **45**, 46
parameters

– abspos, 9, 10, 13–17, 22, **22**, 30, 40, 78, 85, 87, 94–96, 98, 101–103, 108, 110, 112, 117, 129, 134–138, 140, 142–144, 147–151, 153, 154
– addtoBeam, **103**, 104
– addtoFiber, 117, **117**, 118
– addtoFiberIn, 123, 138, 139
– addtoFiberIn1, 117, 123, 138, 139
– addtoFiberIn2, 117, 123, 128, 138, 139
– addtoFiberOut, 123, 138, 139, 157
– addtoFiberOut1, 123, 124, 139
– addtoFiberOut2, 123, 139
– addtoOptComp, 25, 26, **26**
– addtoWire, 119, **119**
– addtoWireIn, 123
– addtoWireIn1, 123
– addtoWireIn2, 123
– addtoWireOut, 123
– addtoWireOut1, 123
– addtoWireOut2, 123
– allowbeaminside, 38, 74, 100, **100**, 148
– angle, 23, **23**, 40, 44, 50, 70, 77, 79, 83, 84, 86–88, 97, 98, 103, 113–116, 134, 135, 140, 144–146, 149, 155, 160
– aomalign, 51, **51**, 162
– aomcomp, 51, **52**
– aomgratingcount, **51**, 162
– aomheight, **50**, 51
– aomreflalign, **51**, 52, 53
– aomsize, **51**, 53
– aomwidth, **50**, 51
– arrowcompangle, **33**
– arrowcompheight, 33, **33**
– arrowcompshape, 33, **33**
– arrowcompsize, **33**, 138
– arrowcompwidth, 33, **33**
– ArrowInside, 104
– ArrowInsideMaxLength, 104, **104**
– ArrowInsideMinLength, 104, **104**, 145, 152
– asphere coefficients, **29**, 30
– asphereconstant, **29**, 30
– aspherеheight, **29**, 30

– asphereradiusleft, **29**, 30
– asphereradiusright, **29**, 30
– aspherewidth, **29**, 30
– axiconangle, **42**
– axiconheight, **41**
– axiconwidth, **42**
– barcompangle, 34, **34**, 35
– barcompheight, 34, **34**
– barcompshape, **34**, 35
– barcompsize, **34**
– barcompwidth, 34, **34**
– beam, 21, 25–28, 31, 32, 36–45, 47–50, 54–56, 74, 75, 77, **103**, 126, 128, 170, 171
– beamalign, 35, 46, 86, 87, 95–97, **97**, 133, 134, 147, 151, 160
– beamangle, 11, 35, 50, 92, 93, 95, 96, **96**, 97, 108, 110, 111, 144, 145, 155, 160
– beamdiffractionorder, 52, 53, **53**
– beamdiv, 35, 97, 106, **106**, 108, 110, 129, 134, 136, 144, 145, 169
– beaminside, 11, 99, **99**, 100, 102, 127, 131–133, 144, 171
– beaminsidefirst, 85, 96, 99, **99**, 100, 101, 111, 137, 140, 142, 148, 172
– beaminsidelast, 84, 86, 87, **99**, 100, 101, 108, 137, 154, 171
– beammode, 53, **101**
– beamnodealign, 88, **101**, 102, 154
– beampathcount, 98, **98**, 154
– beampathskip, 88, **97**, 98, 154–156
– beampos, 10, 11, 30, 40, 44, 46, 47, 86, 92, 94, 96, **96**, 99, 100, 102, 103, 106, 108, 112, 147, 151, 155, 160
– beamwidth, 11, 14, 15, 17, 35, 41, 46, 49, 51–54, 74, 85, 87, 88, 98, 101, 102, 104, 106, **106**, 107, 110, 111, 127–129, 134, 136, 137, 140, 141, 144, 145, 154, 169
– bssize, **47**, 90, 142, 151, 152
– bsstyle, 17, 47, **47**, 152
– caxisinv, **36**
– caxislength, 36, **36**, 37
– compname, 17, 18, 78, **78**, 89–91, 94, 98, 101, 112, 117, 118, 135, 138, 140, 141, 144, 148–150, 152, 154, 157, 162
– compoffset, **23**, 24, 152
– compshift, 17, 23, **23**, 24, 79, 88, 134, 138, 139, 144, 150, 155
– conn, 168, 170

– connjoin, 168
– coupleralign, 64, **64**, 65, 66, 80, 118, 124, 128, 139, 150
– couplersep, 63, **63**, 64, 70, 80, 157
– couplersize, 63, **63**, 64, 70, 80, 157
– couplertype, 63, 64, **64**, 70, 80, 143, 146
– crystalheight, 36, **36**
– crystalsize, 36, **36**, 142
– crystalwidth, 36, **36**
– detsize, **76**, 88, 146, 148, 152–154
– dettype, 13, 17, 76, **76**, 135, 138
– diffractionorders, **52**, 53
– doveprismsize, 38, **38**, 92
– eleccouplerinput, **69**, 70
– eleccouplersep, **69**
– eleccouplersize, 69, **69**
– eleccouplertype, 69, **69**, 70
– elecmixersize, **72**
– endbox, **22**, 170
– extnode, 43, 81, **81**, 82, 83, 88, 130, 132, 133, 138, 140, 144, 149, 150, 153, 163, 170
– extnodealign, 81, **81**, 82, 140, 169
– extnodes, 82, **82**
– fade, 105
– fadefunc, 105, **105**
– fadefuncname, 105, **105**, 156
– fadepoints, 104, 105, **105**
– fadeto, 104, 105, **105**, 156
– fdlsize, **61**, 108
– fiber, 15–17, 25, 34, 35, 65, 68, 74, 108, 115, 117, 118, 120, **120**, 121, 124, 134, 138, 143, 144, 146, 149, 150, 153, 157
– fiber*, 118
– fiberalign, **114**, 115, 119
– fiberangleA, 115, **115**, 116, 119
– fiberangleB, **115**, 116, 119
– fiberboxcount, 67, 68, **68**
– fiberboxheight, 68, **68**
– fiberboxsepin, 67, **68**
– fiberboxsepout, 67, 68, **68**
– fiberboxsize, 68, **68**

– fiberboxwidth, 68, **68**
– fibercolsize, **75**
– fiberloopradius, 58, **58**, 134
– fiberloops, **57**, 58, 139, 143, 153
– fiberloopsep, 58, **58**, 168
– fiberpolsize, **61**
– fiberstyle, 12, 70, **117**, 118, 119, 143
– fillstyle, 11
– filterangle, 74, **74**
– filtersize, **73**
– filtertype, 73, **73**, 74
– forcebeaminside, 100, **100**
– glanthompsongap, 39, **39**, 40, 149
– glanthompsonheight, 39, **39**
– glanthompsonsize, **39**, 40, 149
– glanthompsonwidth, 39, **39**
– gratingalign, **48**, 49, 155
– gratingcount, 48, **48**
– gratingdepth, 48, **48**
– gratingheight, **48**
– gratinglinewidth, **49**
– gratingtype, 48, **48**
– gratingwidth, **48**, 145
– innercompalign, 24, **24**, 74
– innerheight, **32**, 95, 144
– innerlabel, **21**, 23, 81, 82, 118, 142, 151
– isolatorsize, **59**
– label, 15–17, 19, 21, **21**, 36, 37, 41, 57, 58, 68, 81, 82, 117, 126–128, 130–139, 141–144, 146–153
– labelalign, 19, **19**, 21, 131–133, 137
– labelangle, 10, **19**, 20, 21, 24, 131, 134, 136, 137, 142, 143, 151
– labeloffset, 10, 13–15, 17, 19, **19**, 20, 21, 42, 49, 64, 80, 90, 126, 135, 137–139, 142, 144, 147, 151, 154, 162
– labelref, 10, 20, **20**, 21, 81
– labelstyle, 15–17, **19**, 142, 143, 153
– lamp, **37**
– lens, 9–11, 13–15, 17, 22–28, **28**, 30, 35, 54, 85, 87, 94, 96, 98, 101–103, 108, 110, 112, 129, 134, 136, 137, 140–142, 144, 147, 148, 152, 154, 161
– lensheight, 11, 14, 15, 27, **27**, 142

– lensradius, 11, 13–15, 28, **28**, 85, 137
– lensradiusleft, 27, **27**, 28
– lensradiusright, **27**, 28
– lenswidth, 28, **28**
– linestyle, 11
– loadbeam, 85–87, 101, **110**, 111
– loadbeampoints, **109**, 110, 129, 134, 136, 137, 141, 155, 169
– mirrordepth, 17, 43, 44, **44**, 47, 81, 82
– mirrorlinewidth, **42**
– mirrorradius, 42, **42**, 43, 96, 145, 151
– mirrortype, 17, 25, 43, **43**, 44, 45, 47, 81, 82, 85, 96, 97, 104, 106, 111, 137, 142, 145, 146, 148, 151, 155
– mirrorwidth, **42**, 85, 104, 107, 142, 145, 147, 148, 152, 155
– n, 10, 11, 38, 85–88, 91–93, **93**, 94–96, 101–103, 108, 112, 129, 133, 135, 137, 140, 141, 144, 148, 152, 154–156
– namingscheme, 170
– newBeam, **103**
– newFiber, **117**, 134
– newFiberIn, 123
– newFiberIn1, 123
– newFiberIn2, 123
– newFiberOut, 123
– newFiberOut1, 123
– newFiberOut2, 123
– newOptComp, 25, 26, **26**
– newWire, **119**
– newWireIn, 123
– newWireIn1, 123
– newWireIn2, 123
– newWireOut, 123
– newWireOut1, 123
– newWireOut2, 123
– npos, 21
– oapmirroraperture, 46, **46**, 47
– optampsize, **58**
– optboxheight, **32**, 33
– optboxsize, 11, 12, 20–22, **33**, 68, 78, 81, 82, 86, 87, 90, 118, 134, 135, 137, 138, 142, 144, 146, 148–153, 162
– optboxwidth, 13, 15–17, 20, **32**, 33, 99, 100, 134–136, 139, 141–143, 146, 147,

149, 154
– optcircangle, 62, **62**, 150
– optcircangleA, **62**
– optcircangleB, **62**
– optcircsize, **62**
– optdiodesize, **38**
– optdipolecomp, 127, **127**
– optdipolesize, **126**, 127, 141, 151
– optional, 25, 26, **26**
– optmzmsize, **59**, 149
– opttripolecomp, **127**, 128
– outerheight, **32**, 142, 145, 152
– parmirrorheight, **45**, 46
– parmirrorwidth, **45**, 46
– pentaprismsize, **56**, 83
– phlinewidth, 32, **32**
– phwidth, 32, **32**, 95, 136, 137, 144, 145
– plateheight, 31, **31**, 95–97, 106, 108, 112, 129, 142, 144, 151, 169
– platelinewidth, **31**, 136
– platesize, **31**
– platewidth, 31, **31**
– polcontrolsize, **59**
– polcontroltype, 59, **59**
– polsize, **40**, 136, 149
– poltype, 40, **40**, 98, 136, 149
– position, 10–17, 20, 21, **21**, 22, 35, 36, 62, 65, 70, 75, 76, 84, 86, 90, 95–97, 99, 100, 106, 108, 112, 114, 118, 120–122, 124, 130–139, 141–147, 149–154, 157, 169
– prismalign, **54**, 95, 156, 167
– prismangle, 54, **54**, 95, 98, 101, 154, 155
– prismsize, **54**, 92
– prismtype, 54, **54**, 98, 101, 154, 167, 168
– pswarning, 53, **108**
– raprismalign, **55**
– raprismsize, **55**
– raytrace, 38, **91**, 92, 103, 104, 134
– refractiveindex, **95**
– reverse, **49**, 50, 95
– rotateref, 23, **23**, 83, 84, 88, 97, 130, 132, 135

– savebeam, 86, 88, **110**, 111, 112, 154
– savebeampoints, 84, **109**, 110, 134, 137, 145
– showifcnodes, 67, 97, 127, 161, **161**
– showinterfaces, 162, **162**
– showoptdots, 55, **77**
– sourceheight, 36, **36**
– sourcesize, **36**
– sourcewidth, **35**, 36
– startinside, 96, 101, **111**, 112, 134
– startinsidecount, 98, **100**, 101, 154
– startnode, 116, **116**, 117, 143, 146
– stopinside, 86, 87, 101, 110, 112, **112**, 129, 134, 136
– stopinsidecount, 46, 101, **101**
– stopnode, 116, **116**, 118, 143, 146
– switchsize, **60**
– switchstyle, 60, **60**
– synthshape, 71, **71**, 150
– synthsize, **70**
– synthtype, **70**, 71, 150
– thicklens, **28**
– usefiberstyle, 57, **57**
– useNA, 93, **93**, 127, 148
– usewirestyle, 57, **57**
– variable, 17, 44, **44**, 64, 145, 151
– voltage, **37**
– wedgeangleleft, **41**
– wedgeangleright, **41**
– wedgeangles, 41, **41**
– wedgeheight, **41**
– wedgewidth, 41, **41**
– wire, 33, **121**, 122, 124, 138, 149, 153
– wire*, 118
– wirealign, **119**
– wireangleA, **119**, 150
– wireangleB, **119**, 138, 149, 150
– wirestyle, 70, **119**
parmirrorheight, **45**, 46
parmirrorwidth, **45**, 46
\pentaprism, 56, **56**, 83, 99, 100, 141

pentaprismsize, **56**, 83
phlinewidth, 32, **32**
phwidth, 32, **32**, 95, 136, 137, 144, 145
PiezoMirror, 43, 45, **45**
\pinhole, 32, **32**, 95, 136, 137, 141, 142, 144, 145, 147, 152
plateheight, 31, **31**, 95–97, 106, 108, 112, 129, 142, 144, 151, 169
platelinewidth, **31**, 136
platesize, **31**
platewidth, 31, **31**
Polarization, **40**, 149
\polarization, 40, **40**, 87, 98, 136, 142, 149
\polcontrol, 59, **59**, 139, 143, 150
polcontrolsize, **59**
polcontroltype, 59, **59**
polsize, **40**, 136, 149
poltype, 40, **40**, 98, 136, 149
position, 10–17, 20, 21, **21**, 22, 35, 36, 62, 65, 70, 75, 76, 84, 86, 90, 95–97, 99, 100, 106, 108, 112, 114, 118, 120–122, 124, 130–139, 141–147, 149–154, 157, 169
prismalign, **54**, 95, 156, 167
prismangle, 54, **54**, 95, 98, 101, 154, 155
prismsize, **54**, 92
prismtype, 54, **54**, 98, 101, 154, 167, 168
\psbezier, 75
pspicture, 78, 85
pst-eucl, 6
pst-intersect, 6
pst-node, 6, 21, 118
pst-optexp, 6, 24, 89, 125
pst-plot, 6
pstricks, 6
pstricks-add, 6, 104
pswarning, 53, **108**

R

raprismalign, **55**
raprismsize, **55**
raytrace, 38, **91**, 92, 103, 104, 134
refractiveindex, **95**

reverse, **49**, 50, 95
\rightangleprism, 55, **55**, 167
rotateref, 23, **23**, 83, 84, 88, 97, 130, 132, 135

S

savebeam, 86, 88, **110**, 111, 112, 154
savebeampoints, 84, **109**, 110, 134, 137, 145
SemitransMirror, 43, **45**, 146
showifcnodes, 67, 97, 127, 161, **161**
showinterfaces, 162, **162**
showoptdots, 55, **77**
sourceheight, 36, **36**
sourcesize, **36**
sourcewidth, **35**, 36
startinside, 96, 101, **111**, 112, 134
startinsidecount, 98, **100**, 101, 154
startnode, 116, **116**, 117, 143, 146
stopinside, 86, 87, 101, 110, 112, **112**, 129, 134, 136
stopinsidecount, 46, 101, **101**
stopnode, 116, **116**, 118, 143, 146
styles
- ArrowCompStyle, 34, **34**, 61
- BarCompStyle, 35, **35**
- Beam, 14, 15, 17, 40, 46, 50–53, 74, 85, 88, 93–95, 97, 101–103, **103**, 104, 106, 107, 110, 111, 129, 134–137, 140–142, 144–148, 152, 154–156, 160, 162, 169
- CrystalCaxis, 37, **37**
- CrystalLamp, **37**
- DetectorStyle, 76, **76**
- ExtendedMirror, 24, 25, 43, **44**, 45, 81, 82, 137, 145
- FdlArrow, **61**
- Fiber, 57, 61, 62, 70, 114, 115, 117, **117**, 118, 123, 138, 153, 157
- FiberIn, 123, 157
- FiberIn1, 123
- FiberIn2, 123
- FiberOut, 123, 157
- FiberOut1, 61, 123
- FiberOut2, 61, 123
- FilterStyle, 74, **74**
- IfcNodeStyle, **161**

- IfcStyle, 162, **162**
- IsolatorArrow, 60, **60**
- OptCircArrow, 62, **62**, 150
- OptComp, 25, **25**, 26, 74, 76, 145
- OptionalStyle, 25, **25**, 26
- PiezoMirror, 43, 45, **45**
- Polarization, **40**, 149
- SemitransMirror, 43, **45**, 146
- SynthStyle, 71, **71**
- VariableCoupler, **65**
- VariableStyle, **26**, 44
- Wire, 57, 70, 72, 119, **119**, 123, 138, 149, 150, 153
- WireIn, 123
- WireIn1, 123
- WireIn2, 123
- WireOut, 123
- WireOut1, 72, 123
- WireOut2, 72, 123

switchsize, **60**
switchstyle, 60, **60**
synthshape, 71, **71**, 150
synthsize, **70**
SynthStyle, 71, **71**
synthtype, **70**, 71, 150

T

thicklens, **28**
\transmissiongrating, 49, **49**, 50

U

usefiberstyle, 57, **57**
useNA, 93, **93**, 127, 148
usewirestyle, 57, **57**

V

variable, 17, 44, **44**, 64, 145, 151
VariableCoupler, **65**
VariableStyle, **26**, 44
voltage, **37**

W

\wdmcoupler, 63, **63**, 65, 66, 128, 139, 146, 150, 157
\wdmsplitter, 16, 17, 63, **63**, 65, 66, 121, 124, 139, 144, 146, 150, 153
wedgeangleleft, **41**
wedgeangleright, **41**
wedgeangles, 41, **41**
wedgeheight, **41**
wedgewidth, 41, **41**
Wire, 57, 70, 72, 119, **119**, 123, 138, 149, 150, 153
wire, 33, **121**, 122, 124, 138, 149, 153
wire*, 118
wirealign, **119**
wireangleA, **119**, 150
wireangleB, **119**, 138, 149, 150
WireIn, 123
WireIn1, 123
WireIn2, 123
WireOut, 123
WireOut1, 72, 123
WireOut2, 72, 123
wirestyle, 70, **119**

A. Revision history

This revision history is a list of changes relevant to users of this package. Changes of a more technical nature which do not affect the user interface or the behavior of the package are not included in the list. If an entry in the revision history states that a feature has been *improved* or *extended*, this indicates a modification which either does not affect the syntax and behavior of the package or is syntactically backwards compatible (such as the addition of an optional argument to an existing command). Entries stating that a feature has been *deprecated*, *modified*, *fixed*, *renamed*, or *removed* demand attention. They indicate a modification which may require changes to existing documents. The numbers on the right indicate the relevant section of this manual, more information regarding backward compatibility can be found in Sec. 12.3.

6.1 2022-02-06

Fixed bug in `\oapmirror`

6.0 2021-02-26

Modified `\wdmcoupler` to handle an arbitrary number of input nodes 5.10
Modified `\wdmsplitter` to handle an arbitrary number of output nodes . . . 5.10
Removed deprecated `lampscale`, use style `CrystalLamp`.
Removed deprecated `pollinewidth`, use style `Polarization`.
Removed deprecated `\optgrid`, use `\optgrating`.
Removed deprecated `optgridwidth`, use `gratingwidth`.
Removed deprecated `optgridheight`, use `gratingheight`.
Removed deprecated `optgriddepth`, use `gratingdepth`.
Removed deprecated `optgridcount`, use `gratingcount`.
Removed deprecated `optgridtype`, use `gratingtype`.
Removed deprecated `optgridlinewidth`, use `gratinglinewidth`.
Removed deprecated `align`, use `coupleralign`.
Removed deprecated `namingschema`.

5.2 2014-11-26

Added component `\asphericlens` 4.2
Added component `\axicon` 4.16
Added component `\optwedge` 4.15

5.1 2014-11-19

Added component `\oapmirror` 4.19
Added support for filled beams with path interfaces.

5.0 2014-10-30

Added support for arbitrary interface shapes (experimental, requires the `pst-intersect` package and `pstricks` version 2.53).
Added component `\parabolicmirror` 4.18

4.10 2014-06-04

Added value `firstcomp` to parameter `beamalign` 9.2.3
Modified `\optsource` to handle also the beam alignment 4.9

4.9 2014-06-03

Added new component `\optsource` 4.9
Improved naming of the last component name, which can now be accessed also by its number besides using `N`.
Fixed bug related to `extnodes` together with `compname`

4.8 2014-02-28

Added component `\optaom` 4.23
Added option `beammode` 9.2.4
Added option `showinterfaces` 12.1

4.7 2014-01-12

Added syntax `<`⟨*num*⟩ for accessing relative component IDs 8.1

4.6 2013-11-27

Added option `extnodes` 8.5
Added option `optdipolecomp` 10
Added option `opttripolecomp` 10

4.5 2013-07-21

Added option `usewirestyle` 5
Added line style `fade`, with options `fadeto`, `fadepoints`, `fadefunc`, and `fadefuncname` . . . . . 9.2.7 .

4.4 2013-07-14

Added component `\transmissiongrating` 4.22
Modified `innerlabel` to explicitly set `labelalign=c`
Fixed trailing spaces in `savebeampoints` (and `savebeam`).
Allow Postscript code in `angle`.

4.3a 2013-07-02

Fixed tripole alignment bugs introduced in version 4.3.
Again fixed the alignment of reflective `\optprism` 4.24
Fixed improper alignment of `\optprism` label 12.3.1

4.3 2013-06-28

Added option `raprismalign` 4.25
Fixed alignment of `\rightangleprism` and rotation of its label.

4.2 2013-06-07

Added option `startinsidecount` 9.2.4
Added option `stopinsidecount` 9.2.4
Added option `beampathcount` 9.2.3
Added option `beamnodealign` 9.2.5
Added `\oeBeamVecMedian` 8.9
Added `\oeBeamCenter` 8.8
Modified the orientation of a reflective `\optprism` 12.3.3
Fixed bug of `\optprism` and `\rightangleprism` inside an `optexp` environment.
Fixed bug of wrong `\rightangleprism` rotation in special cases.
Fixed buggy handling of multiple saved beams 9.5

4.1 2013-05-21

Added option `prismtype` 4.24

4.0 2013-04-15

Added electrical components 6
Added wire connections with `\drawwire` 9.7
Added component `\optarrowcomp` 4.7
Added component `\optbarcomp` 4.8
Improved `\optdetector` 7.3
Extended `\drawfiber` to support component ranges 9.6
Extended `couplersep` to support automatic calculation 5.10
Fixed appearance of `\optfiber` 12.3.4
Fixed `extnodealign` to affect also multipoles 8.5
Fixed `pswarning`.
Removed old, buggy beam connection code 12.3.4
Removed option `conn` 12.3.4
Removed option `connjoin` 12.3.4
Removed `\newOptexpDipoleNolabel` 12.3.4

3.6 2013-03-20

Added option `compoffset` 3.3
Added option `beampathskip` 9.2.3
Added option `innercompalign` 3.3
Fixed some bugs in the raytracing and beam code.

3.5 2013-02-22

Added option `filterangle` 7.1
Fixed wrong output fiber style in `\optcoupler`
Fixed strange Postscript error which occured with some interpreters.

3.4 2013-02-03

Fixed bugs when using `fillstyle` for some components.
Extended option `switchsize` 5.6
Modified coupler center node to be on the base line 8.3
Extended error checking for `\drawbeam` and `\drawfiber` 9.1
Fixed some bugs in the connection code.

3.3a 2012-09-18

Fixed bug which was exposed by an update of pst-node.
Fixed trailing spaces.

3.3 2012-08-17

Extended `\opttripole` and `\optdipole` 10
Modified reference angle for label of `\opttripole` 12.3.5
Added option `optdipolesize` to support two interfaces for `\optdipole` . . . 10
Added option `gratingalign` 4.21
Added option `forcebeaminside` 9.2.4
Extended option `mirrorradius` 4.17
Improved `\drawbeam` and `\drawwidebeam` to be able to use `ArrowInside` without linestyle 9.2.7
Fixed orientation of `\oenodeBeamUp` and `\oenodeBeamLow` 8.8
Fixed a bug related to `beaminside` and ambiguous components.
Fixed some trailing spaces.
Added more examples 11

3.2 2012-07-26

Added component `\glanthompson` 4.13
Added access to beam vectors with `\oeBeamVec`, `\oeBeamVecUp` and `\oeBeamVecLow` 8.9
Fixed wrong computation of node distance in `\fiberbox`
Fixed bug in `\wdmsplitter` for `coupleralign=b` and `couplertype=none`
Fixed bug when filling components placed with `position=end/start`
Added value `absolute` to parameter `labelref` 3.1
Added examples 11

3.1 2012-07-17

Added component `\fiberbox` 5.11
Extended connection macros to not required curly braces around node parenthesis any more 8.1
Modified `\fibercollimator` to have `allowbeaminside=false` by default . . 7.2
Extended fiber couplers to allow using only two nodes 5.10
Fixed a bug concerning node expressions with `\drawfiber`

3.0 2012-07-09

Modified beam connections with `\drawbeam` to support raytracing 9.2
Added wide beams with `\drawwidebeam` 9.3
Added `\drawfiber` for fiber connections 9.6
Added `optexp` environment for layering of components and connections . . 9.9
Added german documentation.
Modified naming of component nodes 8
Modified `extnode` to work with more components 8.5
Modified `angle` and `rotateref` to affect all components 3.3
Modified `endbox` to affect all components 3.2
Extended `position` by values `start` and `end` 3.2
Extended `abspos` by values `start` and `end` 3.2
Removed deprecated lens code which used `lenswidth` and `lensheight` for construction 4.1
Added option `platesize` 4.4
Added option `phwidth` 4.5
Modified option `caxislength` 4.10
Deprecated option `lampscale` 4.10
Added style `CrystalCaxis` 4.10
Added style `CrystalLamp` 4.10
Added option `optboxsize` 4.6
Extended option `detsize` 7.3
Added style `DetectorStyle` 7.3
Removed deprecated `\detector`, use `\optdetector` 7.3
Extended option `doveprismsize` 4.12
Added style `Polarization` 4.14
Deprecated option `pollinewidth` 4.14
Removed option `polwidth` 4.14
Removed option `pol` 4.14
Added style `VariableStyle` 4.17
Added mirror type `semitrans` 4.17
Renamed `\optgrid` to `\optgrating` 4.21
Renamed `optgridwidth` to `gratingwidth` 4.21
Renamed `optgridheight` to `gratingheight` 4.21
Renamed `optgriddepth` to `gratingdepth` 4.21
Renamed `optgridcount` to `gratingcount` 4.21
Renamed `optgridtype` to `gratingtype` 4.21
Renamed `optgridlinewidth` to `gratinglinewidth` 4.21

Added option `prismalign` 4.24
Modified faulty alignment of `\rightangleprism` 4.25
Extended option `optampsize` 5.2
Extended option `optmzmsize` 5.3
Added option `polcontroltype` 5.4
Extended option `isolatorsize` 5.5
Added style `IsolatorArrow` 5.5
Extended option `fdlsize` 5.7
Added style `FdlArrow` 5.7
Extended option `fiberpolsize` 5.8
Added component `\optcirculator` 5.9
Extended option `couplersize` 5.10
Extended option `couplertype` 5.10
Renamed option `align` to `coupleralign` 5.10
Added style `VariableCoupler` 5.10
Added style `FilterStyle` 7.1
Extended option `fibercolsize` 7.2
Removed deprecated option `labelrelative` 3.1
Removed deprecated option `iwidth` 4.5
Removed deprecated option `owidth` 4.5
Removed deprecated option `bswidth` 4.20
Deprecated `\newOptexpDipoleNolabel`, use `\newOptexpDipole` 10.2
Deprecated option `refractiveindex` 9.2.2
Deprecated option `conn` 9.2
Extended option `fiber` 9.8

2.1 2009-11-05

Added component `\optfiberpolarizer` 5.8
Added option `compshift` 3.3
Added option `label` 3.1
Added option `connjoin`
Added options `addtoBeam` and `newBeam` 9.2
Added style `OptComp` and related options `addtoOptComp` and `newOptComp` . . 3.5
Added option `bsstyle` 4.20
Extended `\fibercollimator` to use up to four reference nodes 7.2
Improved `thicklens` to work also with plain lenses 4.1
Use pst-doc class for the documentation.

2.0 2008-07-27

Added fiber-optical components 5
Added component `\optdiode` 4.11
Added component `\pentaprism` 4.26
Added component `\rightangleprism` 4.25
Added component `\doveprism` 4.12
Added component `\optprism` 4.24
Added `\drawbeam` 9.2
Added component connections (options `fiber`, `conn` and `beam`) 9
Added option `compname` 8.1
Added option `extnode` 8.5
Renamed `\detector` to `\optdetector` 7.3

1.2 2008-06-17

Modified lens design to use interface curvatures 4.1
Added options `lensradiusleft` and `lensradiusright` 4.1
Added option `thicklens` 4.1
Added option `lenstype` 4.1
Added option `mirrorradius` (curved mirrors) 4.17
Added option `optgridtype` (binary gratings) 4.21
Added `\newOptexpDipole` 10.2
Added `\newOptexpDipoleNolabel` 10.2
Added `\newOptexpTripole` 10.2
Added `\newOptexpFiberDipole` 10.2
General improvements of TEX and Postscript code

1.1 2007-09-06

Improved labeling features 3.1
Added parameter `labelref` 3.1
Replaced `labelrelative` by `labelref=relative` 3.1
Renamed `\polarisation` to `\polarization` 4.14
Renamed `polwidth` to `polsize` 4.14
Renamed `pol` to `poltype` 4.14
Renamed `bswidth` to `bssize` 4.20
Renamed `iwidth` to `innerheight` 4.5
Renamed `owidth` to `outerheight` 4.5
Added support for fillstyle for all components

1.0 2007-07-18

First CTAN version